

Building a Semantic Layer for Natural-Language Querying

Complete bundle — implementation guide, design spec, published write-ups, and all nine metadata artifacts verbatim.

All figures in this document are measured against a warehouse built from the committed CSVs at a frozen as-of date of **2026-08-07**, so every number here is reproducible by running `python verify_acceptance.py`.

Contents

- **Part 1** — The Implementation Guide (`README.md`)
- **Part 2** — The Design Spec and Acceptance Criteria (`docs/design.md`)
- **Part 3** — The LinkedIn Post (`content/linkedin-post.md`)
- **Part 4** — The LinkedIn Article (`content/linkedin-article.md`)
- **Part 5** — The Medium Post, With Complete Executable Code (`content/medium-post.md`)
- **Part 6** — Appendix: the nine metadata artifacts, in full

Part 1 — The Implementation Guide

Source: [README.md](#)

A step-by-step implementation guide, with a runnable reference implementation.

This project answers one question: *what has to be true before you can trust a natural-language answer to a business question?*

It walks eight build phases (P0–P7). Each phase emits one real metadata artifact, and each artifact closes one specific way natural-language querying fails. Nothing here is illustrative — every file in `metadata/` is parsed by the code, every figure in this guide is computed from the committed CSVs, and every claim is asserted by a test.

The domain is SAP **Plan-to-Deliver**: manufacturing through delivery. Manufacturing phases are what make "why" answerable, which is the difference between a layer that reports and a layer that explains.

The architectural rule

Everything below follows from one decision, so it is worth stating before anything else:

The model picks which governed metric a question means. It never writes SQL and never computes arithmetic.

A model asked to write SQL can produce a query that runs, returns a plausible number, and is quietly wrong — the wrong date column, a join that fans out, a cohort nobody agreed to. A model asked only to *select from a governed list* can be wrong in exactly one way: it picks the wrong item. And that failure is visible, because a validator checks the selection against the semantic model **before any SQL exists**.

So the pipeline is: natural language → a validated `QueryIntent` → a deterministic compiler. The metadata artifacts are what make the middle step checkable.

Contents

1. The problem: the wrong answer is the believable one
2. Quickstart
3. The domain: Plan-to-Deliver
4. The eight build phases
5. How metadata answers a question
6. What this does not do
7. Appendix

1. The problem

Two questions an executive actually asks:

Q1. "On-time delivery for India warehouses last month" **Q2.** "Why did on-time delivery drop for India warehouses last month?"

Point a competent text-to-SQL system at a clean SAP-derived warehouse and ask Q1. It joins the order table to the delivery table — reasonable, the delivery table is where delivery dates live — and computes:

54 on-time delivery rows / 61 delivery rows = 88.52%

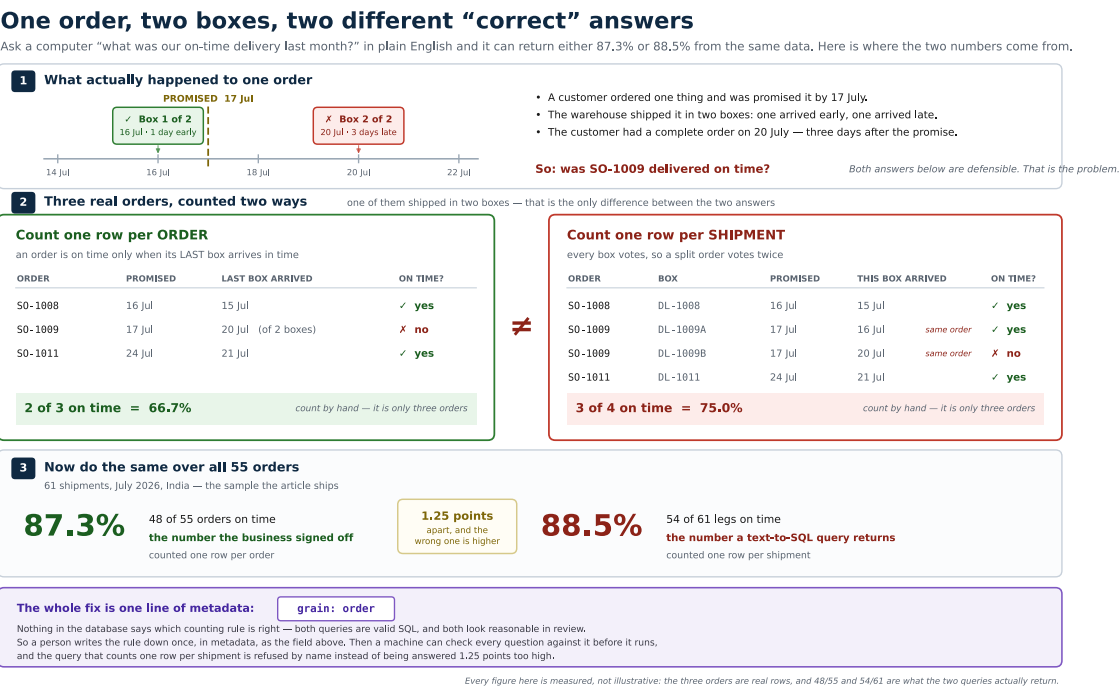
The correct answer is **87.3%** (48 of 55 orders). The error is **1.25 percentage points**, and it is in the *flattering* direction. That combination is the whole problem. The wrong answer is not absurd. It does not trigger anyone's disbelief. It is close enough to pass review, close enough to put in a board deck, and *higher* than the truth — so nobody goes looking. It survives precisely because it is plausible.

Where the 1.25pp comes from

Six of the 55 eligible India/July orders shipped in **two legs** — partial shipment, entirely normal SAP behaviour. Measured at delivery-leg grain the row count becomes 61:

	On time	Denominator	OTD
Grain-correct (order grain)	48	55	87.27%
Naive (delivery grain)	54	61	88.52%

S0-1009 is the row that shows why it matters: leg A arrived on time, leg B arrived late. At delivery grain that is one success and one failure. At order grain it is a single **late** order — the order is not delivered until the last leg lands. The naive count credits it with a success it did not earn.



The same disagreement without the vocabulary: one order, two boxes, two defensible counting rules, and three orders counted by hand. (Also available as docs/diagrams/grain_two_ways.png . Generated by docs/diagrams/render_grain.py ; every figure in it — including the hand count — is recomputed from the built warehouse by tests/test_diagram.py , so the simplified version cannot drift from the real one.)

Worse, the bias direction depends on the data. A fully on-time split order pulls the naive figure up; a fully late one pulls it down. So the error is not even a consistent offset you could learn to correct for.

The same trap appears in first-pass yield, via exclusions rather than grain. Write the query without the governed exclusions and you get 52/55 = **94.55%** instead of the correct 51/54 = **94.4%**. Two separate slips, each worth one row:

- S0-1004 's inspection lot QL-4004 was reworked and eventually accepted. Reading the final decision instead of the first counts it as a first-pass success, taking the numerator from 51 to 52.

- **P0-3007** was cancelled and never inspected, so dropping that exclusion leaves it in the denominator: 54 to 55.

Both errors push the same way. Again overstating, again by an amount small enough to look like rounding.

Grain mistakes do not announce themselves with absurd values. They move the number by roughly the amount people argue about in review meetings. Eyeballing the output will not catch this. A validator that checks declared grain will.

And Q2 — *why* — is not answerable at all without process metadata. There is no column called "reason". The cause has to be *derived* from manufacturing phase timestamps, which is what Phase 4 exists to make possible.

2. Quickstart (no API key needed)

```
pip install -r requirements.txt
python src/build_warehouse.py
python src/ask.py --offline Q1
python src/ask.py --offline Q2
python -m pytest
```

No **ANTHROPIC_API_KEY** is required for any of this. The resolver — the one component that calls a model — is bypassed in offline mode by hand-authored **golden intents** in **tests/golden_intents.json**. Every other stage runs for real: real validation, real SQL compilation, real DuckDB execution, real data-quality rules, real provenance.

That is deliberate. A guide whose examples you cannot reproduce is a guide you have to take on faith.

python src/ask.py --offline Q1 prints:

```
QUESTION "On-time delivery for India warehouses last month"

ANSWER  On-Time Delivery % is 87.3% -- 48 of 55 eligible orders delivered on or before the promised date

metric      on_time_delivery_pct -- Share of eligible customer orders delivered on or before the
promised delivery date. An order counts as delivered only when its final leg arrives.
arithmetic   48 / 55 = 87.2727%
grain        order (one row per order)
window       July 2026 (2026-07-01 to 2026-07-31) on promised_delivery_date
filters      region = IN
exclusions   cancelled_orders, not_yet_due
lineage      SAP SD -> SAP EWM -> SAP TM -> fact_order_delivery
trust        TRUSTED
gates        7/7 passed: dimensions_declared, exclusions_applied, filters_bound, grain_matches,
metric_approved, no_fanout, time_window_bounded
```

Every line after **ANSWER** is provenance, and all of it is derived rather than asserted: the arithmetic is the numerator and denominator the SQL actually returned, the exclusions are the ones the compiler actually applied, and the trust badge is the outcome of the data-quality rules that actually ran.

--offline Q2 adds the diagnostic half:

BREAKDOWN

Quality inspection	4
Transportation	2
Production execution	1
unattributed	0

Largest contributor: Quality inspection (4 of 7 late orders).

COMPARISON

prior window	June 2026 (2026-06-01 to 2026-06-30) on promised_delivery_date
prior value	96.2% (25 / 26)
change	down 8.9pp (96.2% -> 87.3%)

Other flags: `--list` shows the stored questions, `--context` prints the exact metadata slice the resolver is given, and `--sql` prints the compiled SQL alongside the answer. With credentials configured, pass a question in quotes instead of `--offline` and the resolver maps it for you.

Or without cloning anything

The same architecture also ships as a single file with the data and metadata embedded, for reading in one sitting or pasting into a terminal:

```
pip install duckdb pyyaml
python standalone/semantic_layer_demo.py
```

It builds its own in-memory warehouse and prints the governed answer, the wrong answer the same rows yield, that wrong query refused by name, the per-phase diagnostic, and four more classes of refused question. One metric instead of three and no separate artifact files — otherwise the same seven gates and the same compiler. `tests/test_standalone.py` checks its numbers against this project's, so the short version cannot quietly disagree with the long one.

3. The domain: Plan-to-Deliver

An order is late because *something in the chain took longer than it was supposed to*. To attribute that, you need the chain itself as metadata: which phases exist, which system records each one, which timestamp marks it, and how long it is supposed to take.

Twelve phases, from `metadata/04_process_model.yml`:

#	Phase	Module	Business event	Timestamp	Std. days
1	Demand & order capture	SAP SD	Sales order created	VBAK-ERDAT	0
2	Requirements planning	SAP PP (MRP)	Planned order created	PLAF-PSTTR	1
3	Production order release	SAP PP	Production order released	AFKO-GSTRP	1
4	Material staging & issue	SAP MM	Goods issue to production	MSEG-BUDAT	1
5	Production execution	SAP PP	Operation confirmed	AFRU-BUDAT	3
6	Quality inspection	SAP QM	Usage decision	QALS-VDATUM	1
7	Goods receipt to stock	SAP MM / EWM	GR to unrestricted stock	MKPF-BUDAT	1
8	Picking & packing	SAP EWM	Picking completed	picking_date	1
9	Goods issue / dispatch	SAP LE-SHP	Actual goods issue	LIKP-WADAT_IST	0
10	Transportation	SAP TM	Shipment dispatched	VTTK-DPTBG	3
11	Delivery confirmation	SAP TM / LE	POD received	final_delivery_date	0
12	Billing	SAP SD	Invoice created	VBRK-FKDAT	1

Phases 1–11 sum to **12 days**: the delivery promise horizon. Billing follows delivery and cannot consume promise slack, so it is excluded from the slack computation.

The attribution rule

For each order and each phase: $\text{variance_days} = \text{actual_duration} - \text{standard_duration}$. The attributed cause is $\text{argmax}(\text{variance_days})$, ties broken to the earliest phase.

Two conventions in that sentence are load-bearing:

Spans are end-anchored. A span belongs to the phase it *ends at*. Naming a variance after the phase it starts from puts every standard one phase out of step — which is exactly the defect that shipped in an early version of the fact table and had to be corrected.

Spans are contiguous. Each starts where the previous ended, so every day between `order_date` and `final_delivery_date` is owned by exactly one phase, and $\text{sum}(\text{variances}) = \text{elapsed} - 12$ for every delivered order. An unowned span would let a real delay produce no variance anywhere — the pipeline would return "no cause found" for an order that was demonstrably late. The warehouse build asserts this reconciliation directly.

Nine of the twelve phases are attributable (`phase_seq` 2–10). Phase 1 is the chain's start point rather than a span, phase 11 is a zero-day POD event folded into the transportation span, and phase 12 comes after delivery. Declaring the other three as possible values would let a filter pass the gates and return **zero rows dressed as an answer**.

Here is the whole Q2 answer, hand-checkable, straight from the fact table — the seven late India/July orders and their variances:

Order	Promised	Delivered	var_confirm	var_qm	var_shipment	Attributed to
S0-1002	2026-07-13	2026-07-16	0	+4	–1	quality_inspection
S0-1003	2026-07-14	2026-07-19	0	0	+5	transportation
S0-1004	2026-07-14	2026-07-18	0	+5	–1	quality_inspection
S0-1005	2026-07-15	2026-07-18	+4	0	–1	production_execution
S0-1006	2026-07-15	2026-07-19	0	+5	–1	quality_inspection
S0-1009	2026-07-17	2026-07-20	0	0	+3	transportation
S0-1010	2026-07-18	2026-07-22	0	+5	–1	quality_inspection

Four to quality inspection, two to transportation, one to production execution. Notice that transportation ran *under* standard on five of the seven — the layer discriminates rather than always blaming the last visible step, which is the failure mode of attribution built on "whatever was late at the end".

The dataset

132 orders: 20 hand-authored in `data/seed_core.csv` (every defect, every late order, the full manufacturing event chain for each) plus 112 well-behaved orders from `data/generate.py`, seeded at `random.Random(20260807)` with no wall-clock and no network, so re-running it leaves the CSVs byte-identical.

Segment	Orders	Cancelled	Eligible	On time	Late	OTD
IN — July 2026 (Q1, Q2)	56	1	55	48	7	87.27%
IN — June 2026 (comparison)	27	1	26	25	1	96.15%
IN — August 2026	4	0	0 (not yet due)	—	—	n/a
US — July 2026	30	2	28	26	2	92.86%
US — June 2026	15	0	15	14	1	93.33%

The size is not arbitrary: 87.3% is impossible with six orders — each order would move the number 16.7pp. Percentages have to be believable to be worth reasoning about.

Eight deliberate defects are seeded, each teaching exactly one lesson: a cancelled order (`SO-1007`), an in-transit null promised into the future (`SO-1013`), a rework loop (`SO-1004`), six split deliveries, a re-promised date (`SO-1011`), a make-to-stock order with no production order (`SO-1015`), QM-dominated delays, and delays attributed elsewhere.

4. The eight build phases

Every phase below follows the same shape: **INPUT** → **WORK** → **OUTPUT** → **GATE** → **NL FAILURE CLOSED**. The gate is how you know the phase is done. The last line is the point of doing it at all.

Phase 0 — Scope and KPI contract

INPUT: the questions the business actually asks. **WORK:** agree the metric definitions, owners, stewards, and what "done" means, in writing, before touching data. **OUTPUT:** `metadata/00_kpi_contract.md`.

Metric Definition Grain Owner Steward
On-Time Delivery % Share of eligible customer orders delivered on or before the promised delivery date `order` VP Supply Chain Supply Chain Analytics Lead

The contract also settles what "last month" means, which is not obvious. The governed reading is **promised_delivery_date within the month** — the promise is what the customer was told and what the metric is judged against. Reading it as the *delivery* date instead selects a different cohort and returns a different number, and both readings are defensible until someone writes one of them down.

GATE: every metric has a named owner, a grain, and an exclusion list; the two demonstrator questions have expected answers with arithmetic.

NL FAILURE CLOSED: building a semantic layer with no acceptance test for "done". Without this file, "on-time delivery" means whatever the query author assumed, and two people asking the same question get two different numbers — with neither able to show the other is wrong.

Phase 1 — Technical metadata harvest

INPUT: the source system's data dictionary (SAP DDIC via SE11/SE16). **WORK:** extract tables, fields, data elements, domains, keys, nullability, source module. **OUTPUT:** `metadata/01_technical_metadata.json` — 20 fields across VBAK, VBEP, AFKO, AFRU, QALS, MSEG, MKPF, LIKP, VTTK, VBRK.

```
{
  "table": "LIKP", "field": "WADAT_IST", "data_element": "WADAT_IST",
  "data_type": "DATS", "length": 8, "is_nullable": true,
  "source_system": "SAP LE-SHP",
  "description": "Actual Goods Movement Date",
  "target_column": "goods_issue_date"
}
```

GATE: every field carries a `target_column`, so the mapping to the warehouse is declared rather than inferred from name similarity.

NL FAILURE CLOSED: there is nothing to bind business terms to. "When did it ship" cannot resolve to `LIKP-WADAT_IST` if nothing records that `WADAT_IST` is the actual goods-movement date and that it is nullable. A model guessing among `WADAT`, `WADAT_IST` and `LFDAT` will guess plausibly and wrongly.

Phase 2 — Standardize and declare grain

INPUT: Phase 1's field inventory and the raw extracts. **WORK:** canonical names, real types, conformed dimensions — and declare the grain of every table. **OUTPUT:** `sql/02_staging_model.sql` plus `metadata/02_standardized_metadata.json`.

```
{
  "table": "fact_order_delivery",
  "grain": "order",
  "primary_key": "order_id",
  "row_count": 132, "unique_keys": 132,
  "why_this_grain_matters": "6 orders ship in two legs. Measuring OTD at
    delivery-leg grain double-counts split shipments and produces 88.52%
    instead of 87.27%."
}
```

The grain declaration is the single highest-value line of metadata in this project. It is what turns \$1's 88.52% from an opinion into a rule violation.

GATE: `count(*) = count(DISTINCT primary_key)` for each declared grain — 132 orders over 132 distinct `order_id`, 131 production orders over 131 distinct `production_order_id`. A test asserts it against the built warehouse, so any future fan-out fails the build rather than the board deck.

NL FAILURE CLOSED: `VBELN` vs `vbeln` vs `sales_doc` — the model cannot tell they are one thing. And with no declared grain, no downstream component can tell that joining to delivery legs is an error rather than a choice.

Phase 3 — Business glossary and column bindings

INPUT: Phase 2's canonical columns and the vocabulary people actually use. **WORK:** define each business term, list its synonyms and allowed values, and bind each term to a physical column. **OUTPUT:** `metadata/03_glossary.yml` and `metadata/03_column_bindings.yml`.


```
- term: On-Time Delivery %
  synonyms: [OTD, on time delivery, delivery performance, delivery reliability,
             promise adherence, OTIF, on time in full]
  owner: VP Supply Chain
  calculation: (delivered_on_time / eligible_orders) * 100
```

```
- term: Region
  table: fact_order_delivery
  column: region
  scope: business
```

The `scope` field is a governance decision, not bookkeeping. `business`-scoped bindings are named in the glossary and offered to the resolver; `internal` ones — variance columns, flags, raw phase timestamps — are deliberately withheld. Exposing 27 more terms widens the guess space without answering a question anyone asks. A test enforces that every business-scoped binding has a glossary entry, so this stays a decision rather than drift.

GATE: every bound column exists in the warehouse (checked against `PRAGMA table_info()`), and every business-scoped term is defined in the glossary.

NL FAILURE CLOSED: "OTD for India last month" is unresolvable. No metric in `05_semantic_model.yml` carries a `synonyms` key — "OTD" is a dead end without the glossary. And nothing can validate that "India" is a legal region value rather than a typo that will silently return zero rows.

Phase 4 — Business process and event lineage

INPUT: the process as the business runs it, and Phase 1's timestamp fields. **WORK:** model the twelve phases, bind each to its system, event and timestamp, set standard durations, and define the variance attribution rule. **OUTPUT:**

`metadata/04_process_model.yml`.

```
- phase_seq: 6
  phase_key: quality_inspection
  phase_name: Quality inspection
  sap_module: SAP QM
  business_event: Usage decision
  timestamp_field: qm_final_decision_date
  standard_duration_days: 1
```

```
- column: var_qm
  phase_key: quality_inspection
  span: prod_confirm_date -> qm_final_decision_date
  standard_duration_days: 1
  note: >
    Uses the FINAL decision, because attribution asks how long inspection
    actually took including any rework re-inspection. first_pass_yield_pct is
    the opposite case and must use the FIRST decision.
```

That note is the subtlety worth stealing. The *same* inspection process feeds two metrics that need **opposite** readings of it: attribution asks how long inspection took (final decision), first-pass yield asks whether it passed the first time (first decision). One timestamp column cannot serve both, and picking either one silently breaks the other metric.

GATE: the nine variance standards sum to 12 — the same 12 that `available_slack_days` subtracts — and `sum(variances) = elapsed - 12` for every delivered order. Standard durations are compared row by row against `dim_process_phase` in the SQL, so the YAML and the warehouse cannot disagree.

NL FAILURE CLOSED: the layer answers *what* and never *why*. Q2 is not a harder version of Q1; without process metadata it is an unanswerable question, because "cause" is not a column anywhere in SAP. This file is what turns it into arithmetic over timestamps.

Phase 5 — Semantic model and metrics

INPUT: everything above. **WORK:** declare entities, joins **with cardinality**, dimensions with allowed values, and metrics with grain, expression, exclusions and approval state. **OUTPUT:** `metadata/05_semantic_model.yml` — the single definition of every metric, read by both BI and NL querying so the number cannot drift between a dashboard and a chat answer.

```
- name: on_time_delivery_pct
  label: On-Time Delivery %
  entity: fact_order_delivery
  grain: order
  time_dimension: promised_delivery_date
  expression:
    type: ratio_percent
    numerator: "sum(CASE WHEN is_on_time THEN 1 ELSE 0 END)"
    denominator: "sum(CASE WHEN is_eligible THEN 1 ELSE 0 END)"
  exclusions:
    - key: cancelled_orders
      predicate: "order_status <> 'CANC'"
      rationale: A cancelled order was never due; including it inflates the denominator.
    - key: not_yet_due
      predicate: "promised_delivery_date <= DATE '__AS_OF_DATE__'"
      rationale: >
        An order promised after the as-of date is not late, it is pending.
  approval_state: approved
```

Two things here do more work than they look like they do.

Join cardinality is declared, not discovered.

```
joins:
- to: fact_production_order
  cardinality: many_to_one      # PERMITTED -- 131 production orders, 131 distinct orders
- to: raw_deliveries
  cardinality: one_to_many      # REFUSED -- 6 orders ship in two legs
  note: FAN-OUT HAZARD ... yields 88.52% instead of 87.27%.
```

Both edges are needed. `many_to_one` is required — `fact_production_order` carries no region and no promised date, so both manufacturing metrics are unfilterable without it. `one_to_many` is the hazard. A gate that refused all joins would be useless; a gate that refused none would produce §1's wrong answer. The distinction has to be in the metadata because it cannot be inferred from the query.

approval_state is what makes refusal possible. The contract defines four metrics; three are approved and implemented. The fourth, `production_cycle_time_days`, is **deferred**: it is a plain average over columns the other three already materialize, so it teaches nothing new for the cost of building it. Asking for it produces a refusal, not an approximation — which is the correct behaviour for a metric with an agreed definition and no implementation.

GATE: every metric has a grain matching its entity, an explicit exclusion list, a named owner, and an approval state. Every dimension resolves to a real column. The compiler asserts 87.27% / 88.52% / 94.4% / 94.55% / 92.6% from the built warehouse, so the narrative in §1 is under test rather than retold.

NL FAILURE CLOSED: the model invents arithmetic. Without a governed expression it writes its own `SUM / COUNT` — and gets 88.52% by joining to delivery legs, or 94.55% by counting rework as first-pass success. With one, the arithmetic is fixed

and the model's only job is choosing which fixed arithmetic to run.

Phase 6 — Quality contract

INPUT: the metrics and the columns they read. **WORK:** write rules bound to **specific metric inputs**, each with a threshold and a severity. **OUTPUT:** `metadata/06_dq_rules.yml` — six rules across six DQ dimensions.

Rule	Dimension	Severity	Bound to	Catches
<code>dq_completeness_delivery_date</code>	completeness	degrading	OTD numerator	missing delivery dates among past-due orders
<code>dq_validity_enum_values</code>	validity	blocking	all three metrics	status/plant outside declared allowed values
<code>dq_uniqueness_order_grain</code>	uniqueness	blocking	OTD	the split-delivery fan-out — the 88.52% defect
<code>dq_timeliness_warehouse_freshness</code>	timeliness	degrading	all three metrics	a stale mart answering as if current
<code>dq_consistency_phase_ordering</code>	consistency	blocking	OTD, adherence	phase timestamps out of order, invalidating attribution
<code>dq_accuracy_otd_certification</code>	accuracy	blocking	OTD	recomputed OTD drifting >0.1pp from the certified figure

Binding rules to metric inputs rather than to tables is what makes the result usable. A failing rule degrades **a specific answer**, not a dashboard-wide red light that everyone learns to ignore.

Each answer carries a badge: `TRUSTED`, `DEGRADED (rule X failing)`, or `BLOCKED`. The badge gates *presentation* — a `BLOCKED` input returns a refusal **instead of** the number, not the number with a warning beside it. A caveat next to a figure loses to the figure every time; someone will screenshot the number and drop the caveat.

GATE: each of the eight seeded defects trips its intended rule, and each rule stays quiet on the others. Verified by corrupting a scratch copy of the warehouse inside a rolled-back transaction, one defect at a time, and asserting the whole verdict set. There is exactly one expected cross-fire: a duplicated order row trips `dq_uniqueness_order_grain` and `dq_accuracy_otd_certification`, because it genuinely changes the published number. That is the certification working as a backstop, not leakage.

NL FAILURE CLOSED: confidently wrong answers from incomplete inputs. A pipeline with perfect metadata and stale data still produces a number, and without a trust badge the answer looks identical to a good one.

Phase 7 — Publish, resolve, and defend

INPUT: everything above. **WORK:** publish the metric as a catalog asset with lineage and governance fields, and wire the query path. **OUTPUT:** `metadata/07_catalog_asset.json` plus the runnable pipeline.

```
{
  "asset_id": "semantic.metric.on_time_delivery_pct",
  "asset_type": "certified_metric",
  "certification_state": "certified",
  "data_subject_category": "business_customer_contact",
  "personal_data_category": "limited_personal_data",
  "processing_purpose": "Supply chain performance measurement",
  "legal_basis": "legitimate_interest_contract_performance",
  "retention_period": "P7Y",
  "lineage": ["SAP SD (VBAK/VBAP) - sales order created", "..."]
}
```

These governance fields make personal data discoverable and support India DPDP readiness work. They do not enforce anything. Enforcement lives in access control, consent management, retention automation, and masking at the platform layer. A catalog entry claiming `retention_period: P7Y` does not delete anything on schedule, and treating it as if it does is worse than having no field at all — it converts an open question into a false answer.

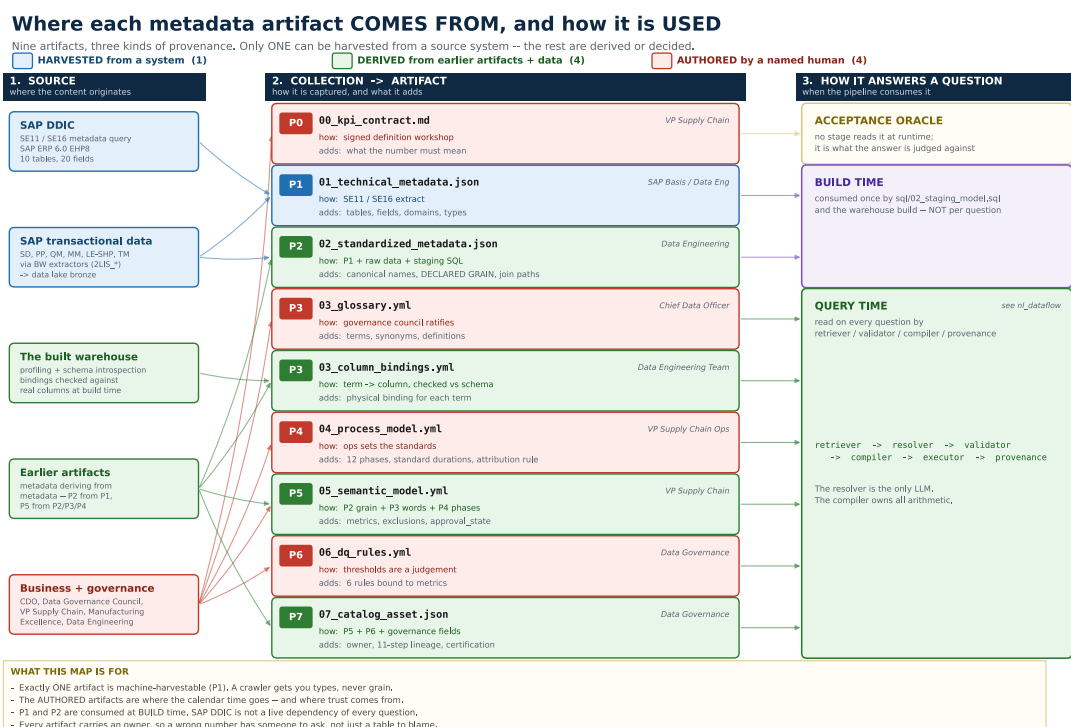
The `personal_data_fields` block is where that honesty gets specific: `customer_id` is classified as a pseudonymous identifier with the note that it "resolves to a legal entity in SAP KNA1; for named-individual sole proprietors it is personal data under DPDP." That is a real edge case, recorded rather than smoothed over.

GATE: the published asset's `definition_sql` matches the semantic model's expression, its lineage chain reaches from SAP tables to the consumer, and its `quality_contract` names the rules that actually exist in `06_dq_rules.yml`.

NL FAILURE CLOSED: an answer with no provenance is an answer you cannot defend in a review. "The system said 87.3%" is not a claim anyone can act on. "87.3% = 48/55 eligible India orders promised in July, at order grain, excluding cancelled and not-yet-due, from `fact_order_delivery` via SAP SD → EWM → TM, all seven gates passed, six DQ rules green" is.

Where each of the nine artifacts actually comes from

Eight phases produced nine files. The question that always lands next, usually from whoever is being asked to fund the work, is "can't we just harvest this from SAP?"



(Also available as `docs/diagrams/metadata_sources.png`. Generated from a declared graph by `docs/diagrams/render_sources.py`; every arrow carries a `probe` string that must occur in the artifact it points at, so a provenance claim the file itself does not support fails the suite instead of shipping.)

The answer is **one of the nine**. `01_technical_metadata.json` comes out of the SAP data dictionary via SE11 / SE16, repeatably and cheaply. The other eight split into:

- **DERIVED (4)** — computed from earlier artifacts plus the data. `02` from `01` plus the raw rows plus staging SQL; `03_column_bindings.yml` from the glossary checked against real warehouse columns; `05` from P2's grain, P3's words

and P4's phases; 07 from 05 + 06 plus governance fields. Cheap to keep current, because every input is itself a versioned artifact.

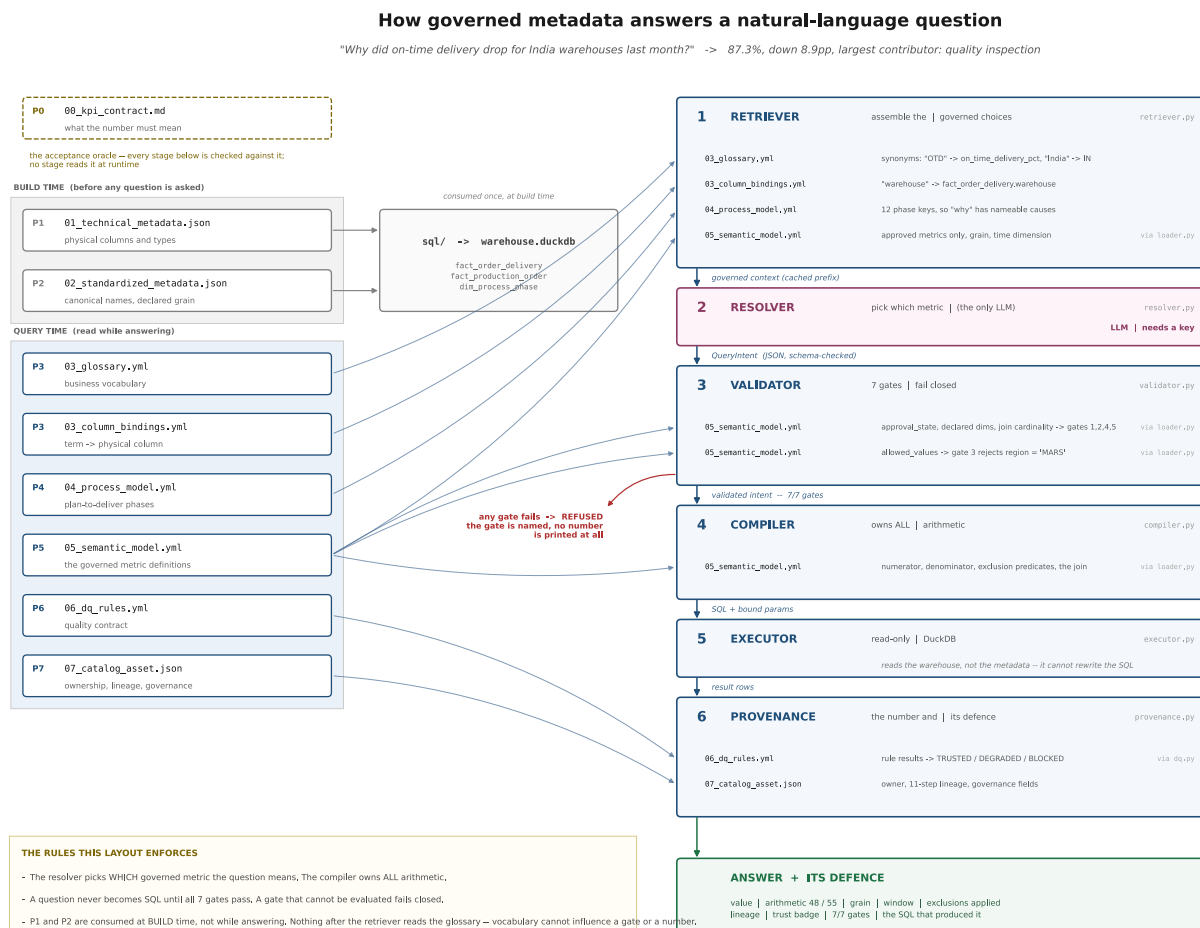
- **AUTHORED (4)** — decided by a person with a name. What "on-time" means (P0), which synonyms are sanctioned (P3 glossary), what a standard QM usage decision takes (P4), what completeness threshold is tolerable (P6).

That split is the planning fact worth carrying away, because it predicts where the calendar goes. A crawler gets you tables, fields, domains and types. It cannot get you grain, exclusions, an approved definition, or a standard duration — and those are exactly the fields that decide whether the answer is *right* rather than merely well-formed. The 88.5% error lives in a field no harvester produces.

The right-hand band of the figure carries the other half: **when** each artifact is consumed. P0 is never read at runtime at all — it is the oracle the answer is judged against. P1 and P2 are consumed once, at build time. Only P3–P7 are read per question, which is the handoff into the next section.

5. How metadata answers a question

Six stages. Exactly one calls a model.



(Also available as docs/diagrams/nl_dataflow.png. The figure is generated from a declared graph by docs/diagrams/render_dataflow.py; --check fails if it drifts from the code, and its edges are tested against the module sources so the picture cannot claim a read the code does not make.)

Build time vs query time

The distinction the diagram exists to make: **P1 and P2 artifacts are consumed at build time**, by sql/02_staging_model.sql and the warehouse build. They are not inputs to a question. **P3–P7 artifacts are consumed at query time**, per question.

So the SAP data dictionary is not a live dependency of every query — which is both the truth and considerably less alarming than the alternative reading.

Which stage reads which artifact

Stage	Reads	What it supplies
1. Retriever	<code>03_glossary.yml</code>	synonyms — "OTD", "OTIF" → the metric
	<code>03_column_bindings.yml</code>	business terms → physical columns
	<code>04_process_model.yml</code>	the twelve phases, marked attributable or not
	<code>05_semantic_model.yml</code> (via <code>loader.py</code>)	approved metrics, dimensions, allowed values
2. Resolver (the only LLM)	the retriever's context	→ a <code>QueryIntent</code> , nothing else
3. Validator	the <code>SemanticModel</code> only	all seven gates
4. Compiler	the <code>SemanticModel</code> only	numerator, denominator, exclusion predicates, joins
5. Executor	the warehouse	numerator and denominator
6. Provenance	<code>06_dq_rules.yml</code> , <code>07_catalog_asset.json</code>	trust badge, lineage, certification

Two of those rows are worth reading twice. **The validator and the compiler never open a file.** Both receive a `SemanticModel` that `loader.py` parsed. That is not an implementation detail — a stage that cannot open a file cannot quietly read a different one, and in particular the compiler has no access to the glossary at all. Vocabulary is resolved *before* the gates, so which synonym a user typed can never influence arithmetic.

The retriever: governance by omission

The retriever assembles the metadata slice the resolver may choose from, and that slice *is* the model's universe of possible answers. Which makes it a governance surface:

- Omit a filter column and "India warehouses" becomes unanswerable no matter how capable the model is.
- Include an unapproved metric and you have handed it a way to be confidently wrong — serving an unsigned-off number with the same confidence as a certified one. The context filters on `approval_state == "approved"` for exactly this reason.
- Offer a dimension the gates always refuse and you have offered a refusal. `delivery_id` is *reachable* from the order fact, but only through the `one_to_many` edge, so `no_fanout` rejects it every time. The context advertises only what will survive validation, and it imports the fan-out judgement from the validator rather than reimplementing it — two copies of that rule would eventually disagree, and the disagreement would surface as an unexplainable refusal.

Run `python src/ask.py --offline Q1 --context` to see the exact string.

The seven gates

All seven run on every intent, **before** compilation, so a refused intent never becomes SQL:

Gate	Refuses
<code>metric_approved</code>	a metric that is undefined or not approved
<code>dimensions_declared</code>	a group-by that is not a declared dimension of the metric's entity
<code>filters_bound</code>	a filter column that does not bind, or a value outside <code>allowed_values</code>
<code>grain_matches</code>	a query whose grain differs from the metric's certified grain
<code>no_fanout</code>	a path reaching the metric only through a <code>one_to_many</code> join
<code>time_window_bounded</code>	an open-ended window
<code>exclusions_applied</code>	a compiled query missing the metric's declared exclusions

A gate that cannot be evaluated **fails closed**. If the metric is not in the model, `grain_matches` does not pass by default for want of something to compare against — it reports that it could not check, which is a refusal.

Here is a real refusal:

```
REFUSED on_time_delivery_pct: the intent did not pass validation.

gates failed no_fanout
no_fanout      Metric 'on_time_delivery_pct' at grain 'order' would fan out:
                dimension 'delivery_id' lives on 'raw_deliveries', reachable from
                'fact_order_delivery' only through a one_to_many join, which
                duplicates rows before aggregation.

No number is shown. A value that failed a gate is not a caveated answer,
it is an unverified one.
```

That is §1's 88.52% being refused *by name*, before any SQL exists. The message names the offending dimension, the entity it lives on, and the join that would duplicate rows — enough for the user to ask a different question, and enough for an engineer to confirm the refusal was correct.

Grouped answers: sum, then divide

One arithmetic rule the compiler enforces and no amount of prompting would: a grouped answer's total is computed from **summed numerators and denominators**, never from an average of the per-group rates. Averaging rates across groups of unequal size gives a different number — Simpson's paradox — and it is wrong in a direction that depends on the group sizes. Q2's `down 8.9pp` comes from 48/55 against 25/26, not from averaging seven phase-level rates.

6. What this does not do

Stated plainly, because a guide that oversells its own scope is the thing this project is arguing against.

Governance metadata does not enforce. `07_catalog_asset.json` makes personal data discoverable, records purpose and legal basis, and supports DPDP readiness work. It does not restrict access, obtain consent, delete on schedule, or mask anything. Enforcement is a platform concern.

One fact table, one domain. `fact_order_delivery` at order grain plus `fact_production_order`. A real deployment has dozens of facts, conformed dimensions across them, and join paths that need governing at a scale where declaring cardinality by hand stops being practical.

The data is synthetic. 132 orders, deterministic, seeded. Believable enough for the percentages to behave like real ones, small enough to read. Real SAP extracts bring volume, cardinality and messiness that change which failures dominate.

The retriever does no retrieval. With three metrics and eight dimensions the whole context fits in one cacheable prefix, so the entire model is sent every time. Similarity-filtering it would add a failure mode — the right metric filtered out before the resolver ever sees it — for no benefit at this scale. The interface takes the question and returns a string, so real retrieval drops in behind it unchanged.

Deferred, deliberately (design §10.2):

Deferred	Why it is safe to defer
<code>production_cycle_time_days</code>	A plain average over columns the other metrics already materialize. Cheapest later addition, least instructive now.
Golden intents beyond Q1/Q2	Q1 and Q2 exercise every component: descriptive path, diagnostic path, grain, exclusions, attribution, comparison. The others add coverage, not capability.
Live-resolver evaluation	Not verifiable in this environment — no credentials. Documented as instructions, never asserted as a result.

That last one matters. With an `ANTHROPIC_API_KEY` configured, the resolver maps a free -text question onto a `QueryIntent` using `claude-opus-5` with adaptive thinking and a Pydantic output schema, and `tests/test_resolver_live.py` compares the result against the golden intents. Those three tests **skip** without credentials, and this guide does not claim they passed. Every offline branch of the resolver — request shape, prompt-cache split, refusal handling, auth-failure handling — is tested with an injected fake client, so the module is not unexercised; but the live comparison is unverified here and says so.

7. Appendix

`appendix/portability.md` maps every concept in `05_semantic_model.yml` — entity, dimension, metric, grain, exclusion, join cardinality, approval state — onto dbt MetricFlow, Cube, Databricks Unity Catalog, Atlan, Collibra and SAP Datasphere. The YAML here is deliberately tool-neutral; the appendix is where you find out what your platform calls each piece and, more usefully, which pieces it has no name for.

Repository layout

<code>metadata/</code>	the eight artifacts, P0-P7 (nine files: P3 emits two)
<code>data/</code>	<code>seed_core.csv</code> , <code>generate.py</code> , and the SAP-shaped raw extracts
<code>sql/</code>	<code>02_staging_model.sql</code> -- staging views and the order-grain fact
<code>src/semantic/</code>	loader, retriever, resolver, validator, compiler, executor, dq, provenance
<code>src/ask.py</code>	the CLI joining all six stages
<code>docs/</code>	<code>design.md</code> (the spec) and the generated dataflow diagram
<code>standalone/</code>	the whole architecture in one runnable file (two dependencies)
<code>content/</code>	the published write-ups: LinkedIn post, LinkedIn article, Medium post
<code>appendix/</code>	portability mapping
<code>tests/</code>	every number in this guide is an assertion, not prose

Verifying the claims

```
python verify_acceptance.py
```

Checks every acceptance criterion mechanically and exits non-zero on any failure — including that this guide names each phase's NL failure, states the governance limit in the body, and embeds a diagram that is current.

Part 2 — The Design Spec and Acceptance Criteria

Source: [docs/design.md](#)

Date: 2026-08-07 **Status:** Design approved pending user review **Source material:**

[C:\Users\legion\Downloads\export_3382893882042191526.pdf](#) (two posts: semantic metadata enrichment from SAP; On-Time Delivery KPI lineage)

1. Goal

Produce a single deliverable: a **step-by-step implementation guide for building a semantic layer whose success criterion is trustworthy natural-language querying**, accompanied by a small runnable reference implementation.

The guide must show, phase by phase, the **actual metadata artifact** produced at the end of each phase, carried across one continuous worked example with sample data — so a reader sees the same asset get progressively richer, and sees exactly which NL-query failure each phase eliminates.

1.1 Non-goals

- Not a LinkedIn post or article. (Ambiguity resolved: the user reframed the ask to "step by step implementation guide which can help in query in natural language".)
- Not a vendor product evaluation. The semantic model is tool-neutral YAML; portability notes are a short appendix, not a chapter.
- Not a governance/compliance implementation. Catalog metadata supports discovery; it is not a substitute for policy enforcement, consent management, access controls, or retention automation. The guide states this limit explicitly where governance fields appear.
- Not a text-to-SQL research survey. Exactly one architecture is specified and built (Section 4).

2. Organizing thesis

The semantic layer is not the destination. It is the grounding layer that makes natural-language querying trustworthy.

Every metadata phase in the guide justifies itself by closing one concrete NL failure mode:

Missing metadata	NL failure it causes
Business glossary + synonyms	Model cannot map "client number" to <code>party_key</code> ; question fails or binds to the wrong column
Declared grain + join cardinality	Split deliveries and rework loops silently double-count; answer is wrong and looks right
Governed metric definition	Model invents its own arithmetic; two users get two numbers for one question
Business process & event model	Model can answer <i>what</i> happened, never <i>why</i>
Data-quality contract	Answer computed from incomplete inputs is returned with false confidence
Published catalog entry	Answer arrives with no provenance, so no one can defend it in a review

The guide is structured so this table is *demonstrated*, not asserted: each phase's section ends with a before/after of the same NL question.

3. Domain and worked example

Domain: SAP Plan-to-Deliver — manufacturing through delivery. This extends the source posts' Order-to-Cash scope upstream into production, because the diagnostic NL question ("why did it drop?") is unanswerable without manufacturing phases.

3.1 Anchor KPI

On-Time Delivery % (`on_time_delivery_pct`) (orders delivered on or before promised delivery date / eligible orders due for delivery) × 100 Grain: one record per customer order. Exclusions: cancelled orders. Refresh: daily.

3.2 The two exemplar NL queries

These drive every design decision. Query 2 is the reason manufacturing is in scope.

The demo's as-of date is **2026-08-07**, so "last month" resolves to **2026-07**. June 2026 is the comparison window.

Query 1 — descriptive

"On-time delivery for India warehouses last month"

Resolves to: metric `on_time_delivery_pct` ; filters `region=IN` , `month=2026-07` ; grain `order` ; exclusions `cancelled` , `not-yet-due` . Returns:

87.3% — 48 of 55 eligible orders delivered on or before promised date. Provenance: `SAP SD` → `PP` → `QM` → `MM/EWM` → `TM` . Trust: `TRUSTED` .

`87.3%` is `48/55 = 87.2727%` , rounded to one decimal for display. Tests assert the unrounded ratio.

Query 2 — diagnostic

"Why did on-time delivery drop for India warehouses last month?"

Resolves to: same metric and filters, broken out by `delay_attribution_phase` , compared against the prior month. Returns:

87.3%, down 8.9pp from 96.2% in June (25 of 26 eligible). 7 orders were late. Attribution: **quality inspection 4**, transportation 2, production execution 1. On those 4 orders, QM usage decision ran an average **4.75 days over its 1.0-day standard, against 0 days of available slack** — the promise left no room to absorb it. Plant 1010.

Both answers are computed from the committed sample data (Section 5) and asserted in the acceptance criteria (Section 8), so every figure in the guide is reproducible rather than illustrative.

Query 1 needs only the glossary, grain, and metric definition. Query 2 additionally needs per-phase timestamps, per-phase planned baselines, and a variance-attribution rule **defined once in the semantic model**. That is the payoff for Phase 4, and the reason manufacturing is in scope.

3.3 Business process phases (manufacturing → delivery)

Twelve phases. Each is a row in `04_process_model.yml` , which carries the warehouse column that materializes the phase timestamp. The SAP fields below are the *business* provenance of each event and are illustrative: only those with a row in `01_technical_metadata.json` are harvested from DDIC, and `mrp_date` / `release_date` are synthesized by the generator rather than extracted, so they have no DDIC entry.

#	Phase	System / module	Business event	Timestamp field	Std. duration (days)
1	Demand & order capture	SAP SD	Sales order created	VBAK-ERDAT	0
2	Requirements planning	SAP PP (MRP)	Planned order created	PLAF-PSTTR	1
3	Production order release	SAP PP	Production order released	AFKO-GSTRP / GLTRP	1
4	Material staging & issue	SAP MM	Goods issue to production order	MSEG-BUDAT	1
5	Production execution	SAP PP	Operation confirmed	AFRU-BUDAT	3
6	Quality inspection	SAP QM	Usage decision (accept / reject / rework)	QALS-VDATUM	1
7	Goods receipt to stock	SAP MM / EWM	GR posted to unrestricted stock	MKPF-BUDAT	1
8	Picking & packing	SAP EWM	Picking completed	picking_completed_timestamp	1
9	Goods issue / dispatch	SAP LE-SHP	Actual goods issue	LIKP-WADAT_IST	0
10	Transportation	SAP TM	Shipment dispatched	VTTK-DPTBG	3
11	Delivery confirmation	SAP TM / LE	POD / delivery confirmed	delivery_timestamp	0
12	Billing	SAP SD	Invoice created	VBRK-FKDAT	1

3.4 Variance attribution model (makes "why" computable)

Per order, for each phase: `actual_duration_days` from consecutive event timestamps; `variance_days = actual_duration_days - standard_duration_days`.

```
delay_attribution_phase =
  argmax(variance_days) over the 9 variance spans (phase_seq 2..10),
  ties broken to the lowest phase_seq,
  evaluated only for orders where delivery_date > promised_delivery_date
```

Available slack per order = `promised_delivery_date - order_date - Σ standard_duration_days`.

This is defined in `05_semantic_model.yml` as a governed derived dimension, computed in SQL by the compiler. The LLM never derives it.

Phase 1 is the chain's start point rather than a span, phase 11 is a 0-day event folded into `var_shipment`, and phase 12 follows delivery — so 9 of the 12 phases are emittable attribution values. Declaring the other three would let a filter pass the gates and return zero rows dressed as an answer.

3.5 Governed metrics

Four, so the guide can show metric interaction and per-phase KPIs rather than a single toy measure.

Metric	Definition	Grain	Exclusions
on_time_delivery_pct	delivered on/before promised ÷ eligible due	order	cancelled; not-yet-due
mfg_schedule_adherence_pct	production orders confirmed on/before scheduled finish ÷ released orders	production_order	cancelled production orders
first_pass_yield_pct	production orders accepted at the first usage decision ÷ production orders inspected	production_order	rework re-inspections (2nd+ decision)
production_cycle_time_days	avg(gr_date - release_date)	production_order	cancelled orders

- **Failure is refusal, not fabrication.** An unresolvable question fails the validator gate and returns "I can't answer that with governed metrics; here's what's missing" — instead of confident nonsense.

4.2 Validator gates

1. `metric` exists in the semantic model **and** `approval_state == approved`
2. Every `dimension` is declared for that metric's entity
3. Every `filter` column is bound to a real column and its value is in the declared allowed-values set (or passes the declared type/format)
4. `grain` matches the metric's declared grain
5. No requested join path introduces fan-out beyond the declared cardinality
6. `time_window` resolves to a concrete bounded range
7. Required exclusions from the metric definition are present in the compiled predicate

Gate failures return a structured refusal naming the failed gate.

4.3 Claude API contract

Grounded in the `claude-api` skill (read this session; do not answer these from memory when implementing):

Parameter	Value	Reason
<code>model</code>	<code>claude-opus-5</code>	Current default
Structured output	<code>client.messages.parse()</code> with a Pydantic <code>QueryIntent</code> model (<code>output_config.format</code> / <code>json_schema</code> under the hood)	Validated object, retried on schema mismatch; no hand-parsing
<code>thinking</code>	<code>{"type": "adaptive"}</code>	Synonym mapping + relative-date arithmetic is non-trivial
<code>output_config.effort</code>	<code>medium</code> , with a documented sweep note	Per-query latency matters; guide instructs readers to sweep <code>low</code> / <code>medium</code> / <code>high</code> on their own eval set
<code>max_tokens</code>	<code>16000</code>	Non-streaming; intent objects are small but thinking shares the budget
Prompt caching	<code>cache_control: {"type": "ephemeral"}</code> on the last system block (stable semantic-metadata context); volatile NL question in the user turn	Semantic context is reused across every query; Opus 5 minimum cacheable prefix is 512 tokens
Refusal handling	Check <code>response.stop_reason == "refusal"</code> before reading <code>content</code>	Required on Opus 5

Sampling parameters (`temperature` , `top_p` , `top_k`) and `budget_tokens` are **not** used — they return 400 on Opus 5.

4.4 Offline mode (required, not optional)

`ANTHROPIC_API_KEY` is not set in the target environment. The deliverable therefore ships `tests/golden_intents.json` — hand-authored `QueryIntent` objects for 12 canonical questions. Everything except the resolver runs with zero API calls: warehouse build, compiler, validator, provenance, all tests. `ask.py` takes `--offline` to compile a golden intent by question ID. The guide states this up front so a reader without a key still gets a working system.

5. Sample data

Two competing needs: percentages must be **believable** (87.3% is impossible with 6 orders — each order would move the number 16.7pp), and the interesting rows must be **readable**. Resolved by splitting the data into a hand-authored core and a

generated filler.

5.1 Scale

Windows are cut on `promised_delivery_date` , declared as the metric's time dimension. As-of date is 2026-08-07, so every July-promised order is due.

Segment	Orders in window	Cancelled	Eligible	On time	Late	OTD
IN — July 2026 (Query 1 & 2 target)	56	1	55	48	7	87.27%
IN — June 2026 (comparison)	27	1	26	25	1	96.15%
IN — August 2026	4	0	0 (all not-yet-due)	—	—	n/a
US — July 2026	30	2	28	26	2	92.86%
US — June 2026	15	0	15	14	1	93.33%
Total	132	4	124	113	11	—

The 7 late IN/July orders attribute as **quality inspection 4, transportation 2, production execution 1** — the answer Query 2 must produce.

The drop is `96.1538 - 87.2727 = 8.88pp`, reported as 8.9pp.

The August window carries the **not-yet-due** exclusion: 4 orders promised after the as-of date, so the eligible denominator is 0 and the metric must return "no eligible orders" rather than 0% or a divide-by-zero.

Dimension	Values
Plant	<code>1010</code> Pune, IN · <code>1710</code> Plano, US
Warehouse	<code>WH-IN-01</code> · <code>WH-US-01</code>
Carrier	<code>BLUEDART</code> · <code>FEDEX</code>
Region	<code>IN</code> · <code>US</code>

5.2 Hand-authored vs. generated

- `data/seed_core.csv` — **20 hand-authored orders**. Every defect in 5.3, every late order, and the full manufacturing event chain for each. These are the rows the guide prints and walks through line by line, and the rows the tests assert on by ID.
- `data/generate.py` — **deterministic filler**. Fixed seed (`random.Random(20260807)`), no wall-clock, no network. Produces the remaining 112 well-behaved on-time orders needed to hit the denominators above. Committed alongside its output CSVs, so the repo works without running it and the numbers never drift.

Every figure in the guide traces to committed CSVs. The generator makes the aggregates realistic; it does not make them arbitrary.

Raw extract files (SAP-shaped, in `data/`): `sales_orders_raw.csv` , `production_orders_raw.csv` , `inspection_lots_raw.csv` , `goods_movements_raw.csv` , `deliveries_raw.csv` , `shipments_raw.csv` .

5.3 Deliberate defects — each teaches one lesson

All in `seed_core.csv` , all in the IN / July window.

#	Defect	Where	Lesson it forces
1	Cancelled order	<code>SO-1007</code> , <code>order_status = CANC</code>	Exclusion must be in metadata, or the denominator inflates
2	Null <code>delivery_timestamp</code> , in transit, promised date in the future	<code>SO-1013</code>	Must not count as late; completeness rule needs a scoped predicate, not a blanket not-null
3	Rework loop — two QM usage decisions on one lot	<code>SO-1004</code> / lot <code>QL-4004</code>	Fan-out double-count; <code>first_pass_yield_pct</code> needs "first decision only" as a governed exclusion
4	Split deliveries — 6 eligible IN/July orders have 2 delivery rows each , incl. <code>SO-1009</code> → <code>DLV-2009A</code> (on time), <code>DLV-2009B</code> (late)	Grain double-count. Partial shipment is normal SAP behaviour, so this is the realistic trap, not a contrived one	
5	Promised date changed mid-flight	<code>SO-1011</code> (2026-07-18 → 2026-07-24)	Which promise is measured against? Forces an explicit SCD-2 / as-of decision in metadata
6	Make-to-stock order, no production order	<code>SO-1015</code>	LEFT JOIN semantics; manufacturing phases legitimately absent and must not null out the row or the metric
7	Late orders with QM-dominated variance	<code>SO-1002</code> , <code>SO-1004</code> , <code>SO-1006</code> , <code>SO-1010</code>	Produces Query 2's attribution answer, hand-verifiable from 4 printed rows
8	Late orders attributed elsewhere	<code>SO-1003</code> , <code>SO-1009</code> (transport) · <code>SO-1005</code> (production)	Attribution must discriminate, not always blame QM

5.4 The wrong answer, worked

Defects 3 and 4 are load-bearing: they are why declaring the grain stops being ceremony. The guide opens with the wrong answer and derives it explicitly.

An analyst joins `fact_order_delivery` to the delivery table and computes the metric at **delivery** grain instead of **order** grain. 6 of the 55 eligible IN/July orders shipped in two parts, so the row count becomes 61:

- 5 split orders delivered both legs on time (`SO-1012` , `SO-1014` , `SO-1016` , `SO-1017` , `SO-1018`) → 2 on-time rows each
- `SO-1009` delivered leg A on time and leg B late → 1 on-time row, 1 late row. At order grain it is **late** (the order isn't complete until the last leg lands).

Arithmetic:

	On time	Denominator	OTD
Grain-correct (order)	48	55	87.27%
Naive (delivery)	43 non-split + 10 split legs + 1 (<code>SO-1009</code> leg A) = 54	61	88.52%

The naive answer is **1.25pp too high** — and *higher* is the dangerous direction. The error flatters the number, so nobody questions it. It is close enough to pass review and close enough to put in a board deck.

Worse, the bias direction depends on the data: a fully on-time split order pulls the naive figure **up**, a fully late one pulls it **down**. So the error is not even a consistent offset you could learn to correct for. Grain mistakes do not announce themselves with absurd values; they quietly move the number by roughly the amount people argue about in review meetings. A validator that checks declared grain catches this. Eyeballing the output does not.

The same trap appears in `first_pass_yield_pct` : counting `SO-1004` 's rework re-inspection gives $52/55 = 94.55\%$ instead of the correct $51/54 = 94.44\%$ — again, wrong in the flattering direction.

`test_compiler.py` asserts every figure in this section, so the narrative is under test.

6. Build phases and their artifacts

Every phase section in the guide follows the same five-part shape: **INPUT** → **WORK** → **OUTPUT (a real file, shown in full or in a faithful excerpt)** → **GATE (how you know the phase is done)** → **NL FAILURE CLOSED (before/after on a real question)**.

Phase	Name	Artifact emitted	NL failure closed
P0	Scope & KPI contract	<code>00_kpi_contract.md</code> — the question inventory (12 NL questions the layer must answer), metric owners, decision the KPI supports	Building a semantic layer with no acceptance test for "done"
P1	Technical metadata harvest	<code>01_technical_metadata.json</code> — SAP DDIC extract: tables, fields, data elements, domains, keys, nullability, source system	Nothing to bind business terms to
P2	Standardize & model	<code>02_staging_model.sql</code> + <code>02_standardized_metadata.json</code> — canonical names, typed columns, conformed dimensions, grain declared per table	<code>VBELN</code> vs <code>vbeln</code> vs <code>sales_doc</code> — model can't tell they're one thing
P3	Business glossary & semantic binding	<code>03_glossary.yml</code> (terms, definitions, synonyms, allowed values, owner, steward) + <code>03_column_bindings.yml</code> (term → physical column)	Model can't map "client number" → <code>party_key</code> ; can't validate "India" as a region value
P4	Business process & event lineage	<code>04_process_model.yml</code> — 12 phases, systems, events, actors, timestamp bindings, standard durations, variance rules	Model answers <i>what</i> , never <i>why</i> ; no phase attribution possible
P5	Semantic model & metrics	<code>05_semantic_model.yml</code> — entities, dimensions, joins with cardinality , 4 metrics with grain/exclusions/business rules/approval state	Model invents arithmetic; double-counts on split deliveries and rework
P6	Quality contract	<code>06_dq_rules.yml</code> — rules per DQ dimension, each bound to a specific metric input, each with observed score vs. business expectation vs. enforced control	Confidently wrong answers from incomplete inputs
P7	Publish & NL resolution	<code>07_catalog_asset.json</code> (catalog/marketplace entry, lineage, governance fields) + the runnable resolver	Answers with no provenance; nothing to defend in a review

6.1 Phase 6 data-quality rules

Bound to metric inputs, so a failing rule degrades a specific answer rather than a dashboard-wide red light.

Dimension	Rule	Bound to
Completeness	<code>delivery_timestamp</code> not null where <code>promised_delivery_date < current_date</code> and <code>order_status <> 'CANC'</code> ; threshold ≤ 2% null	<code>on_time_delivery_pct</code> numerator
Validity	<code>delivery_status</code> ∈ { <code>DELIVERED</code> , <code>IN_TRANSIT</code> , <code>FAILED</code> , <code>RETURNED</code> }; <code>plant</code> ∈ { <code>1010</code> , <code>1710</code> }	filter binding
Uniqueness	exactly one row per <code>order_id</code> in <code>fact_order_delivery</code>	grain guarantee (defect 4)
Timeliness	mart <code>last_refreshed_at</code> within 26 hours	freshness badge on every answer
Consistency	<code>release_date</code> ≤ <code>gr_date</code> ≤ <code>goods_issue_date</code> ≤ <code>delivery_date</code>	variance attribution validity
Accuracy	recomputed <code>on_time_delivery_pct</code> within 0.1pp of certified monthly snapshot	metric certification

Each answer returns a **trust badge** — `TRUSTED` / `DEGRADED (rule X failing)` / `BLOCKED` — derived from these rules. A `BLOCKED` input returns a refusal, not a number.

6.2 Phase 7 governance fields (with stated limits)

`07_catalog_asset.json` carries: owner, steward, data-subject category, personal-data category, processing purpose, legal basis, access group, retention period, masking policy, certification state, lineage chain, related metrics.

The guide states plainly: these fields make personal data **discoverable** and support India DPDP readiness work. They do not enforce anything. Enforcement lives in access control, consent management, retention automation, and masking at the platform layer.

7. Deliverable structure

semantic-layer-nl/	
README.md	the guide (primary deliverable)
data/	
seed_core.csv	20 hand-authored orders (all defects, all late orders)
generate.py	deterministic filler, fixed seed 20260807
sales_orders_raw.csv	
production_orders_raw.csv	
inspection_lots_raw.csv	
goods_movements_raw.csv	
deliveries_raw.csv	
shipments_raw.csv	
metadata/	
00_kpi_contract.md	
01_technical_metadata.json	
02_standardized_metadata.json	
03_glossary.yml	
03_column_bindings.yml	
04_process_model.yml	
05_semantic_model.yml	
06_dq_rules.yml	
07_catalog_asset.json	
sql/	
02_staging_model.sql	
src/	
build_warehouse.py	raw CSV → staging → marts (DuckDB)
semantic/	
loader.py	parse + structurally validate the YAML model
retriever.py	select the metadata slice for a question
resolver.py	Claude API: NL → QueryIntent (json_schema)
validator.py	the 7 gates
compiler.py	QueryIntent → SQL (deterministic)
provenance.py	answer + lineage + trust badge
dq.py	run 06_dq_rules.yml, emit badge
ask.py	CLI: python ask.py "... " [--offline ID]
tests/	401 tests, 3 skipped without an API key
golden_intents.json	Q1 and Q2 (offline mode); see 10.2
test_data.py	Section 5.1 denominators vs. committed CSVs
test_metadata.py	every artifact parses, no placeholders
test_warehouse.py	staging + marts, attribution SQL
test_loader.py	the model parses and structurally validates
test_retriever.py	unapproved metrics are never offered
test_validator.py	each gate rejects what it must
test_compiler.py	grain / fan-out / exclusion regressions
test_provenance.py	lineage chain and trust badge
test_dq.py	each defect trips its intended rule
test_end_to_end.py	CLI answers Q1 and Q2 as specified
test_diagram.py	all 3 diagrams match the code they depict
test_guide.py	README claims vs. artifacts and warehouse
test_content.py	all three published documents, same treatment
test_standalone.py	the single-file demo runs, and agrees with the project it was extracted from

test_bundle.py	the "everything" PDF really is everything
test_resolver_live.py	skipped without ANTHROPIC_API_KEY
docs/	
design.md	this document
md2pdf.py	markdown → styled HTML → PDF (headless Chrome)
build_bundle.py	all 5 documents + all 9 artifacts → one PDF
semantic-layer-complete.md	the assembled bundle, regenerated not edited
semantic-layer-complete.pdf	the same, printed (no page count stated here: the bundle contains this file, so any number written here changes the number)
diagrams/	
render_dataflow.py	renders the swimlane [--check]
nl_dataflow.svg / .png	embedded in the guide
render_sources.py	renders the provenance map [--check]
metadata_sources.svg / .png	embedded in the guide
render_grain.py	renders the two-counting-rules figure that opens the Medium post -- no jargon, one order, three hand-counted rows [--check]
grain_two_ways.svg / .png	the Medium post's lead image
content/	the three published deliverables
linkedin-post.md	part 1, ~250 words
linkedin-article.md	part 2, the long-form walkthrough
medium-post.template.md	part 3's prose, with {{...}} splice markers
medium-post.md	part 3, generated -- never hand-edited
medium-publishing-checklist.md	title, tags, images, canonical URL, sequencing: the working notes, kept out of the article
build_medium_post.py	splices real code and real output into the template, so the published code cannot drift from the tested code
medium-kit.md	part 3 as a publication kit, generated: cover, the article with the code lifted out, checklist
medium-code.md	the lifted code on its own, with its sha256
build_medium_kit.py	derives both from medium-post.md, so the split copies cannot drift from the source either
*.html / *.pdf	each part rendered for review; the HTML is compared byte-for-byte against a fresh render, so a stale copy fails the suite
standalone/	
semantic_layer_demo.py	the whole architecture in one runnable file: embedded data, inline metadata, 7 gates, compiler, diagnostic. pip install duckdb pyyaml
appendix/	
portability.md	mapping to dbt MetricFlow, Cube, Databricks Unity Catalog, Atlan, Collibra, SAP Datasphere
verify_acceptance.py	measures Section 8, one verdict per criterion

7.1 Guide reading order

1. Why NL querying fails without a semantic layer (the two exemplar queries, wrong answers first)
2. The Plan-to-Deliver process and the anchor KPI
3. Phases P0–P7, one section each, artifact shown
4. The NL resolution path end to end (retriever → resolver → validator → compiler → provenance)

5. Running it yourself (with and without an API key)
6. What this does **not** do (governance limits, scale limits, single-fact-table limits)
7. Appendix: portability

8. Acceptance criteria

The deliverable is done when all of the following hold:

1. `python src/build_warehouse.py` builds the DuckDB warehouse from the committed CSVs with no manual steps and no network access.
2. `python src/ask.py --offline Q1` returns **87.3%** (48/55) for India / 2026-07 with the full provenance chain and a **TRUSTED** badge.
3. `python src/ask.py --offline Q2` returns **87.3%, -8.9pp vs. 96.2%**, with attribution **quality inspection 4, transportation 2, production execution 1**.
4. `pytest` passes with **no ANTHROPIC_API_KEY** set; live resolver tests skip cleanly.
5. `test_compiler.py` asserts the grain-correct **87.27% (48/55)** and the naive delivery-grain **88.52% (54/61)** from Section 5.4, plus FPY **94.44% (51/54)** vs. naive **94.55% (52/55)** — the "wrong answer first" narrative is under test, including that both errors overstate.
6. `test_dq.py` proves each of the 8 seeded defects trips its intended rule and no others.
7. `test_data.py` asserts every denominator in Section 5.1 against the committed CSVs, so spec and data cannot drift.
8. `python data/generate.py` is deterministic: re-running it leaves the CSVs byte-identical.
9. Every one of the 9 metadata artifacts exists as a real, parseable file — no placeholders, no **TBD**. (Nine files across eight phases: P3 emits both `03_glossary.yml` and `03_column_bindings.yml`.)
10. Each phase section in `README.md` names the specific NL failure it closes, with a before/after.
11. The governance-limits statement appears in the guide body, not buried in an appendix.
12. With a valid `ANTHROPIC_API_KEY`, `python src/ask.py "on-time delivery for India warehouses last month"` produces an intent matching golden intent `Q1`.

Two criteria were added after the spec's original list of 12. They are numbered 12 and 13 by `verify_acceptance.py` and labelled additive rather than renumbering the list above, so the spec's numbering still matches the plan it came from:

- **12 (additive)**. The dataflow diagram is committed, current, and embedded: `docs/diagrams/nl_dataflow.svg` and `.png` exist, `render_dataflow.py --check` exits 0, and `README.md` embeds the SVG.
- **13 (additive)**. The metadata sourcing map is committed, current, embedded — and *artifact-backed*: every one of its 13 source→artifact edges carries a **probe** string that must occur in the artifact it points at, and the figure's central claim (exactly one machine-harvestable artifact) still holds. A provenance diagram is the one kind that nothing else can catch lying, since "this came from SAP DDIC" is unfalsifiable unless something checks the file.

The third figure, `grain_two_ways`, has no acceptance criterion — it was added for the Medium post's plain-English opening, after the criteria were settled, and renumbering to accommodate it would break the correspondence above. `tests/test_diagram.py` covers it to the same standard: every figure in it is recomputed from the warehouse and from the standalone demo's embedded rows, so the friendly simplification cannot drift into a lie.

9. Where this lives — settled

`C:\Users\legion` is not a git repository, so committing this design document needed a home. Three options were on the table: leave it uncommitted, `git init` a specs-only repo under `C:\Users\legion\docs\`, or give the deliverable its own repo and move the spec into it.

Resolved as (c). The deliverable is a self-contained runnable project and benefits from its own history, and a spec that travels with the code it specifies is the one that gets updated when the code changes. This document is `docs/design.md` in

C:\Users\legion\semantic-layer-nl\ , built on branch `feat/semantic-layer-mvp` .

10. MVP scope

The first increment must be **runnable and honest**, not fully featured. Definition of the MVP: *a reader clones the project, runs two commands with no API key, and gets the guide's two headline answers with correct provenance — and the tests prove the numbers.*

10.1 In the MVP

Area	MVP content
Metadata artifacts	All 8. These are the deliverable — the guide is <i>about</i> the artifact each phase emits. Cutting any would gut the thesis.
Business process	All 12 phases in <code>04_process_model.yml</code> , with SAP field bindings and standard durations
Metrics	3 — <code>on_time_delivery_pct</code> (full narrative), <code>mfg_schedule_adherence_pct</code> (manufacturing KPI), <code>first_pass_yield_pct</code> (proves a second grain + the rework exclusion)
Sample data	Full Section 5 dataset — <code>seed_core.csv</code> + generator, all 8 defects
Pipeline	<code>build_warehouse.py</code> , <code>loader</code> , <code>validator</code> , <code>compiler</code> , <code>provenance</code> , <code>dq</code>
NL path	Offline via golden intents for Q1 and Q2 ; <code>resolver.py</code> present and callable, exercised only when a key exists
Tests	<code>test_compiler.py</code> , <code>test_dq.py</code> , <code>test_data.py</code> — acceptance criteria 1–11
Guide	<code>README.md</code> covering P0–P7 with each phase's artifact and the NL failure it closes

10.2 Deferred past the MVP

Deferred	Why it's safe to defer
<code>production_cycle_time_days</code>	A plain average over columns the other three metrics already materialize. Cheapest later addition, least instructive to build now.
Golden intents beyond Q1/Q2 (10 of 12)	Q1 and Q2 exercise every component: descriptive path, diagnostic path, grain, exclusions, attribution. The other 10 add coverage, not capability.
Live-resolver eval / effort sweep	Cannot be run or verified in this environment (no API key). Documented as instructions, not asserted as results.

Two rows that were on this list are no longer deferred, and are recorded here because what changed is worth keeping:

Was deferred	What actually shipped
<code>appendix/portability.md</code> as a stub table	Ships as the full mapping: every concept across dbt MetricFlow, Cube, Unity Catalog and Datasphere, a second table for Atlan and Collibra, and the three gaps a platform move inherits (grain is rarely first-class, <code>allowed_values</code> rarely enforces, <code>approval_state</code> is a tag almost everywhere).
<code>test_validator.py</code> per-gate rejection tests	Ships with 22 tests. Deferring these turned out to be the wrong call: writing them found that <code>exclusions_applied</code> — alone among the seven — had never been observed refusing anything. It was implemented, registered, and asserted <code>True</code> on the happy path, which is indistinguishable from a gate that always passes. It now has four rejection tests (dropped block, empty list, missing <code>predicate</code> , missing <code>key</code>), plus one that fails if any future gate is added without a rejection test.

10.3 MVP acceptance

Acceptance criteria 1–11 in Section 8 apply as written. Criterion 12 (live resolver) is documented but **not verifiable here** — the guide will state that plainly rather than imply it was tested.

11. Decisions already settled

Recorded so they are not relitigated during implementation:

Decision	Choice
Artifact type	One implementation guide + runnable reference implementation. Not a post, not three documents.
Success criterion	Trustworthy NL querying
Domain	SAP Plan-to-Deliver (manufacturing through delivery), continuing the source posts' narrative
"Phases of metadata"	The metadata artifact emitted at the end of each build phase; the same asset gets progressively richer
NL layer approach	Tool-neutral YAML semantic model plus runnable demo (DuckDB + Claude API + validation gate)
Query generation	NL → validated intent JSON → deterministic compiler. The model never writes SQL.
Sample data	132 orders (20 hand-authored + deterministic filler), 2 plants / 2 warehouses / 2 carriers, 8 deliberate defects. Sized so 87.3% and 96.2% are real, not illustrative.
Offline operation	Required — golden intents make everything but the resolver runnable without an API key
Model	<code>claude-opus-5</code> , adaptive thinking, <code>messages.parse()</code> with a Pydantic intent schema
Build strategy	MVP first (Section 10): runnable end to end with no API key, 3 metrics, all 8 artifacts, Q1 + Q2 answering correctly under test

Part 3 — The LinkedIn Post

Source: [content/linkedin-post.md](#)

Part 1 of three: this short post, the long-form LinkedIn article ([content/linkedin-article.md](#)), and the Medium post carrying the complete executable code ([content/medium-post.md](#)). ~300 words, one image (the one-order-two-boxes figure at [docs/diagrams/grain_two_ways.png](#) ; see the note below on the alternatives).

Two reports land on your desk. One says you signed up **1,200 customers** last month. The other says **1,000**. Same database, same afternoon — and one of them is just wrong. Nobody in the room can tell you which.

The wrong one counted *orders* instead of *customers*: anyone who bought twice got counted twice. It is an honest mistake, the number looks completely reasonable, and it always errs *upward* — which is exactly why it is the one that ends up on the board slide. Nobody re-checks a number that flatters them.

Now swap "customers" for any figure your business actually reports. Ours was **on-time delivery**, and the same bug gave us two answers — **88.5%** or **87.3%**. Some orders arrived in two boxes, so counting *shipments* instead of *orders* counted those orders twice: 55 orders came in 61 boxes.

No amount of prompt engineering fixes this. The model didn't misunderstand the question — it answered a perfectly reasonable question against a table where nobody had written down what one row is supposed to mean.

So I built the fix, and wrote up every step. The model's only job is to pick an approved metric and fill in a small, checkable form: which metric, which filters, which dates. It never writes SQL and never does the arithmetic — a deterministic compiler does that, from governed metadata. Seven checks run before any SQL exists, and one refuses the double-counted question outright instead of answering it.

The real payoff is the question a dashboard can't take: *why did it drop?* On-time delivery fell **8.9pp** — and the obvious suspect was innocent. Transportation actually ran ahead of plan; the delay was quality inspection, **4.75** days over a one-day standard, with no slack to absorb it.

Full write-up, sample data, and a single file you can paste into a terminal and run — two dependencies, no API key — in the comments.

What's your team's real answer to "*which of these two numbers is right?*" If it's "ask Priya," you already have a semantic layer. It just isn't written down.

Notes for posting

- **Image:** [docs/diagrams/grain_two_ways.png](#) (2600×1677) — one order shipped in two boxes, counted two ways to two different numbers. It states the whole problem without assuming any vocabulary, which is the same job the opening now does in words, and it is the frame that survives the feed. The pipeline swimlane [docs/diagrams/nl_dataflow.png](#) (2730×1872) is the alternative if you would rather lead with the fix than the failure; it carries the argument on its own for a reader who only looks at the picture.
- **Second figure** — [docs/diagrams/metadata_sources.png](#) (2730×1872): where each of the nine artifacts comes from and when it is consumed. Lead with this one instead if the audience is the people who *fund* metadata work rather than the people who query it — "only 1 of 9 artifacts is machine-harvestable" is the line that changes a budget conversation, and it is the honest counter to "can't we just point a catalog at it?". Do not post both as a carousel on the first post; a single figure gets read, two get skimmed. Better as the follow-up post a few days later, which also gives the first one a reason to resurface.
- **First comment:** the repo link plus `python src/ask.py --offline Q1`. LinkedIn suppresses reach on posts with outbound links in the body, which is why the link sits in a comment.
- **Sequencing the three parts.** Post this first; publish the LinkedIn article a day or two later and link it from the comments of this one; publish the Medium post ([content/medium-post.md](#) , the complete runnable file) last and link it from the

article's closing section. Reversed, the short post has nothing left to promise. Do not ship all three the same day — they compete for the same readers, and the article is the one that needs the runway.

- **No hashtag block.** Three at most if you want them: #DataEngineering #SemanticLayer #DataGovernance.
- The closing question is real, not rhetorical bait — most teams genuinely do not have a documented answer, and the replies are the useful part of posting this.

Part 4 — The LinkedIn Article

Source: `content/linkedin-article.md`

An implementation guide, with the sample metadata each phase actually produces.

Ask a database a plain question — "how many customers did we have last month?" — and it can hand you two different answers from the same rows. One counts customers; the other counts orders, so anyone who bought twice is counted twice. Both queries run, both look reasonable, and the wrong one is always the larger — so it is the one that reaches the slide. Swap "customers" for any figure your business actually reports and the trap is the same.

The same bug produced the two numbers this guide is about. Same warehouse, same afternoon, same question:

"On-time delivery for India warehouses last month."

88.5% and **87.3%**.

Same DuckDB file. Same SQL dialect. Same 132 orders. One of them is wrong, and in the review nobody could say which — which is the actual problem, and it is not a problem prompt engineering can reach.

Here is where 88.5% comes from. The `deliveries_raw` table has one row per delivery *leg*. An order that ships in two legs — partial shipment on Tuesday, remainder on Friday — has two rows. Join orders to legs and count, and that order votes twice. Worse: if the first leg arrived before the promised date, the order looks on time twice over, while the second leg that actually blew the promise is one row among 61.

61 legs. 55 orders.

- Order grain: **48 / 55 = 87.27%**
- Delivery grain: **54 / 61 = 88.52%**

The gap is 1.25 points. It is small enough to look like rounding, and it always errs upward. A wrong number that flatters you is the one that survives to the board deck.

The model did not hallucinate. It wrote syntactically valid, semantically reasonable SQL against a table whose grain nobody had written down anywhere it could read. **The failure was in the metadata, so the fix has to be in the metadata.**

The architectural rule

Everything below follows from one decision:

The model never writes SQL and never does arithmetic. It reads a governed metadata slice and emits a validated JSON object — which metric, which filters, which time window. A deterministic compiler turns that object into SQL.

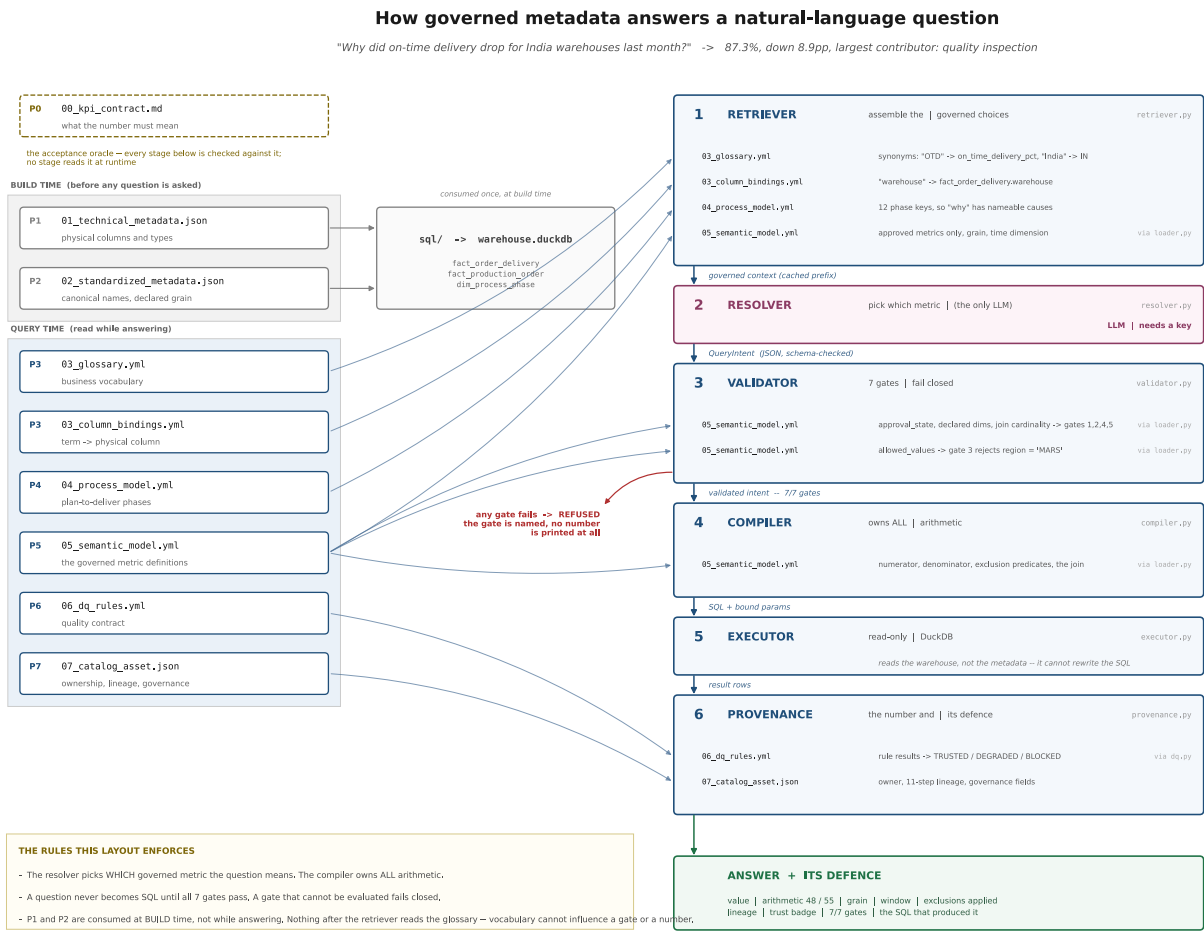
This is the inversion that matters. Text-to-SQL asks a language model to be right about grain, joins, exclusions and fiscal calendars on every single call. This architecture asks it to be right about *one* much easier thing: which of three approved metrics you meant. Everything that must be exactly right is compiled from metadata, by code, the same way every time.

Two users asking the same question in different words get the same number, because the number was never in the model's hands.

The pipeline is six stages:

`retriever → resolver → validator → compiler → executor → provenance`

Only the **resolver** calls a model. Only the **compiler** writes SQL. And validation happens *before* compilation, so a refused question never becomes SQL at all.



Full resolution: [docs/diagrams/nl_dataflow.svg](#) — scaled to fit a page here, so open it directly to read the edge labels.

Each arrow is a stage reading a specific artifact for a specific reason. Two rows repay a second look: **the validator and the compiler never open a file**. Both receive a **SemanticModel** that the loader parsed, which means the compiler has no access to the glossary at all. Vocabulary is resolved *before* the gates, so which synonym a user typed can never reach the arithmetic.

Eight phases, nine artifacts

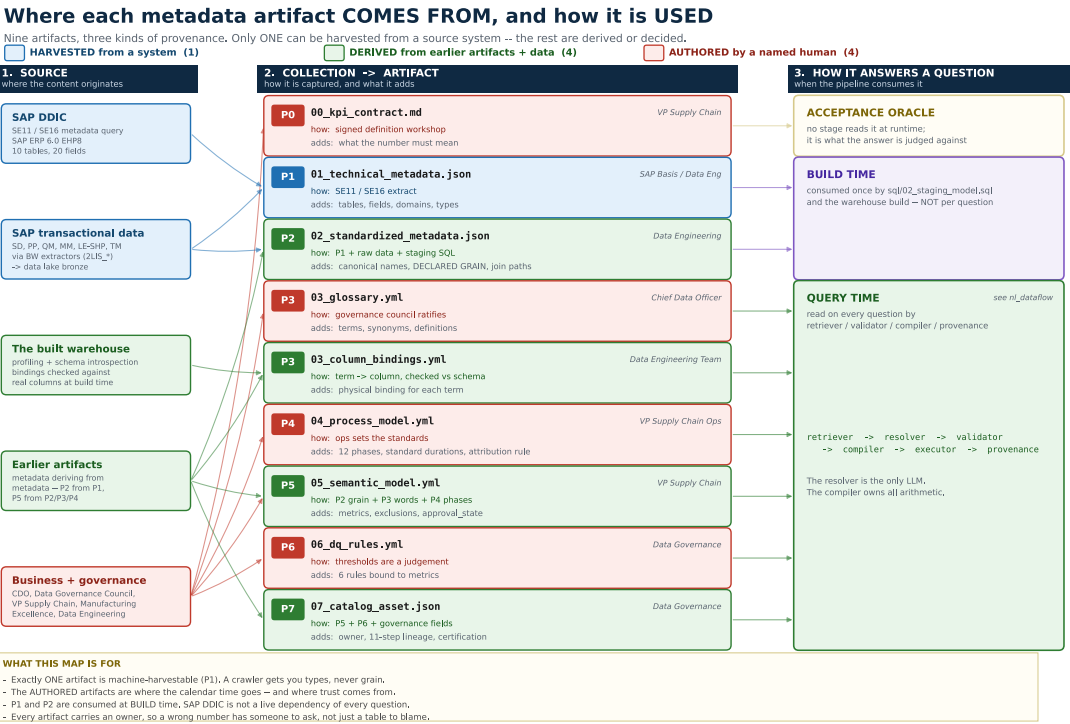
Each phase produces a real file and closes one specific way natural-language querying fails. That pairing is the whole method — a phase that doesn't close a named failure is a phase you can skip.

Phase	Artifact	The NL failure it closes
P0	00_kpi_contract.md	Nobody agrees what "on-time" means, so no answer can be wrong
P1	01_technical_metadata.json	Column names harvested from SAP DDIC, with no business meaning
P2	02_standardized_metadata.json	Types and keys inconsistent across source systems
P3	03_glossary.yml + 03_column_bindings.yml	"OTD", "delivery reliability", "promise adherence" reach no column
P4	04_process_model.yml	The model can answer <i>what</i> happened, never <i>why</i>
P5	05_semantic_model.yml	The model invents its own arithmetic — this is where 88.5% dies
P6	06_dq_rules.yml	A number computed from incomplete inputs is returned with full confidence
P7	07_catalog_asset.json	The answer arrives with no provenance, so nobody can defend it

Nine files, eight phases: P3 emits two, because a glossary term and its physical binding are different governance objects with different owners.

Where the metadata comes from, and where it goes

The question that lands next — usually from whoever is being asked to fund this — is "can't we just harvest all that from SAP?"



Full resolution: docs/diagrams/metadata_sources.svg — every box carries the collection method and the accountable owner, which is worth opening for.

Read it left to right: **source** → **how it was collected** → **the artifact** → **when the pipeline consumes it**. The colours are the answer to the funding question.

One of the nine artifacts is machine-harvestable. `01_technical_metadata.json` comes out of the SAP data dictionary via SE11 / SE16 — repeatable, cheap, automatable. The other eight split two ways:

Provenance	Count	What it means	Artifacts
HARVESTED	1	a repeatable query against a running system	P1
DERIVED	4	computed from earlier artifacts plus the data	P2, P3 bindings, P5, P7
AUTHORED	4	a named human decided something no system knows	P0, P3 glossary, P4, P6

The derived four are cheap to keep current, because their inputs are themselves versioned artifacts: P2 from P1 plus the raw rows plus staging SQL; the column bindings from the glossary checked against real warehouse columns at build time; P5 from P2's grain, P3's vocabulary and P4's phases; P7 from P5 + P6 plus governance fields.

The authored four are where the calendar time goes. Somebody had to decide that "on-time" means the *current* promised date and not the original (P0), that "OTIF" and "promise adherence" are sanctioned synonyms for the same metric (P3), that a standard QM usage decision takes one day (P4), and that 2% missing delivery dates is **degrading** rather than **blocking** (P6). No amount of crawling produces any of those, and none of them are discoverable from the data — they are judgements, made by someone accountable for them.

Which is the whole point, stated as a planning fact: **a harvester gets you tables, fields, domains and types. It cannot get you grain, exclusions, an approved definition or a standard duration** — and those are precisely the fields that decide whether the answer is right rather than merely well-formed. The 88.5% error lives in a field no crawler emits. This is why "point a catalog at the warehouse and turn on NL query" produces a demo and not a trustworthy system.

And then how it gets used

The right-hand band is the other half of the question, and it separates three things that get conflated:

- **P0 is never read at runtime.** It is the acceptance oracle — the thing the answer is judged *against*, not an input to producing it. If your KPI contract is being parsed by software, it has stopped being a contract.
- **P1 and P2 are consumed once, at build time,** by `sql/02_staging_model.sql` and the warehouse build. So the SAP data dictionary is not a live dependency of every question — which is both the truth and considerably less alarming than the alternative reading.
- **P3–P7 are read on every question,** by retriever, validator, compiler and provenance respectively.

That last line is the one that matters for cost and for failure modes: six artifacts sit in the hot path of every natural-language question, and if any of them is wrong, every answer is wrong the same way — silently, consistently, and with a provenance block attached that makes it look governed.

Both figures are generated from declared graphs, not drawn. Every arrow in the sourcing map carries a **probe** string that must occur in the artifact it points at, and the two diagrams are cross-checked against each other, so neither can claim a lineage the files don't support.

The synonym layer (P3)

Vocabulary is where NL querying quietly fails, and it is the cheapest phase to build. From the real file, with two catalog fields dropped:

```
- term: On-Time Delivery %
  definition: >
    Share of eligible customer orders delivered on or before the promised
    delivery date. Measured at order grain (one row per order_id). An order
    counts as delivered only when its final delivery leg arrives. Excludes
    cancelled orders and orders not yet due.
  synonyms: [OTD, on time delivery, on-time delivery, delivery performance,
    delivery reliability, promise adherence, OTIF, on time in full]
  owner: VP Supply Chain
  steward: Supply Chain Analytics Lead
```

Note that the definition states the grain *in prose the model reads*. That sentence — "one row per order_id" — is doing real work.

The governed metric (P5)

This is the load-bearing artifact. Abridged from the real file:

```
- name: on_time_delivery_pct
  label: On-Time Delivery %
  entity: fact_order_delivery
  grain: order
  time_dimension: promised_delivery_date
  expression:
    type: ratio_percent
    numerator: sum(CASE WHEN is_on_time THEN 1 ELSE 0 END)
    denominator: sum(CASE WHEN is_eligible THEN 1 ELSE 0 END)
  exclusions:
    - key: cancelled_orders
      predicate: order_status <> 'CANC'
      rationale: A cancelled order was never due; including it inflates the denominator.
    - key: not_yet_due
      predicate: promised_delivery_date <= DATE '2026-08-07'
      rationale: >
        An order promised after the as-of date is not late, it is pending.
  business_rules:
    - Measured against the CURRENT promised date, not the original.
    - Order-grain only. Delivery-grain aggregation double-counts split shipments.
  owner: VP Supply Chain
  approval_state: approved
  lineage: [SAP SD, SAP EWM, SAP TM, fact_order_delivery]
```

Four things to notice, because each one is a defect that would otherwise ship:

1. **grain: order** is declared, not inferred. This is the field that kills 88.5%.
2. **Exclusions carry a rationale**. Six months on, an exclusion without a stated reason gets deleted by someone who assumes it was a mistake.
3. **approval_state: approved** is read by the retriever, so an unapproved metric is never shown to the model. It cannot be asked for because it was never offered.
4. **Ratios are declared as numerator and denominator**, never as a pre-averaged rate. The compiler sums numerators and sums denominators, then divides. Averaging percentages across groups of unequal size gives a different, wrong number — and it looks fine.

The data-quality contract (P6)

Rules bind to specific metrics, so a defect degrades only what it actually touches:

```
- rule_id: dq_completeness_delivery_date
  dimension: completeness
  threshold: 2.0
  comparison: lte
  severity: degrading
  bound_to: [on_time_delivery_pct]
```

A trust badge — **TRUSTED**, **DEGRADED**, **BLOCKED** — ships with every answer. **BLOCKED** *replaces* the number rather than annotating it, because a caveated wrong number gets pasted into a slide with the caveat trimmed off.

The seven gates

Between the model's JSON and any SQL, seven checks run. All seven run every time, and a gate that cannot be evaluated **fails closed**:

Gate	Refuses
metric_approved	a metric that isn't approved, or doesn't exist
dimensions_declared	grouping by something the model invented
filters_bound	region = 'MARS' — a value outside the declared allowed set
grain_matches	the 88.5% query. It never becomes SQL.
no_fanout	a dimension only reachable through a one-to-many join
time_window_bounded	an unbounded or inverted date range
exclusions_applied	a metric whose certified exclusions can't be applied

A refusal names the gate and the reason. This is the actual output when the 88.5% query is put through the pipeline:

```
REFUSED on_time_delivery_pct: the intent did not pass validation.

gates failed grain_matches
grain_matches      Metric 'on_time_delivery_pct' is certified at grain 'order'; the
                    intent asks for grain 'delivery'. Aggregating at the wrong grain
                    changes the number.

No number is shown. A value that failed a gate is not a caveated answer,
it is an unverified one.
```

That is more useful than a number. "I can't answer that, and here is which rule stopped me" is a system you can put in front of a CFO. One that always produces a number is not.

Note **filters_bound** in particular. Without a declared allowed-value set, a filter on a misspelled region returns zero rows — and zero rows gets presented as an answer.

The question BI tools can't take

Descriptive questions ("what was it?") only need a glossary, a grain and a metric. The question people actually ask is diagnostic:

"Why did on-time delivery drop for India warehouses last month?"

That requires per-phase timestamps, per-phase planned baselines, and an attribution rule **defined once, in metadata**. This is why the manufacturing process is in scope: the answer to "why" lives upstream in production, not in the delivery table.

The process model declares twelve phases across SAP SD, PP, QM, MM/EWM, LE and TM — demand capture, MRP, production order release, material staging, production execution, quality inspection, goods receipt, picking, dispatch, transportation, delivery confirmation, billing. Nine of them are attributable spans. The other three aren't, and declaring them would let a filter pass every gate and return zero rows dressed as an answer.

For each order and phase: $\text{variance_days} = \text{actual_duration} - \text{standard_duration}$. The attribution is one **argmax** over the nine spans, ties broken to the earliest phase, evaluated only for orders that actually missed.

The real output, with the answer block's metadata lines elided:

ANSWER	On-Time Delivery % is 87.3% -- 48 of 55 eligible orders delivered on or before the promised date		
	... (metric, arithmetic, grain, window, filters, exclusions, lineage, trust, gates)		
BREAKDOWN			
	Quality inspection	4	
	Transportation	2	
	Production execution	1	
	unattributed	0	
	Largest contributor: Quality inspection (4 of 7 late orders).		
COMPARISON			
	prior window	June 2026 (2026-06-01 to 2026-06-30) on promised_delivery_date	
	prior value	96.2% (25 / 26)	
	change	down 8.9pp (96.2% -> 87.3%)	

unattributed 0 is printed even though it is zero, on purpose: an attribution that silently drops the orders it could not explain is an attribution you cannot audit.

On those 4 orders, the QM usage decision ran an average of **4.75 days over its 1.0-day standard, against 0.0 days of available slack** — the promise date left no room to absorb it. Plant 1010.

And the part that makes the case: **transportation ran under standard on 5 of the 7 late orders**. Logistics was the obvious suspect and logistics was faster than planned. Without per-phase baselines you get a plausible, confident, wrong story — and you go optimize the wrong department.

What this does *not* do

Worth stating plainly, because governance metadata invites overreading.

Catalog metadata supports **discovery**. It is not policy enforcement, consent management, access control, or retention automation. An **owner** field records who is accountable; it does not stop anyone from querying anything. Those are separate systems, and a semantic layer that implies otherwise is worse than one that stays silent.

Also honestly scoped: one fact table with declared joins, not a general warehouse; a frozen as-of date so the numbers are reproducible; a single-node engine. And the live resolver path is documented but **not** asserted as tested here — it needs API credentials this environment doesn't have, and the verification script reports that criterion as **SKIP** rather than quietly counting it as a pass.

Reproducing it

Everything above is measured, not illustrative. Five commands, no API key, no network:

```
pip install -r requirements.txt
python src/build_warehouse.py
python src/ask.py --offline Q1      # 87.3%, 48/55, TRUSTED, 7/7 gates
python src/ask.py --offline Q2      # the diagnostic breakdown above
python verify_acceptance.py         # measures every acceptance criterion
```

401 tests hold it in place, and the guide's prose is under test alongside the code — every figure in it is asserted against the artifacts or the warehouse, because a reproduction guide whose numbers have drifted spends the reader's trust before it spends their time.

If you would rather not clone anything: the companion **Medium post** carries the whole thing as a single self-contained file — embedded data, inline metadata, all seven gates, the compiler and the diagnostic — that runs on `pip install duckdb pyyaml` and reproduces every number above. It is the same file this repository tests, byte for byte.

The one-line version

Natural-language querying does not fail because models can't write SQL. It fails because the metadata that makes a question answerable *correctly* — grain, cardinality, exclusions, approved definitions, process baselines — was never written down where anything could read it.

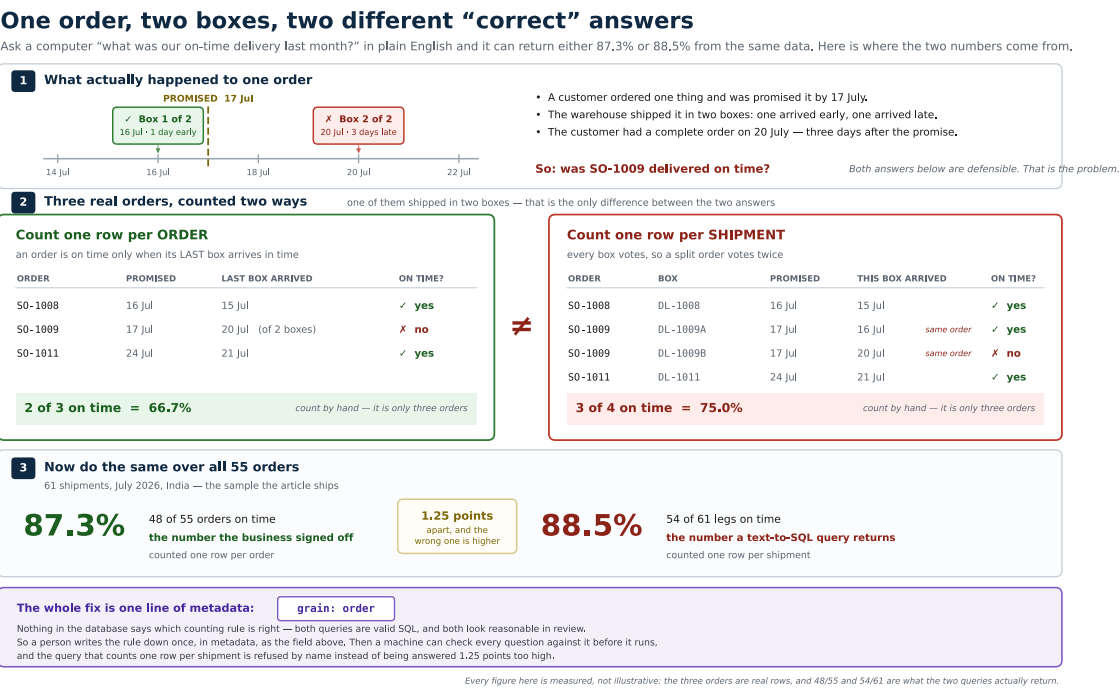
Build that, and the model's job shrinks to something it is genuinely good at.

What is your team's answer to "which of these two numbers is right"? If it's "ask Priya," you have a semantic layer — it just isn't written down.

Part 5 — The Medium Post, With Complete Executable Code

Source: `content/medium-post.md`

A governed semantic layer that refuses the wrong number — complete, executable Python, two dependencies, no API key.



The entire argument in one picture. A courier splits one order into two boxes; the two defensible ways to count that order disagree, and the disagreement scales to the whole month. Every figure in it is measured from the data the code below ships — a test recomputes all of them, including the three-order example. The full-resolution original is `docs/diagrams/grain_two_ways.svg` in the repository linked at the end.

Two numbers came out of the same DuckDB file on the same afternoon, from the same question:

"On-time delivery for India warehouses last month."

88.5% and 87.3%. One of them is wrong, and in the review nobody could say which.

That last part is the actual problem, and prompt engineering cannot reach it.

Start here: one order, two boxes

If `grain` and `fan-out` are words you already use at work, skip to The rule everything else follows from. Nothing in this section is needed twice.

A customer orders one thing and is promised it by **17 July**. The warehouse ships it in two boxes:

- **Box 1** arrives **16 July** — one day early.
- **Box 2** arrives **20 July** — three days late.

The customer had a complete order on 20 July. So: *was that order delivered on time?*

Nothing in the database answers that question. There are two counting rules, and both are defensible:

- **One row per order.** An order is on time only when its *last* box arrives in time. By this rule the order is **late**.
- **One row per shipment.** Every box is counted on its own. By this rule the order is **one box on time and one box late**.

Pick either. Just notice that nobody has written down which one the company means.

Count three real orders by hand

These are three actual orders out of the data the code below ships. Two arrived in a single box; **S0-1009** is the one that shipped in two:

Order	Promised	Boxes	Last box arrived	On time, counting orders?
S0-1008	16 Jul	1	15 Jul	yes
S0-1009	17 Jul	2	20 Jul	no
S0-1011	24 Jul	1	21 Jul	yes

Counting **orders**, two of the three were on time — **66.7%**.

Counting **shipments** there are four rows rather than three, because **S0-1009** contributes two: its first box beat the promise and its second missed it. Three of four — **75.0%**.

Same three orders, same dates, two answers — and the higher one came from the rule nobody had written down. Across the full month, 55 orders arriving in 61 shipments, those same two rules give **87.3%** and **88.5%**.

Five words the rest of this post needs

Word	What it means here
grain	What one row of a table stands for. The orders table's grain is one row per order; the delivery table's is one row per <i>shipment</i> . Almost nothing in a database records this.
fan-out	What happens when you join a table to a finer-grained one: one order becomes two rows, and gets counted twice. That is the 88.5%.
metric	A number with its definition written down — numerator, denominator, grain, which rows to exclude, and a named person who approved it.
gate	A check that runs <i>before</i> the query does and refuses it if something the metric declared is not satisfied.
provenance	The record of how one answer was produced: which metric, which arithmetic, which rows, which checks passed.

Everything below is those five ideas as a single Python file you can run.

TL;DR — Natural-language querying does not fail on SQL syntax. It fails on *grain*: the model writes a valid query against a table whose row meaning nobody wrote down. The fix is architectural — the model picks a governed metric and emits validated JSON, a deterministic compiler writes the SQL, and seven gates run in between. And once that metadata exists, the same file answers the question no BI tool can take — *why* the number dropped — by attributing each late order to a manufacturing phase. This post is that system as a single file you can paste into a terminal.

You will need Python 3.10 or newer and two packages. No credentials, no network, no warehouse of your own.

```
pip install duckdb pyyaml
python semantic_layer_demo.py
```

In this post

- Plain English first: one order, two boxes, and the five words that follow
- Where the wrong number comes from: six split shipments
- The metadata fields that decide correctness, and why no crawler emits them
- The one word in the warehouse build that changes the answer
- The seven gates, including the one that refuses the wrong query by name
- Why on-time delivery dropped — the diagnostic question BI tools cannot take
- The complete file, verbatim

New to this? The first section needs no SQL. Only have five minutes? Read the seven gates and why it dropped — that pair is the argument.

Where the wrong number comes from

The delivery table has one row per delivery *leg*. An order that ships in two legs — partial shipment on Tuesday, remainder on Friday — has two rows.

Join orders to legs and count, and that order votes twice. Worse: if the first leg arrived before the promised date, the order looks on time twice over, while the second leg that actually blew the promise is one row among 61.

- Order grain: **48 / 55 = 87.27%**
- Delivery grain: **54 / 61 = 88.52%**

The gap is 1.25 points. Small enough to look like rounding, and it always errs upward.

A wrong number that flatters you is the one that survives to the board deck.

The model did not hallucinate. It wrote syntactically valid, semantically reasonable SQL against a table whose grain nobody had written down anywhere it could read. **The failure was in the metadata, so the fix has to be in the metadata.**

The rule everything else follows from

The model never writes SQL and never does arithmetic. It reads a governed metadata slice and emits a validated JSON object — which metric, which filters, which time window. A deterministic compiler turns that object into SQL.

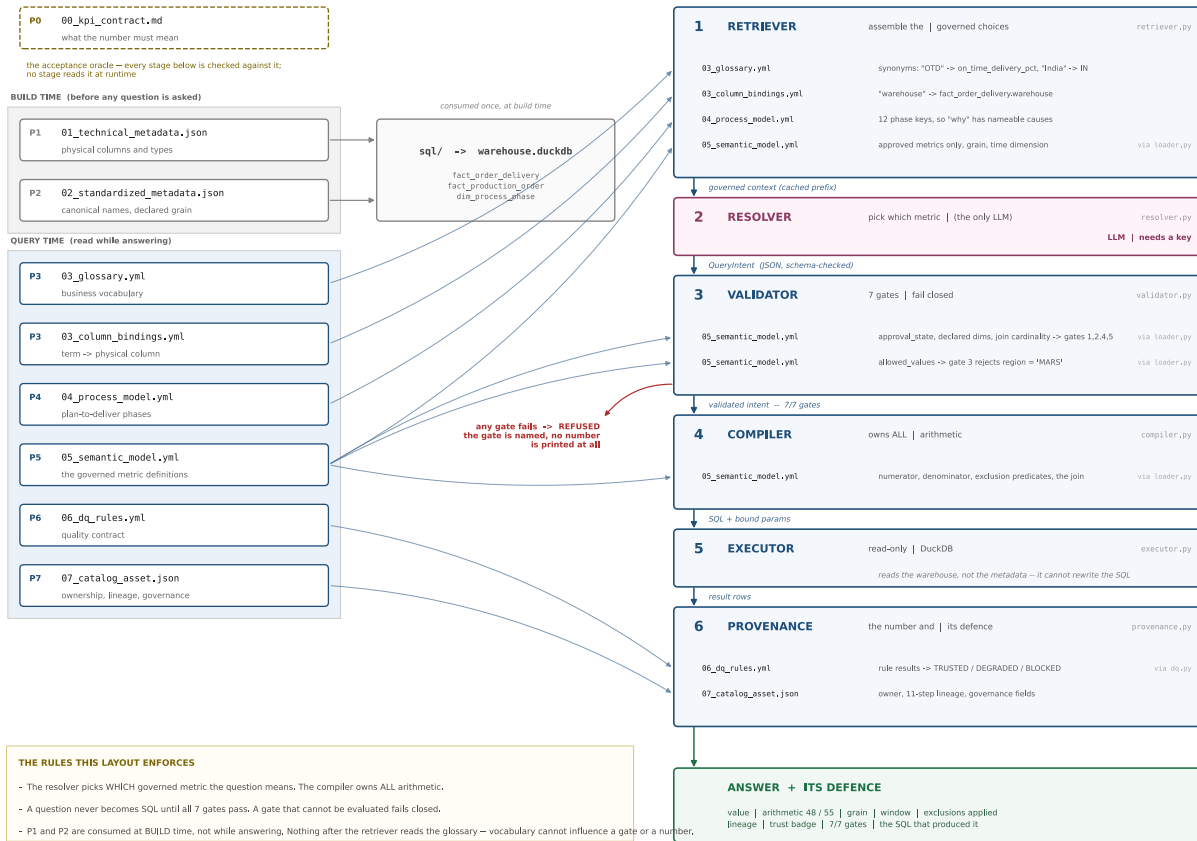
Text-to-SQL asks a language model to be right about grain, joins, exclusions and fiscal calendars on every single call. This asks it to be right about *one* much easier thing: which approved metric you meant.

The pipeline is six stages, and only one of them touches a model:

```
retriever → resolver → validator → compiler → executor → provenance
           ↑           ↑
           the only    seven gates, before
           model call   any SQL exists
```

How governed metadata answers a natural-language question

"Why did on-time delivery drop for India warehouses last month?" -> 87.3%, down 8.9pp, largest contributor: quality inspection



Every arrow is one stage reading one artifact for one reason. The figure is generated from the code it describes, so it cannot drift from the pipeline it depicts. The full-resolution original is [docs/diagrams/nl_dataflow.svg](#) in the repository.

Two users asking the same question in different words get the same number, because the number was never in the model's hands.

The data, and the one difference that matters

Two tables. **orders** has one row per customer order; **delivery_legs** has one row per physical shipment. Six of the July orders shipped in two legs — those six are the entire disagreement between 87.3% and 88.5%:

DL-1009A,SO-1009,1,2026-07-16
DL-1009B,SO-1009,2,2026-07-20
DL-1010,SO-1010,1,2026-07-22
DL-1011,SO-1011,1,2026-07-21
DL-1012A,SO-1012,1,2026-07-18
DL-1012B,SO-1012,2,2026-07-18
DL-1014A,SO-1014,1,2026-07-19
DL-1014B,SO-1014,2,2026-07-19
DL-1015,SO-1015,1,2026-07-19
DL-1016A,SO-1016,1,2026-07-21
DL-1016B,SO-1016,2,2026-07-21
DL-1017A,SO-1017,1,2026-07-22
DL-1017B,SO-1017,2,2026-07-22
DL-1018A,SO-1018,1,2026-07-23
DL-1018B,SO-1018,2,2026-07-23

S0-1009 is the one that costs you the number. It was promised 2026-07-17. Its first leg landed 07-16 — on time — and its second landed 07-20. At order grain it is late. At delivery grain it is one hit and one miss, and the hit came first.

The file embeds the June and July 2026 India slice of the full project: 83 orders, 87 delivery legs, verbatim. That is the whole population behind every figure in this post, so the numbers reproduce exactly rather than approximately.

The metadata: the fields no crawler emits

Read this and ask which fields a catalog pointed at the warehouse could have produced:

```
- name: on_time_delivery_pct
  label: On-Time Delivery %
  description: >
    Share of eligible customer orders delivered on or before the promised delivery
    date. An order counts as delivered only when its final leg arrives.
  entity: fact_order_delivery
  grain: order
  time_dimension: promised_delivery_date
  expression:
    type: ratio_percent
    numerator: "sum(CASE WHEN is_on_time THEN 1 ELSE 0 END)"
    denominator: "sum(CASE WHEN is_eligible THEN 1 ELSE 0 END)"
  exclusions:
    - key: cancelled_orders
      predicate: "order_status <> 'CANC'"
      rationale: A cancelled order was never due; including it inflates the denominator.
    - key: not_yet_due
      predicate: "promised_delivery_date <= DATE '__AS_OF_DATE__'"
      rationale: >
        An order promised after the as-of date is not late, it is pending. Counting it
        as a miss understates performance.
  business_rules:
    - Measured against the CURRENT promised date, not the original.
    - Order-grain only. Delivery-grain aggregation double-counts split shipments.
  owner: VP Supply Chain
  approval_state: approved
  lineage: [SAP SD, SAP EWM, SAP TM, fact_order_delivery]
```

entity and **time_dimension**, maybe. **Not grain**. **Not exclusions**. **Not business_rules**. **Not approval_state**.

Those are decisions, made by a named human, and they are exactly the fields that decide whether an answer is right rather than merely well-formed.

Four things worth noticing, because each one is a defect that would otherwise ship:

1. **grain: order** is declared, not inferred. This is the field that kills 88.5%.
2. **Exclusions carry a rationale**. Six months on, an exclusion without a stated reason gets deleted by someone who assumes it was a mistake.
3. **approval_state** is read before the model sees anything. The second metric in the file is **draft**, so the retriever never offers it — it cannot be asked for because it was never on the menu.
4. **The ratio is declared as a numerator and a denominator**, never as a pre-averaged rate. The compiler sums numerators and sums denominators, then divides. Averaging percentages across groups of unequal size gives a different, wrong number, and it looks completely fine.

The one-to-many join is declared too, which is what lets a gate see the fan-out coming:

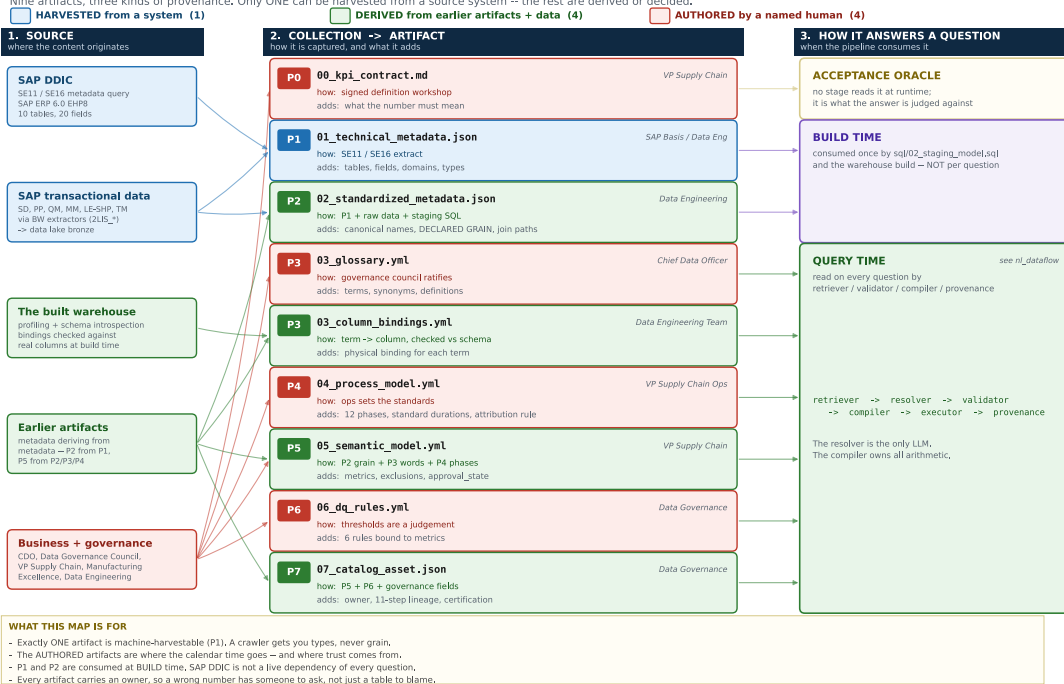
entities:

- name: fact_order_delivery
grain: order # one row per order_id -- the load-bearing declaration
primary_key: order_id
joins:
 - to: raw_delivery_legs
on: order_id
cardinality: one_to_many # this is the hop that turns 87.27% into 88.52%
- name: raw_delivery_legs
grain: delivery
primary_key: delivery_id

Which is the part that gets underestimated on a plan. In the full nine-artifact project, exactly one artifact is machine-harvestable — the technical metadata harvested from the SAP data dictionary. Everything else is derived from earlier artifacts, or authored by someone accountable for the decision:

Where each metadata artifact COMES FROM, and how it is USED

Nine artifacts, three kinds of provenance. Only ONE can be harvested from a source system -- the rest are derived or decided.



Colour is provenance: harvested, derived, authored. A harvester gets you tables, fields, domains and types. It cannot get you grain, exclusions, an approved definition or a standard duration. The full-resolution original is [docs/diagrams/metadata_sources.svg](#) in the same repository.

The warehouse: one word decides the answer

```
def build_warehouse() -> duckdb.DuckDBPyConnection:
    """In-memory DuckDB. The one interesting line is the aggregation to ORDER grain.

    `final_delivery_date = max(delivery_date)` is what "an order counts as delivered only
    when its final leg arrives" means in SQL. Taking min() -- or joining without
    aggregating at all -- is the 88.5% bug, and it is a one-word difference.
    """
    con = duckdb.connect(":memory:")
    variance_cols = PHASE_VARIANCE_CSV.splitlines()[0].split(",")[1:]
    for name, ddl, csv_text in (
        ("raw_orders",
         "order_id VARCHAR, region VARCHAR, plant VARCHAR, carrier VARCHAR, "
         "promised_delivery_date DATE, order_status VARCHAR", ORDERS_CSV),
        ("raw_delivery_legs",
         "delivery_id VARCHAR, order_id VARCHAR, delivery_leg INTEGER, "
         "delivery_date DATE", DELIVERY_LEGS_CSV),
        ("raw_phase_variance",
         "order_id VARCHAR, " + ", ".join(f"{c} INTEGER" for c in variance_cols),
         PHASE_VARIANCE_CSV),
    ):
        _load(con, name, ddl, csv_text)

    con.execute(f"""
    CREATE TABLE fact_order_delivery AS
    WITH final_leg AS (
        SELECT order_id,
               max(delivery_date) AS final_delivery_date,
               count(*)           AS delivery_count
        FROM raw_delivery_legs
        GROUP BY order_id          -- <-- collapse to one row per ORDER
    )
    SELECT o.order_id, o.region, o.plant, o.carrier,
           o.promised_delivery_date, o.order_status,
           f.final_delivery_date,
           coalesce(f.delivery_count, 0) AS delivery_count,
           -- The two exclusions, materialised as a single eligibility flag.
           (o.order_status <> 'CANC'
            AND o.promised_delivery_date <= DATE '{AS_OF_DATE}') AS is_eligible,
           (f.final_delivery_date IS NOT NULL
            AND f.final_delivery_date <= o.promised_delivery_date) AS is_on_time
    FROM raw_orders o
    LEFT JOIN final_leg f USING (order_id)
    """)
    return con
```

`max(delivery_date)` is what "an order counts as delivered only when its final leg arrives" means in SQL. Taking `min()` — or joining without aggregating at all — is the 88.5% bug, and it is a one-word difference in a query nobody would flag in review.

The intent: the only thing the model is allowed to produce

```
@dataclass(frozen=True)
class QueryIntent:
    """What a language model is permitted to emit. Note what is NOT in here: no SQL, no
    column expressions, no arithmetic, no join instructions. Every field is a reference
    to something the metadata already declares, which is what makes it checkable."""
    intent_type: str                # descriptive | diagnostic
    metric: str
    grain: str
    time_window: tuple[str, str]
    time_label: str = ""
    dimensions: tuple[str, ...] = ()
    filters: tuple[Filter, ...] = ()
    compare_to: tuple[str, str] | None = None
    compare_label: str = ""

    def to_json(self) -> str:
        return json.dumps({
            "intent_type": self.intent_type, "metric": self.metric, "grain": self.grain,
            "time_window": list(self.time_window), "dimensions": list(self.dimensions),
            "filters": [{"column": f.column, "operator": f.operator, "value": f.value}
                        for f in self.filters],
        }, indent=2)
```

Note what is *not* in there: no SQL, no column expressions, no arithmetic, no join instructions. Every field is a reference to something the metadata already declares, which is precisely what makes it checkable by a machine before it runs.

The resolver: the only stage that calls a model

The demo ships an offline keyword resolver so it runs with no credentials. The real one is this, and the two are interchangeable by design — every stage downstream takes a `QueryIntent` and cannot tell which produced it:


```

def resolve_with_claude(question: str, model: dict) -> QueryIntent:    # pragma: no cover
    """The real resolver. Not called by `main` -- shown because it is the whole point.

    Two things make this safe, and neither is prompt wording. First, the retriever hands
    over only APPROVED metrics, so an unapproved one cannot be picked: it was never
    offered. Second, whatever comes back goes through all seven gates before it can
    become SQL, so a wrong intent is refused rather than answered.
    """

    import anthropic

    approved = [m for m in model["metrics"] if m["approval_state"] == "approved"]
    slice_for_model = {
        "metrics": [{k: m[k] for k in ("name", "label", "grain", "description")
                        if k in m} for m in approved],
        "dimensions": [{"name": d["name"], "allowed_values": d.get("allowed_values")}
                        for d in model["dimensions"]],
        "glossary": [{"term": t["term"], "metric": t["metric"],
                        "synonyms": t["synonyms"]} for t in model["glossary"]],
    }

    tool = {
        "name": "emit_query_intent",
        "description": "Emit the governed query intent for this question.",
        "input_schema": {
            "type": "object",
            "properties": {
                "intent_type": {"enum": ["descriptive", "diagnostic"]},
                "metric": {"enum": [m["name"] for m in approved]},
                "grain": {"type": "string"},
                "time_window": {"type": "array", "items": {"type": "string"},
                                "minItems": 2, "maxItems": 2},
                "dimensions": {"type": "array", "items": {"type": "string"}},
                "filters": {"type": "array", "items": {
                    "type": "object",
                    "properties": {"column": {"type": "string"},
                                    "operator": {"enum": ["=", "in", "between",
                                                            ">=", "<="]},
                                    "value": {}},
                    "required": ["column", "operator", "value"]},
                },
                "required": ["intent_type", "metric", "grain", "time_window"],
            },
        },

    }

    reply = anthropic.Anthropic().messages.create(
        model="claude-sonnet-5",
        max_tokens=1024,
        system=RESOLVER_SYSTEM_PROMPT,
        tools=[tool],
        tool_choice={"type": "tool", "name": "emit_query_intent"},
        messages=[{"role": "user", "content":
            f"Metadata slice:\n{json.dumps(slice_for_model, indent=2)}\n\n"
            f"Question: {question}"}],
    )

    args = next(b.input for b in reply.content if b.type == "tool_use")

```

```
return QueryIntent(  
    intent_type=args["intent_type"], metric=args["metric"], grain=args["grain"],  
    time_window=tuple(args["time_window"]),  
    dimensions=tuple(args.get("dimensions", ())),  
    filters=tuple(Filter(f["column"], f["operator"], f["value"])  
                  for f in args.get("filters", ())),  
)
```

Two things make this safe, and neither of them is prompt wording.

The menu is filtered before the model sees it. `approved` excludes the draft metric, so the tool's `enum` cannot contain it. An unapproved metric is unaskable because it was never offered — not because a gate would have caught it afterwards.

Whatever comes back is still validated. The system prompt matters less than the fact that a wrong intent gets refused rather than answered:

You select a governed metric. You do not write SQL and you do not do arithmetic.

Return ONLY a QueryIntent. Every value must come from the metadata slice provided:

- metric: an exact name from the approved metric list
- grain: the metric's declared grain, copied verbatim, never inferred from the question
- filters: only columns listed as dimensions, only values from allowed_values
- time_window: explicit start and end dates, never open-ended

If the question cannot be answered from this slice, say so instead of approximating.

An approximation that looks like an answer is worse than a refusal.

The validator: seven gates, and they fail closed

```
GATE_NAMES = ("metric_approved", "dimensions_declared", "filters_bound", "grain_matches",
              "no_fanout", "time_window_bounded", "exclusions_applied")

@dataclass
class ValidationResult:
    passed: dict[str, bool] = field(default_factory=dict)
    failures: list[tuple[str, str]] = field(default_factory=list)

    @property
    def ok(self) -> bool:
        return len(self.passed) == len(GATE_NAMES) and all(self.passed.values())

def validate(qi: QueryIntent, model: dict) -> ValidationResult:
    """Never raises. A hallucinated metric name is a normal, expected input."""
    r = ValidationResult()
    metric = get_metric(model, qi.metric)

    def gate(name: str, ok: bool, why: str = "") -> None:
        r.passed[name] = ok
        if not ok:
            r.failures.append((name, why))

    gate("metric_approved",
        metric is not None and metric.get("approval_state") == "approved",
        f"Metric {qi.metric!r} is not an approved metric. Approved: "
        + ", ".join(m["name"] for m in model["metrics"]
                    if m["approval_state"] == "approved"))

    undeclared = [d for d in qi.dimensions if d not in model["_dims_by_name"]]
    gate("dimensions_declared", not undeclared,
        f"Dimensions {undeclared} are not declared in the semantic model.")

    unbound = [f.column for f in qi.filters if f.column not in model["_dims_by_name"]]
    outside = [f"{f.column}={f.value!r}" for f in qi.filters
               if f.column in model["_dims_by_name"]
               and model["_dims_by_name"][f.column].get("allowed_values")
               and f.value not in model["_dims_by_name"][f.column]["allowed_values"]]
    gate("filters_bound", not unbound and not outside,
        f"Filter columns {unbound} do not bind to a declared dimension."
        if unbound else
        f"Filter {'', '.join(outside)} is outside the declared allowed_values. Without "
        f"this gate the query returns zero rows, and zero rows gets read as an answer.")

    if metric is None:
        gate("grain_matches", False,
            f"Cannot check grain: metric {qi.metric!r} is not in the semantic model, so "
            f"there is no certified grain to compare {qi.grain!r} against.")
    else:
        gate("grain_matches", qi.grain == metric["grain"],
```

```

        f"Metric {qi.metric!r} is certified at grain {metric['grain']!r}; the intent "
        f"asks for grain {qi.grain!r}. Aggregating at the wrong grain changes the "
        f"number.")

fanout = []
if metric is not None:
    home = metric["entity"]
    for name in list(qi.dimensions) + [f.column for f in qi.filters]:
        dim = model["_dims_by_name"].get(name)
        if dim is None or dim["entity"] == home:
            continue
        join = next((j for j in model["_entities_by_name"][home].get("joins", []))
                     if j["to"] == dim["entity"]), None)
        # A many_to_one hop cannot duplicate rows and must pass. Only one_to_many
        # fans out. A gate that blocked every join would look correct and quietly
        # break every legitimate cross-entity question.
        if join is None or join["cardinality"] == "one_to_many":
            fanout.append(f"{name} (on {dim['entity']})")
gate("no_fanout", not fanout,
     f"Reaching {'', '.join(fanout)} requires a one_to_many join, which duplicates "
     f"rows before aggregation.")

start, end = qi.time_window
gate("time_window_bounded", bool(start and end) and start <= end,
     f"Time window {qi.time_window} is unbounded or inverted.")

gate("exclusions_applied",
     metric is not None and all(e.get("predicate") for e in
                                metric.get("exclusions", [])),
     f"Metric {qi.metric!r} declares an exclusion with no predicate, so it cannot be "
     f"applied. A metric whose exclusions cannot be enforced must not be answered.")
return r

```

Three rules in there are worth stating out loud.

All seven gates always run. Not "until the first failure" — a user asking a badly-formed question deserves every reason at once, not a game of whack-a-mole.

A gate that cannot be evaluated fails closed. Look at `grain_matches` when the metric does not exist: it reports a failure explaining that there is no certified grain to compare against. The tempting alternative is to skip it.

"I did not check" recorded as "I checked and it was fine" is the single most dangerous line you can write in a governance layer.

no_fanout blocks only one_to_many. A `many_to_one` hop cannot duplicate rows, so refusing every join would look correct and quietly break every legitimate cross-entity question. Gates that are too strict get switched off, and a switched-off gate protects nothing.

And validation runs **before** compilation, so a refused question never becomes SQL:

```

def answer(con, qi: QueryIntent, model: dict) -> tuple[int, str]:
    """Validate, then compile. Never the other way round.

    The refusal is the feature. "I cannot answer that, and here is which rule stopped me"
    is a system you can put in front of a CFO. One that always produces a number is not.
    """
    result = validate(qi, model)
    if not result.ok:
        lines = [f"REFUSED {qi.metric}: the intent did not pass validation.", "",
                 "  gates failed " + ", ".join(g for g, ok in sorted(result.passed.items())
                                              if not ok)]

        for gate_name, why in result.failures:
            lines += _wrap(gate_name, why)
        lines += ["",
                 "  No number is shown. A value that failed a gate is not a caveated",
                 "  answer, it is an unverified one."]
        return 1, "\n".join(lines)

    ans = execute(con, qi, model)
    ans.gates = result.passed
    out = [ans.render()]
    if qi.intent_type == "diagnostic":
        out.append(_diagnose(con, qi, model, ans))
    return 0, "\n\n".join(out)

```

The compiler: the only thing that writes SQL

```
def compile_sql(qi: QueryIntent, model: dict,
               window: tuple[str, str] | None = None) -> str:
    """Deterministic. Same intent in, same SQL out, byte for byte.

    Ratios are compiled as `sum(numerator) / sum(denominator)`, never as an average of
    per-group rates. Averaging rates across groups of unequal size gives a different
    number -- Simpson's paradox -- and no amount of prompting reliably prevents a model
    from doing it. Here it is structurally impossible.
    """
    metric = get_metric(model, qi.metric)
    start, end = window or qi.time_window
    expr = metric["expression"]

    where = [f"{metric['time_dimension']} BETWEEN DATE '{start}' AND DATE '{end}'"]
    where += [e["predicate"] for e in metric.get("exclusions", [])] # unconditional
    for f in qi.filters:
        col = model["_dims_by_name"][f.column]["column"]
        where.append(f"{col} {f.operator} '{f.value}'")

    group = [model["_dims_by_name"][d]["column"] for d in qi.dimensions]
    select = group + [
        f"round(100.0 * {expr['numerator']} / nullif({expr['denominator']}, 0), 4) ",
        f"AS value",
        f"{expr['numerator']} AS numerator",
        f"{expr['denominator']} AS denominator",
    ]
    sql = (f"SELECT {'', '.join(select)}\nFROM {metric['entity']}\n"
          f"WHERE {(chr(10) + ' AND ').join(where)}")
    if group:
        sql += f"\nGROUP BY {'', '.join(group)}\nORDER BY {'', '.join(group)}"
    return sql
```

Two properties, both structural rather than hoped-for. The exclusions are appended unconditionally — no branch on what the question asked, so a question that mentions neither cancellations nor due dates still excludes both. And the ratio is compiled as `sum(numerator) / sum(denominator)`, so averaging rates across unequal groups is not a mistake this code is capable of making.

Running it

The governed answer

1. THE GOVERNED ANSWER

What was OTD for India warehouses last month?

=====

The resolver emitted this, and nothing else:

```
{
  "intent_type": "descriptive",
  "metric": "on_time_delivery_pct",
  "grain": "order",
  "time_window": [
    "2026-07-01",
    "2026-07-31"
  ],
  "dimensions": [],
  "filters": [
    {
      "column": "region",
      "operator": "=",
      "value": "IN"
    }
  ]
}
```

Note `grain` came from the metadata, not from the question.

ANSWER On-Time Delivery % is 87.3% -- 48 of 55 eligible orders delivered on or before the promised date

metric	on_time_delivery_pct
arithmetic	48 / 55 = 87.2727%
grain	order (one row per order)
window	July 2026 (2026-07-01 to 2026-07-31) on promised_delivery_date
filters	region = IN
exclusions	cancelled_orders, not_yet_due
lineage	SAP SD -> SAP EWM -> SAP TM -> fact_order_delivery
trust	TRUSTED
gates	7/7 passed: dimensions_declared, exclusions_applied, filters_bound, grain_matches, metric_approved, no_fanout, time_window_bounded

The provenance block is not decoration. Every line of it is the answer to a question somebody will ask in the meeting: which metric, what arithmetic, at what grain, over what window, with which exclusions, from which systems, and which checks passed.

The same data, counted wrong

2. THE SAME DATA, COUNTED AT DELIVERY GRAIN

=====

This is the query a text-to-SQL system writes. It is syntactically valid, semantically reasonable, and joins orders to legs without collapsing to one row per order:

```
54 / 61 = 88.5246%    <-- reported as 88.5%
```

6 orders in this window shipped in more than one leg, so 6 orders voted twice.
The gap is 1.25 points -- small enough to look like rounding, and it errs UPWARD.
A wrong number that flatters you is the one that survives to the board deck.

That query, refused

3. THAT QUERY, REFUSED BY NAME -- BEFORE ANY SQL EXISTS

=====

REFUSED on_time_delivery_pct: the intent did not pass validation.

```
gates failed grain_matches
grain_matches      Metric 'on_time_delivery_pct' is certified at grain
                   'order'; the intent asks for grain 'delivery'.
                   Aggregating at the wrong grain changes the number.
```

No number is shown. A value that failed a gate is not a caveated answer, it is an unverified one.

A refusal names the gate and the reason.

"I can't answer that, and here is which rule stopped me" is a system you can put in front of a CFO. One that always produces a number is not.

Why it dropped — the question BI tools can't take

Descriptive questions only need a glossary, a grain and a metric. The question people actually ask is diagnostic, and answering it needs per-phase baselines and an attribution rule declared **once, in metadata**:

4. THE QUESTION BI TOOLS CANNOT TAKE

Why did on-time delivery drop for India warehouses last month?

ANSWER On-Time Delivery % is 87.3% -- 48 of 55 eligible orders delivered on or before the promised date

```
metric      on_time_delivery_pct
arithmetic   48 / 55 = 87.2727%
grain        order (one row per order)
window       July 2026 (2026-07-01 to 2026-07-31) on promised_delivery_date
filters      region = IN
exclusions   cancelled_orders, not_yet_due
lineage      SAP SD -> SAP EWM -> SAP TM -> fact_order_delivery
trust        TRUSTED
gates        7/7 passed: dimensions_declared, exclusions_applied, filters_bound, grain_matches,
metric_approved, no_fanout, time_window_bounded
```

BREAKDOWN

Quality inspection	4
Transportation	2
Production execution	1
unattributed	0

Largest contributor: Quality inspection (4 of 7 late orders).

Transportation ran UNDER its standard on 5 of 7 late orders.

Logistics was the obvious suspect and logistics was faster than planned. Without per-phase baselines you get a plausible, confident, wrong story -- and you go optimize the wrong department.

COMPARISON

prior window	June 2026 (2026-06-01 to 2026-06-30) on promised_delivery_date
prior value	96.2% (25 / 26)
change	down 8.9pp (96.2% -> 87.3%)

That last paragraph is the reason to build any of this. Transportation was the obvious suspect, and transportation ran *under* its planned standard on five of the seven late orders. Without per-phase baselines you get a plausible, confident, wrong story — and you go and optimize the wrong department.

Note **unattributed 0** is printed even though it is zero. An attribution that silently drops the orders it could not explain is an attribution nobody can audit.

Four more classes of wrong question

```
5. FOUR MORE CLASSES OF WRONG QUESTION, EACH REFUSED
```

```
=====

-- a filter value outside the declared allowed set
  REFUSED by filters_bound

-- a metric that exists but was never approved
  REFUSED by metric_approved

-- an inverted time window
  REFUSED by time_window_bounded

-- a dimension reachable only through a one-to-many join
  REFUSED by no_fanout
```

`filters_bound` is the quiet one. Without a declared allowed-value set, a filter on a misspelled region returns zero rows — and zero rows gets presented as an answer.

What this file honestly does not do

Governance code invites overreading, so:

- **One metric, one fact table, one grain.** The full project has three approved metrics, nine separate metadata artifacts with different owners, a process model over twelve manufacturing phases, and 401 tests.
- **The per-phase variances are embedded, not derived.** In the full project they come from SAP timestamps in staging SQL. Here they are a table literal — but the attribution *rule* (argmax over declared spans, ties to the earliest phase) is real code, and the file refuses to print a breakdown if the embedded rows ever stop matching the orders it derives as late.
- **The live resolver path is not exercised here.** It needs credentials; the offline resolver stands in. That is a stated gap, not a passing test.
- **This is discovery metadata, not policy enforcement.** An `owner` field records who is accountable. It does not stop anyone from querying anything. Access control, consent and retention are separate systems, and a semantic layer that implies otherwise is worse than one that stays silent.

The complete file

Copy this into `semantic_layer_demo.py` and run it. Nothing else is needed — no repository, no data files, no credentials.

```
"""A governed semantic layer for natural-language querying, in one runnable file.
```

```
    pip install duckdb pyyaml
    python semantic_layer_demo.py
```

No API key, no network, no warehouse to provision. It prints five things:

- | | |
|--|--|
| 1. the grain-correct answer | 87.3% (48 / 55), all 7 gates passed |
| 2. the naive answer the same data yields | 88.5% (54 / 61) |
| 3. that same query REFUSED by name | grain_matches |
| 4. why the number moved | argmax over declared process variances |
| 5. four more gates refusing four more | filters_bound, metric_approved, |
| classes of wrong question | time_window_bounded, no_fanout |

THE ARCHITECTURAL RULE, which everything here follows from:

The model never writes SQL and never does arithmetic. It reads a governed metadata slice and emits a validated JSON object -- which metric, which filters, which time window. A deterministic compiler turns that object into SQL.

Text-to-SQL asks a language model to be right about grain, joins, exclusions and fiscal calendars on every call. This asks it to be right about one much easier thing: which approved metric you meant. `resolve\_offline` does that with keyword matching so this file runs with no credentials; `resolve\_with\_claude` is the real thing and is not called.

WHAT IS HONESTLY ELIDED, because one file has a size limit:

- * The data is the June- and July-2026 India slice of the full project -- 83 orders and 87 delivery legs, verbatim. That is the whole population behind 48/55, 54/61 and the June comparison, so every figure is reproduced exactly rather than approximated.
- * `PHASE\_VARIANCE\_CSV` is derived upstream in the full project from per-phase SAP timestamps (`sql/02\_staging\_model.sql`). Here it is embedded. The attribution rule itself -- argmax over declared spans, ties to the earliest phase -- is real code below, and `\_check\_attribution\_covers\_the\_late\_orders` fails loudly if the embedded rows ever stop matching the orders this file derives as late.
- * One metric, one fact table, one grain. The full project has three metrics, nine metadata artifacts, a process model over twelve manufacturing phases, and 401 tests.

Full project: the nine metadata artifacts, the DQ engine, the diagrams and the test suite live in the repository this file was extracted from.

```
"""
```

```
from __future__ import annotations
```

```
import csv
```

```
import io
```

```
import json
```

```
import re
```

```
import sys
```

```
import textwrap
```

```
from dataclasses import dataclass, field
```

```
from datetime import date
```

```
try:
```

```

import duckdb
import yaml
except ImportError:
    sys.exit("pip install duckdb pyyaml")

# pragma: no cover

# The project is frozen at an as-of date so every number is reproducible. A demo that
# says "last month" relative to today is a demo whose output nobody can check.
AS_OF_DATE = date(2026, 8, 7)

# =====
# 1. THE DATA
# =====
# Two tables, and the difference between them is the entire problem. `orders` has one row
# per customer order. `delivery_legs` has one row per PHYSICAL SHIPMENT -- six of these
# orders shipped in two legs, so joining the two and counting rows counts those orders
# twice. If the first leg arrived before the promised date, the order looks on time twice
# over while the leg that actually blew the promise is one row among 61.

ORDERS_CSV = """\
order_id,region,plant,carrier,promised_delivery_date,order_status
SO-1019,IN,1010,BLUEDART,2026-06-13,DLVD
SO-2040,IN,1010,FEDEX,2026-06-05,CANC
SO-2041,IN,1010,BLUEDART,2026-06-08,DLVD
SO-2042,IN,1010,FEDEX,2026-06-24,DLVD
SO-2043,IN,1010,BLUEDART,2026-06-05,DLVD
SO-2044,IN,1010,BLUEDART,2026-06-29,DLVD
SO-2045,IN,1010,FEDEX,2026-06-17,DLVD
SO-2046,IN,1010,BLUEDART,2026-06-29,DLVD
SO-2047,IN,1010,FEDEX,2026-06-01,DLVD
SO-2048,IN,1010,FEDEX,2026-06-01,DLVD
SO-2049,IN,1010,BLUEDART,2026-06-18,DLVD
SO-2050,IN,1010,BLUEDART,2026-06-30,DLVD
SO-2051,IN,1010,FEDEX,2026-06-29,DLVD
SO-2052,IN,1010,BLUEDART,2026-06-29,DLVD
SO-2053,IN,1010,BLUEDART,2026-06-02,DLVD
SO-2054,IN,1010,FEDEX,2026-06-25,DLVD
SO-2055,IN,1010,FEDEX,2026-06-20,DLVD
SO-2056,IN,1010,FEDEX,2026-06-06,DLVD
SO-2057,IN,1010,FEDEX,2026-06-06,DLVD
SO-2058,IN,1010,BLUEDART,2026-06-05,DLVD
SO-2059,IN,1010,FEDEX,2026-06-15,DLVD
SO-2060,IN,1010,BLUEDART,2026-06-30,DLVD
SO-2061,IN,1010,FEDEX,2026-06-11,DLVD
SO-2062,IN,1010,FEDEX,2026-06-20,DLVD
SO-2063,IN,1010,FEDEX,2026-06-15,DLVD
SO-2064,IN,1010,BLUEDART,2026-06-14,DLVD
SO-2065,IN,1010,BLUEDART,2026-06-07,DLVD
SO-1001,IN,1010,BLUEDART,2026-07-13,DLVD
SO-1002,IN,1010,BLUEDART,2026-07-13,DLVD
SO-1003,IN,1010,FEDEX,2026-07-14,DLVD
SO-1004,IN,1010,BLUEDART,2026-07-14,DLVD
SO-1005,IN,1010,BLUEDART,2026-07-15,DLVD
SO-1006,IN,1010,FEDEX,2026-07-15,DLVD

```

SO-1007, IN, 1010, BLUEDART, 2026-07-16, CANC
SO-1008, IN, 1010, BLUEDART, 2026-07-16, DLVD
SO-1009, IN, 1010, FEDEX, 2026-07-17, DLVD
SO-1010, IN, 1010, BLUEDART, 2026-07-18, DLVD
SO-1011, IN, 1010, BLUEDART, 2026-07-24, DLVD
SO-1012, IN, 1010, FEDEX, 2026-07-19, DLVD
SO-1014, IN, 1010, BLUEDART, 2026-07-20, DLVD
SO-1015, IN, 1010, BLUEDART, 2026-07-21, DLVD
SO-1016, IN, 1010, FEDEX, 2026-07-22, DLVD
SO-1017, IN, 1010, BLUEDART, 2026-07-23, DLVD
SO-1018, IN, 1010, BLUEDART, 2026-07-24, DLVD
SO-2001, IN, 1010, FEDEX, 2026-07-02, DLVD
SO-2002, IN, 1010, FEDEX, 2026-07-30, DLVD
SO-2003, IN, 1010, FEDEX, 2026-07-02, DLVD
SO-2004, IN, 1010, FEDEX, 2026-07-27, DLVD
SO-2005, IN, 1010, FEDEX, 2026-07-16, DLVD
SO-2006, IN, 1010, BLUEDART, 2026-07-29, DLVD
SO-2007, IN, 1010, BLUEDART, 2026-07-30, DLVD
SO-2008, IN, 1010, BLUEDART, 2026-07-08, DLVD
SO-2009, IN, 1010, FEDEX, 2026-07-30, DLVD
SO-2010, IN, 1010, FEDEX, 2026-07-02, DLVD
SO-2011, IN, 1010, BLUEDART, 2026-07-18, DLVD
SO-2012, IN, 1010, BLUEDART, 2026-07-04, DLVD
SO-2013, IN, 1010, BLUEDART, 2026-07-28, DLVD
SO-2014, IN, 1010, BLUEDART, 2026-07-24, DLVD
SO-2015, IN, 1010, BLUEDART, 2026-07-15, DLVD
SO-2016, IN, 1010, BLUEDART, 2026-07-07, DLVD
SO-2017, IN, 1010, BLUEDART, 2026-07-29, DLVD
SO-2018, IN, 1010, BLUEDART, 2026-07-31, DLVD
SO-2019, IN, 1010, BLUEDART, 2026-07-11, DLVD
SO-2020, IN, 1010, BLUEDART, 2026-07-06, DLVD
SO-2021, IN, 1010, FEDEX, 2026-07-23, DLVD
SO-2022, IN, 1010, BLUEDART, 2026-07-22, DLVD
SO-2023, IN, 1010, BLUEDART, 2026-07-24, DLVD
SO-2024, IN, 1010, BLUEDART, 2026-07-09, DLVD
SO-2025, IN, 1010, FEDEX, 2026-07-27, DLVD
SO-2026, IN, 1010, FEDEX, 2026-07-25, DLVD
SO-2027, IN, 1010, FEDEX, 2026-07-05, DLVD
SO-2028, IN, 1010, FEDEX, 2026-07-27, DLVD
SO-2029, IN, 1010, FEDEX, 2026-07-25, DLVD
SO-2030, IN, 1010, BLUEDART, 2026-07-09, DLVD
SO-2031, IN, 1010, BLUEDART, 2026-07-07, DLVD
SO-2032, IN, 1010, BLUEDART, 2026-07-08, DLVD
SO-2033, IN, 1010, FEDEX, 2026-07-30, DLVD
SO-2034, IN, 1010, BLUEDART, 2026-07-04, DLVD
SO-2035, IN, 1010, FEDEX, 2026-07-09, DLVD
SO-2036, IN, 1010, BLUEDART, 2026-07-17, DLVD
SO-2037, IN, 1010, BLUEDART, 2026-07-24, DLVD
SO-2038, IN, 1010, BLUEDART, 2026-07-30, DLVD
SO-2039, IN, 1010, BLUEDART, 2026-07-25, DLVD
""

DELIVERY_LEGS_CSV = ""\
delivery_id,order_id,delivery_leg,delivery_date

DL-1019, SO-1019, 1, 2026-06-18
DL-2041, SO-2041, 1, 2026-06-07
DL-2042, SO-2042, 1, 2026-06-23
DL-2043, SO-2043, 1, 2026-06-04
DL-2044, SO-2044, 1, 2026-06-28
DL-2045, SO-2045, 1, 2026-06-16
DL-2046, SO-2046, 1, 2026-06-28
DL-2047, SO-2047, 1, 2026-05-31
DL-2048, SO-2048, 1, 2026-05-31
DL-2049, SO-2049, 1, 2026-06-17
DL-2050, SO-2050, 1, 2026-06-29
DL-2051, SO-2051, 1, 2026-06-28
DL-2052, SO-2052, 1, 2026-06-28
DL-2053, SO-2053, 1, 2026-06-01
DL-2054, SO-2054, 1, 2026-06-24
DL-2055, SO-2055, 1, 2026-06-19
DL-2056, SO-2056, 1, 2026-06-05
DL-2057, SO-2057, 1, 2026-06-05
DL-2058, SO-2058, 1, 2026-06-04
DL-2059, SO-2059, 1, 2026-06-14
DL-2060, SO-2060, 1, 2026-06-29
DL-2061, SO-2061, 1, 2026-06-10
DL-2062, SO-2062, 1, 2026-06-19
DL-2063, SO-2063, 1, 2026-06-14
DL-2064, SO-2064, 1, 2026-06-13
DL-2065, SO-2065, 1, 2026-06-06
DL-1001, SO-1001, 1, 2026-07-12
DL-1002, SO-1002, 1, 2026-07-16
DL-1003, SO-1003, 1, 2026-07-19
DL-1004, SO-1004, 1, 2026-07-18
DL-1005, SO-1005, 1, 2026-07-18
DL-1006, SO-1006, 1, 2026-07-19
DL-1008, SO-1008, 1, 2026-07-15
DL-1009A, SO-1009, 1, 2026-07-16
DL-1009B, SO-1009, 2, 2026-07-20
DL-1010, SO-1010, 1, 2026-07-22
DL-1011, SO-1011, 1, 2026-07-21
DL-1012A, SO-1012, 1, 2026-07-18
DL-1012B, SO-1012, 2, 2026-07-18
DL-1014A, SO-1014, 1, 2026-07-19
DL-1014B, SO-1014, 2, 2026-07-19
DL-1015, SO-1015, 1, 2026-07-19
DL-1016A, SO-1016, 1, 2026-07-21
DL-1016B, SO-1016, 2, 2026-07-21
DL-1017A, SO-1017, 1, 2026-07-22
DL-1017B, SO-1017, 2, 2026-07-22
DL-1018A, SO-1018, 1, 2026-07-23
DL-1018B, SO-1018, 2, 2026-07-23
DL-2001, SO-2001, 1, 2026-07-01
DL-2002, SO-2002, 1, 2026-07-29
DL-2003, SO-2003, 1, 2026-07-01
DL-2004, SO-2004, 1, 2026-07-26
DL-2005, SO-2005, 1, 2026-07-15
DL-2006, SO-2006, 1, 2026-07-28

```
DL-2007,SO-2007,1,2026-07-29
DL-2008,SO-2008,1,2026-07-07
DL-2009,SO-2009,1,2026-07-29
DL-2010,SO-2010,1,2026-07-01
DL-2011,SO-2011,1,2026-07-17
DL-2012,SO-2012,1,2026-07-03
DL-2013,SO-2013,1,2026-07-27
DL-2014,SO-2014,1,2026-07-23
DL-2015,SO-2015,1,2026-07-14
DL-2016,SO-2016,1,2026-07-06
DL-2017,SO-2017,1,2026-07-28
DL-2018,SO-2018,1,2026-07-30
DL-2019,SO-2019,1,2026-07-10
DL-2020,SO-2020,1,2026-07-05
DL-2021,SO-2021,1,2026-07-22
DL-2022,SO-2022,1,2026-07-21
DL-2023,SO-2023,1,2026-07-23
DL-2024,SO-2024,1,2026-07-08
DL-2025,SO-2025,1,2026-07-26
DL-2026,SO-2026,1,2026-07-24
DL-2027,SO-2027,1,2026-07-04
DL-2028,SO-2028,1,2026-07-26
DL-2029,SO-2029,1,2026-07-24
DL-2030,SO-2030,1,2026-07-08
DL-2031,SO-2031,1,2026-07-06
DL-2032,SO-2032,1,2026-07-07
DL-2033,SO-2033,1,2026-07-29
DL-2034,SO-2034,1,2026-07-03
DL-2035,SO-2035,1,2026-07-08
DL-2036,SO-2036,1,2026-07-16
DL-2037,SO-2037,1,2026-07-23
DL-2038,SO-2038,1,2026-07-29
DL-2039,SO-2039,1,2026-07-24
"""
```

```
# variance_days = actual_duration - standard_duration, per manufacturing phase, for the
# orders that missed. Negative means the phase ran UNDER its planned standard. Only late
# orders are attributed: asking "which phase caused the delay" of an on-time order is a
# question with no answer, and inventing one is how dashboards acquire fake precision.
PHASE_VARIANCE_CSV = """\
order_id,mrp,production_order_release,material_staging,production_execution,quality_inspection,goods_rece
ipt,picking,dispatch,transportation,available_slack_days
SO-1019,0,0,0,0,0,0,0,0,5,0
SO-1002,0,0,0,0,4,0,0,0,-1,0
SO-1003,0,0,0,0,0,0,0,0,5,0
SO-1004,0,0,0,0,5,0,0,0,-1,0
SO-1005,0,0,0,4,0,0,0,0,-1,0
SO-1006,0,0,0,0,5,0,0,0,-1,0
SO-1009,0,0,0,0,0,0,0,0,3,0
SO-1010,0,0,0,0,5,0,0,0,-1,0
"""
```

```
# =====
```

2. THE METADATA

```
# =====  
# In the full project this is nine separate governed artifacts with different owners and  
# different review cadences. Collapsed here into one document, but the fields are the  
# real ones, and the field that matters most is `grain`.  
#  
# Read `metrics[0]` and ask: which of these could a crawler pointed at the warehouse  
# have produced? `entity` and `time_dimension`, maybe. Not `grain`, not `exclusions`,  
# not `business_rules`, not `approval_state`. Those are decisions, made by a named human,  
# and they are exactly the fields that decide whether an answer is right rather than  
# merely well-formed.
```

```
SEMANTIC_MODEL_YAML = """
```

```
entities:
```

- name: fact_order_delivery
 grain: order # one row per order_id -- the load-bearing declaration
 primary_key: order_id
 joins:
 - to: raw_delivery_legs
 on: order_id
 cardinality: one_to_many # this is the hop that turns 87.27% into 88.52%
- name: raw_delivery_legs
 grain: delivery
 primary_key: delivery_id

```
dimensions:
```

- name: region
 entity: fact_order_delivery
 column: region
 allowed_values: [IN, US, EU, APAC] # without this, region='MARS' returns zero rows
- name: plant
 entity: fact_order_delivery
 column: plant
 allowed_values: ["1010", "1710"]
- name: carrier
 entity: fact_order_delivery
 column: carrier
 allowed_values: [BLUEDART, FEDEX, DHL]
- name: delivery_id
 entity: raw_delivery_legs # only reachable through the one_to_many join
 column: delivery_id

```
metrics:
```

- name: on_time_delivery_pct
 label: On-Time Delivery %
 description: >
 Share of eligible customer orders delivered on or before the promised delivery
 date. An order counts as delivered only when its final leg arrives.
 entity: fact_order_delivery
 grain: order
 time_dimension: promised_delivery_date
 expression:
 type: ratio_percent


```

    numerator: "sum(CASE WHEN is_on_time THEN 1 ELSE 0 END)"
    denominator: "sum(CASE WHEN is_eligible THEN 1 ELSE 0 END)"
exclusions:
  - key: cancelled_orders
    predicate: "order_status <> 'CANC'"
    rationale: A cancelled order was never due; including it inflates the denominator.
  - key: not_yet_due
    predicate: "promised_delivery_date <= DATE '__AS_OF_DATE__'"
    rationale: >
      An order promised after the as-of date is not late, it is pending. Counting it
      as a miss understates performance.
business_rules:
  - Measured against the CURRENT promised date, not the original.
  - Order-grain only. Delivery-grain aggregation double-counts split shipments.
owner: VP Supply Chain
approval_state: approved
lineage: [SAP SD, SAP EWM, SAP TM, fact_order_delivery]

```

```

- name: production_cycle_time_days
  label: Production Cycle Time (days)
  entity: fact_order_delivery
  grain: order
  time_dimension: promised_delivery_date
  expression: {type: average, numerator: "avg(0)", denominator: "1"}
  approval_state: draft          # deliberately unapproved: the retriever never offers it
  owner: VP Manufacturing

```

Vocabulary. The cheapest phase to build and the one whose absence quietly breaks NL
 # querying: a user who types "OTIF" is asking for a governed metric, and without this
 # the resolver has nothing to map it to.

glossary:

```

- term: On-Time Delivery %
  metric: on_time_delivery_pct
  definition: >
    Share of eligible customer orders delivered on or before the promised delivery
    date. Measured at order grain (one row per order_id). An order counts as delivered
    only when its final delivery leg arrives. Excludes cancelled orders and orders not
    yet due.
  synonyms: [OTD, on time delivery, on-time delivery, delivery performance,
    delivery reliability, promise adherence, OTIF, on time in full]

```

The process model is what makes "why did it drop" answerable. Nine attributable spans
 # with a planned standard each, and ONE attribution rule stated once, in metadata, rather
 # than reinvented per dashboard.

process:

```

  attribution_rule: variance_argmax
  tie_break: earliest_phase_seq
  variances:
    - {phase_seq: 2, key: mrp, label: MRP,
      standard_duration_days: 1}
    - {phase_seq: 3, key: production_order_release, label: Production order release,
      standard_duration_days: 1}
    - {phase_seq: 4, key: material_staging, label: Material staging,
      standard_duration_days: 1}

```

```

- {phase_seq: 5, key: production_execution, label: Production execution,
standard_duration_days: 3}
- {phase_seq: 6, key: quality_inspection, label: Quality inspection,
standard_duration_days: 1}
- {phase_seq: 7, key: goods_receipt, label: Goods receipt,
standard_duration_days: 1}
- {phase_seq: 8, key: picking, label: Picking,
standard_duration_days: 1}
- {phase_seq: 9, key: dispatch, label: Dispatch,
standard_duration_days: 1}
- {phase_seq: 10, key: transportation, label: Transportation,
standard_duration_days: 2}
"""

```

```
def load_model() -> dict:
```

```

    """Parse the governed metadata. The as-of date is substituted at read time, so the
    document itself never carries a date that can silently expire."""

```

```
    text = SEMANTIC_MODEL_YAML.replace("__AS_OF_DATE__", AS_OF_DATE.isoformat())
```

```
    model = yaml.safe_load(text)
```

```
    model["_metrics_by_name"] = {m["name"]: m for m in model["metrics"]}
```

```
    model["_dims_by_name"] = {d["name"]: d for d in model["dimensions"]}
```

```
    model["_entities_by_name"] = {e["name"]: e for e in model["entities"]}
```

```
    return model
```

```
def get_metric(model: dict, name: str) -> dict | None:
```

```
    return model["_metrics_by_name"].get(name)
```

```
# =====
```

```
# 3. THE WAREHOUSE
```

```
# =====
```

```
def build_warehouse() -> duckdb.DuckDBPyConnection:
```

```
    """In-memory DuckDB. The one interesting line is the aggregation to ORDER grain.
```

```

    `final_delivery_date = max(delivery_date)` is what "an order counts as delivered only
    when its final leg arrives" means in SQL. Taking min() -- or joining without
    aggregating at all -- is the 88.5% bug, and it is a one-word difference.

```

```
    """
```

```
    con = duckdb.connect(":memory:")
```

```
    variance_cols = PHASE_VARIANCE_CSV.splitlines()[0].split(",")[1:]
```

```
    for name, ddl, csv_text in (
```

```
        ("raw_orders",
```

```
        "order_id VARCHAR, region VARCHAR, plant VARCHAR, carrier VARCHAR, "
```

```
        "promised_delivery_date DATE, order_status VARCHAR", ORDERS_CSV),
```

```
        ("raw_delivery_legs",
```

```
        "delivery_id VARCHAR, order_id VARCHAR, delivery_leg INTEGER, "
```

```
        "delivery_date DATE", DELIVERY_LEGS_CSV),
```

```
        ("raw_phase_variance",
```

```
        "order_id VARCHAR, " + ", ".join(f"{c} INTEGER" for c in variance_cols),
```

```
        PHASE_VARIANCE_CSV),
```

```

):
    _load(con, name, ddl, csv_text)

con.execute(f"""
CREATE TABLE fact_order_delivery AS
WITH final_leg AS (
    SELECT order_id,
           max(delivery_date) AS final_delivery_date,
           count(*)           AS delivery_count
    FROM raw_delivery_legs
    GROUP BY order_id          -- <-- collapse to one row per ORDER
)
SELECT o.order_id, o.region, o.plant, o.carrier,
       o.promised_delivery_date, o.order_status,
       f.final_delivery_date,
       coalesce(f.delivery_count, 0) AS delivery_count,
       -- The two exclusions, materialised as a single eligibility flag.
       (o.order_status <> 'CANC'
        AND o.promised_delivery_date <= DATE '{AS_OF_DATE}') AS is_eligible,
       (f.final_delivery_date IS NOT NULL
        AND f.final_delivery_date <= o.promised_delivery_date) AS is_on_time
    FROM raw_orders o
    LEFT JOIN final_leg f USING (order_id)
""")
return con

```

```

def _load(con: duckdb.DuckDBPyConnection, table: str, ddl: str, csv_text: str) -> None:
    """Explicit DDL rather than `read_csv_auto`, so `promised_delivery_date` is a DATE and
    not a string that compares lexicographically and looks like it worked."""
    rows = list(csv.reader(io.StringIO(csv_text)))
    con.execute(f"CREATE TABLE {table} ({ddl})")
    marks = ", ".join("?" * len(rows[0]))
    con.executemany(f"INSERT INTO {table} VALUES ({marks})", rows[1:])

```

```

# =====
# 4. THE INTENT -- the only thing the model is allowed to produce
# =====

```

```

@dataclass(frozen=True)
class Filter:
    column: str
    operator: str      # one of = in between >= <=
    value: object

```

```

@dataclass(frozen=True)
class QueryIntent:
    """What a language model is permitted to emit. Note what is NOT in here: no SQL, no
    column expressions, no arithmetic, no join instructions. Every field is a reference
    to something the metadata already declares, which is what makes it checkable."""
    intent_type: str      # descriptive | diagnostic

```

```

metric: str
grain: str
time_window: tuple[str, str]
time_label: str = ""
dimensions: tuple[str, ...] = ()
filters: tuple[Filter, ...] = ()
compare_to: tuple[str, str] | None = None
compare_label: str = ""

def to_json(self) -> str:
    return json.dumps({
        "intent_type": self.intent_type, "metric": self.metric, "grain": self.grain,
        "time_window": list(self.time_window), "dimensions": list(self.dimensions),
        "filters": [{"column": f.column, "operator": f.operator, "value": f.value}
                     for f in self.filters],
    }, indent=2)

```

```

# =====
# 5. THE RESOLVER -- the only stage that calls a model
# =====

```

```

def resolve_offline(question: str, model: dict) -> QueryIntent:
    """Keyword resolution, so this file runs with no credentials.

```

This is a stand-in for `resolve\_with\_claude` below, and the substitution is the point: every stage downstream receives a `QueryIntent` and cannot tell which produced it. If swapping the resolver changed anything about how the answer is computed, the architecture would be leaking.

```

    """
    q = question.lower()

```

```

    metric = None
    for term in model["glossary"]:
        candidates = [term["term"]] + term["synonyms"]
        if any(c.lower() in q for c in candidates):
            metric = term["metric"]
            break
    if metric is None:
        raise ValueError(
            f"No governed metric matches {question!r}. The glossary knows: "
            + ", ".join(sorted({s for t in model['glossary'] for s in t['synonyms']})))
    )

```

```

# Two traps here, both of which this code originally fell into, and both of which are
# about the DIMENSION VALUES rather than about the metric:

```

```

#
# substring: "IN" occurs inside "inspection"
# stopword: "IN" IS an English word -- "delivery reliability for FEDEX in APAC"
#           resolves to region=IN if the code is matched case-insensitively
#
# So the region CODE is matched case-sensitively against the original question, and
# the human words are matched case-insensitively as declared aliases. Note that both

```

```

# bugs produce a perfectly LEGAL filter, which every gate downstream passes -- the
# gates protect the arithmetic, not the reading comprehension. That asymmetry is the
# reason the real resolver is a model handed `allowed_values`, and the reason it is
# confined to choosing rather than computing.
aliases = {"IN": ["india", "indian"], "EU": ["europe", "european", "emea"],
          "US": ["usa", "united states", "north america"],
          "APAC": ["asia pacific", "asia-pacific"]}
filters = []
for dim in ("region", "plant", "carrier"):
    for allowed in model["_dims_by_name"][dim]["allowed_values"]:
        hit = re.search(rf"\b{re.escape(allowed)}\b", question) or any(
            re.search(rf"\b{re.escape(a)}\b", q) for a in aliases.get(allowed, []))
        if hit:
            filters.append(Filter(dim, "=", allowed))
            break

diagnostic = any(w in q for w in ("why", "cause", "reason", "drop", "driver"))
return QueryIntent(
    intent_type="diagnostic" if diagnostic else "descriptive",
    metric=metric,
    grain=get_metric(model, metric)["grain"], # from METADATA, never from the text
    time_window=("2026-07-01", "2026-07-31"),
    time_label="July 2026",
    filters=tuple(filters),
    compare_to=("2026-06-01", "2026-06-30") if diagnostic else None,
    compare_label="June 2026" if diagnostic else "",
)

```

```

RESOLVER_SYSTEM_PROMPT = """\

```

```

You select a governed metric. You do not write SQL and you do not do arithmetic.

```

```

Return ONLY a QueryIntent. Every value must come from the metadata slice provided:

```

- metric: an exact name from the approved metric list
- grain: the metric's declared grain, copied verbatim, never inferred from the question
- filters: only columns listed as dimensions, only values from allowed_values
- time_window: explicit start and end dates, never open-ended

```

If the question cannot be answered from this slice, say so instead of approximating.

```

```

An approximation that looks like an answer is worse than a refusal.

```

```

"""

```

```

def resolve_with_claude(question: str, model: dict) -> QueryIntent: # pragma: no cover
    """The real resolver. Not called by `main` -- shown because it is the whole point.

```

```

Two things make this safe, and neither is prompt wording. First, the retriever hands
over only APPROVED metrics, so an unapproved one cannot be picked: it was never
offered. Second, whatever comes back goes through all seven gates before it can
become SQL, so a wrong intent is refused rather than answered.

```

```

"""

```

```

import anthropic

```

```

approved = [m for m in model["metrics"] if m["approval_state"] == "approved"]

```

```

slice_for_model = {
    "metrics": [{k: m[k] for k in ("name", "label", "grain", "description")
                    if k in m} for m in approved],
    "dimensions": [{"name": d["name"], "allowed_values": d.get("allowed_values")}
                    for d in model["dimensions"]],
    "glossary": [{"term": t["term"], "metric": t["metric"],
                    "synonyms": t["synonyms"]} for t in model["glossary"]],
}

tool = {
    "name": "emit_query_intent",
    "description": "Emit the governed query intent for this question.",
    "input_schema": {
        "type": "object",
        "properties": {
            "intent_type": {"enum": ["descriptive", "diagnostic"]},
            "metric": {"enum": [m["name"] for m in approved]},
            "grain": {"type": "string"},
            "time_window": {"type": "array", "items": {"type": "string"},
                            "minItems": 2, "maxItems": 2},
            "dimensions": {"type": "array", "items": {"type": "string"}},
            "filters": {"type": "array", "items": {
                "type": "object",
                "properties": {"column": {"type": "string"},
                               "operator": {"enum": ["=", "in", "between",
                                                       ">=", "<="]},
                               "value": {}}},
                "required": ["column", "operator", "value"]}},
        },
        "required": ["intent_type", "metric", "grain", "time_window"],
    },
}

reply = anthropic.Anthropic().messages.create(
    model="claude-sonnet-5",
    max_tokens=1024,
    system=RESOLVER_SYSTEM_PROMPT,
    tools=[tool],
    tool_choice={"type": "tool", "name": "emit_query_intent"},
    messages=[{"role": "user", "content":
        f"Metadata slice:\n{json.dumps(slice_for_model, indent=2)}\n\n"
        f"Question: {question}"}],
)

args = next(b.input for b in reply.content if b.type == "tool_use")
return QueryIntent(
    intent_type=args["intent_type"], metric=args["metric"], grain=args["grain"],
    time_window=tuple(args["time_window"]),
    dimensions=tuple(args.get("dimensions", ())),
    filters=tuple(Filter(f["column"], f["operator"], f["value"])
                    for f in args.get("filters", ())),
)

```

```

# =====
# 6. THE VALIDATOR -- seven gates, between the model's JSON and any SQL
# =====

```

```

# Three rules that are not negotiable:
#
# * All seven gates always run. A user deserves every reason at once, not the first.
# * A gate reports pass only when it actually verified its property. A gate that could
#   not be evaluated FAILS CLOSED -- "I did not check" must never be recorded as
#   "I checked and it was fine".
# * Validation runs BEFORE compilation, so a refused question never becomes SQL.

GATE_NAMES = ("metric_approved", "dimensions_declared", "filters_bound", "grain_matches",
              "no_fanout", "time_window_bounded", "exclusions_applied")

@dataclass
class ValidationResult:
    passed: dict[str, bool] = field(default_factory=dict)
    failures: list[tuple[str, str]] = field(default_factory=list)

    @property
    def ok(self) -> bool:
        return len(self.passed) == len(GATE_NAMES) and all(self.passed.values())

def validate(qi: QueryIntent, model: dict) -> ValidationResult:
    """Never raises. A hallucinated metric name is a normal, expected input."""
    r = ValidationResult()
    metric = get_metric(model, qi.metric)

    def gate(name: str, ok: bool, why: str = "") -> None:
        r.passed[name] = ok
        if not ok:
            r.failures.append((name, why))

    gate("metric_approved",
        metric is not None and metric.get("approval_state") == "approved",
        f"Metric {qi.metric!r} is not an approved metric. Approved: "
        + ", ".join(m["name"] for m in model["metrics"]
                    if m["approval_state"] == "approved"))

    undeclared = [d for d in qi.dimensions if d not in model["_dims_by_name"]]
    gate("dimensions_declared", not undeclared,
        f"Dimensions {undeclared} are not declared in the semantic model.")

    unbound = [f.column for f in qi.filters if f.column not in model["_dims_by_name"]]
    outside = [f"{f.column}={f.value!r}" for f in qi.filters
               if f.column in model["_dims_by_name"]
               and model["_dims_by_name"][f.column].get("allowed_values")
               and f.value not in model["_dims_by_name"][f.column]["allowed_values"]]
    gate("filters_bound", not unbound and not outside,
        f"Filter columns {unbound} do not bind to a declared dimension."
        if unbound else
        f"Filter {'', '.join(outside)} is outside the declared allowed_values. Without "
        f"this gate the query returns zero rows, and zero rows gets read as an answer.")

    if metric is None:
        gate("grain_matches", False,

```

```

        f"Cannot check grain: metric {qi.metric!r} is not in the semantic model, so "
        f"there is no certified grain to compare {qi.grain!r} against.")
    else:
        gate("grain_matches", qi.grain == metric["grain"],
            f"Metric {qi.metric!r} is certified at grain {metric['grain']!r}; the intent "
            f"asks for grain {qi.grain!r}. Aggregating at the wrong grain changes the "
            f"number.")

    fanout = []
    if metric is not None:
        home = metric["entity"]
        for name in list(qi.dimensions) + [f.column for f in qi.filters]:
            dim = model["_dims_by_name"].get(name)
            if dim is None or dim["entity"] == home:
                continue
            join = next((j for j in model["_entities_by_name"][home].get("joins", []))
                        if j["to"] == dim["entity"]), None)
            # A many_to_one hop cannot duplicate rows and must pass. Only one_to_many
            # fans out. A gate that blocked every join would look correct and quietly
            # break every legitimate cross-entity question.
            if join is None or join["cardinality"] == "one_to_many":
                fanout.append(f"{name} (on {dim['entity']})")
    gate("no_fanout", not fanout,
        f"Reaching {'', '.join(fanout)} requires a one_to_many join, which duplicates "
        f"rows before aggregation.")

    start, end = qi.time_window
    gate("time_window_bounded", bool(start and end) and start <= end,
        f"Time window {qi.time_window} is unbounded or inverted.")

    gate("exclusions_applied",
        metric is not None and all(e.get("predicate") for e in
                                    metric.get("exclusions", [])),
        f"Metric {qi.metric!r} declares an exclusion with no predicate, so it cannot be "
        f"applied. A metric whose exclusions cannot be enforced must not be answered.")
    return r

```

```

# =====
# 7. THE COMPILER -- the only thing that writes SQL
# =====

```

```

def compile_sql(qi: QueryIntent, model: dict,
                window: tuple[str, str] | None = None) -> str:
    """Deterministic. Same intent in, same SQL out, byte for byte.

```

Ratios are compiled as ``sum(numerator) / sum(denominator)``, never as an average of per-group rates. Averaging rates across groups of unequal size gives a different number -- Simpson's paradox -- and no amount of prompting reliably prevents a model from doing it. Here it is structurally impossible.

```

"""

```

```

metric = get_metric(model, qi.metric)
start, end = window or qi.time_window

```



```

expr = metric["expression"]

where = [f"{metric['time_dimension']} BETWEEN DATE '{start}' AND DATE '{end}'"]
where += [e["predicate"] for e in metric.get("exclusions", [])] # unconditional
for f in qi.filters:
    col = model["_dims_by_name"][f.column]["column"]
    where.append(f"{col} {f.operator} '{f.value}'")

group = [model["_dims_by_name"][d]["column"] for d in qi.dimensions]
select = group + [
    f"round(100.0 * {expr['numerator']} / nullif({expr['denominator']}, 0), 4) ",
    f"AS value",
    f"{expr['numerator']} AS numerator",
    f"{expr['denominator']} AS denominator",
]
sql = (f"SELECT {'', '.join(select)}\nFROM {metric['entity']}\n"
      f"WHERE {(chr(10) + ' AND ').join(where)}")
if group:
    sql += f"\nGROUP BY {'', '.join(group)}\nORDER BY {'', '.join(group)}"
return sql

```

```

# =====
# 8. EXECUTOR + PROVENANCE
# =====

```

```

@dataclass
class Answer:
    value: float
    numerator: int
    denominator: int
    sql: str
    intent: QueryIntent
    metric: dict
    gates: dict[str, bool]

    def render(self) -> str:
        m, qi = self.metric, self.intent
        filters = ", ".join(f"{f.column} {f.operator} {f.value}"
                           for f in qi.filters) or "none"
        headline = textwrap.wrap(
            f"{m['label']} is {self.value:.1f}% -- {self.numerator} of "
            f"{self.denominator} eligible orders delivered on or before the promised "
            f"date", 71)
        return "\n".join([
            f"ANSWER {headline[0]}",
            *(f" {ln}" for ln in headline[1:]),
            "",
            f" metric {m['name']}",
            f" arithmetic {self.numerator} / {self.denominator} = "
            f"{100 * self.numerator / self.denominator:.4f}%",
            f" grain {m['grain']} (one row per {m['grain']})",
            f" window {qi.time_label or '-'}"

```

```

        f"({qi.time_window[0]} to {qi.time_window[1]}) on {m['time_dimension']}",
        f" filters      {filters}",
        f" exclusions    {'', '.join(e['key'] for e in m.get('exclusions', []))}",
        f" lineage        {'' -> '.join(m['lineage'])}",
        f" trust          TRUSTED",
        f" gates           {sum(self.gates.values())}/{len(self.gates)} passed: "
        + ", ".join(sorted(self.gates)),
    ])

```

```

def execute(con, qi: QueryIntent, model: dict,
            window: tuple[str, str] | None = None) -> Answer:
    """An empty window yields NULLs, not zeros. Zero is a measurement; NULL is the
    absence of one, and coercing it to 0.0% would report a catastrophe that never
    happened."""
    sql = compile_sql(qi, model, window)
    value, num, den = con.execute(sql).fetchone()
    return Answer(float(value) if value is not None else float("nan"),
                  int(num or 0), int(den or 0), sql, qi,
                  get_metric(model, qi.metric), {})

def answer(con, qi: QueryIntent, model: dict) -> tuple[int, str]:
    """Validate, then compile. Never the other way round.

    The refusal is the feature. "I cannot answer that, and here is which rule stopped me"
    is a system you can put in front of a CFO. One that always produces a number is not.
    """
    result = validate(qi, model)
    if not result.ok:
        lines = [f"REFUSED {qi.metric}: the intent did not pass validation.", "",
                 " gates failed " + ", ".join(g for g, ok in sorted(result.passed.items())
                                                if not ok)]

        for gate_name, why in result.failures:
            lines += _wrap(gate_name, why)
        lines += ["",
                 " No number is shown. A value that failed a gate is not a caveated",
                 " answer, it is an unverified one."]
        return 1, "\n".join(lines)

    ans = execute(con, qi, model)
    ans.gates = result.passed
    out = [ans.render()]
    if qi.intent_type == "diagnostic":
        out.append(_diagnose(con, qi, model, ans))
    return 0, "\n\n".join(out)

def _wrap(label: str, text: str, width: int = 78) -> list[str]:
    body = textwrap.wrap(text, width - 23)
    return [f" {label:<20} {body[0]}" + [f" {'':<20} {ln}" for ln in body[1:]]

```

```

# =====

```

```

# 9. THE DIAGNOSTIC -- the question BI tools cannot take
# =====
# "What was it?" needs a glossary, a grain and a metric. "WHY did it drop?" needs
# per-phase baselines and an attribution rule declared once, in metadata. This is why
# the manufacturing process is in scope at all: the answer to "why" lives upstream in
# production, not in the delivery table.

def _diagnose(con, qi: QueryIntent, model: dict, ans: Answer) -> str:
    spans = model["process"]["variances"]
    _check_attribution_covers_the_late_orders(con, qi, model)

    cur = con.execute(f"""
        SELECT v.* FROM raw_phase_variance v
        JOIN fact_order_delivery f USING (order_id)
        WHERE f.is_eligible AND NOT f.is_on_time
              AND f.promised_delivery_date BETWEEN DATE '{qi.time_window[0]}'
              AND DATE '{qi.time_window[1]}'
    """)
    names = [d[0] for d in cur.description]
    rows = [dict(zip(names, r)) for r in cur.fetchall()]

    # The attribution rule, applied. argmax over the declared spans; ties break to the
    # earliest phase_seq so the answer is stable rather than dependent on dict order.
    tally: dict[str, int] = {}
    for row in rows:
        best = max(spans, key=lambda s: (row[s["key"]], -s["phase_seq"]))
        key = best["label"] if row[best["key"]] > 0 else "unattributed"
        tally[key] = tally.get(key, 0) + 1

    # Largest contributor first, ties broken by phase_seq so the order is deterministic
    # rather than dependent on which order the rows happened to arrive in.
    seq = {s["label"]: s["phase_seq"] for s in spans}
    out = [" BREAKDOWN"]
    for label in sorted(tally, key=lambda k: (-tally[k], seq.get(k, 99))):
        if label != "unattributed":
            out.append(f"    {label:<28} {tally[label]}")
    # Printed even when zero, on purpose: an attribution that silently drops the orders it
    # could not explain is an attribution nobody can audit.
    out.append(f"    {'unattributed':<28} {tally.get('unattributed', 0)}")
    top = max((k for k in tally if k != "unattributed"), key=lambda k: tally[k])
    out += ["", f" Largest contributor: {top} ({tally[top]} of {len(rows)} late orders)."]

    # The finding that makes the case, and it is the opposite of the obvious suspect.
    under = [r for r in rows if r["transportation"] < 0]
    if under:
        out += ["", f" Transportation ran UNDER its standard on {len(under)} of "
                f"{len(rows)} late orders.",
                " Logistics was the obvious suspect and logistics was faster than",
                " planned. Without per-phase baselines you get a plausible, confident,",
                " wrong story -- and you go optimize the wrong department."]

    if qi.compare_to:
        # The prior window is compiled from the SAME metric definition, so the two numbers

```

```

# are comparable by construction. A comparison assembled from two hand-written
# queries is where "down 8.9 points" turns out to mean two different things.
prior = execute(con, qi, model, qi.compare_to)
out += ["", "  COMPARISON",
        f"    {'prior window':<26} {qi.compare_label} "
        f"({qi.compare_to[0]} to {qi.compare_to[1]}) on "
        f"{ans.metric['time_dimension']}"]
if prior.denominator:
    direction = "down" if prior.value > ans.value else "up"
    out += [f"    {'prior value':<26} {prior.value:.1f}% "
            f"({prior.numerator} / {prior.denominator})",
            f"    {'change':<26} {direction} "
            f"{abs(prior.value - ans.value):.1f}pp "
            f"({prior.value:.1f}% -> {ans.value:.1f}%)"]
else:
    out += [f"    {'prior value':<26} no eligible orders in the prior window",
            f"    {'change':<26} not computable"]
return "\n".join(out)

```

```

def _check_attribution_covers_the_late_orders(con, qi: QueryIntent, model: dict) -> None:
    """Fail loudly rather than attribute the wrong set.

```

`PHASE\_VARIANCE\_CSV` is embedded, while lateness is DERIVED from the order and leg tables. If those two ever disagree, the breakdown would silently describe a different population than the headline number. That is precisely the class of defect this whole project exists to prevent, so it is checked rather than assumed.

```

window = f"""promised_delivery_date BETWEEN DATE '{qi.time_window[0]}'
              AND DATE '{qi.time_window[1]}'''

derived = {r[0] for r in con.execute(f"""
    SELECT order_id FROM fact_order_delivery
    WHERE is_eligible AND NOT is_on_time AND {window}""").fetchall()}
embedded = {r[0] for r in con.execute(f"""
    SELECT v.order_id FROM raw_phase_variance v
    JOIN fact_order_delivery f USING (order_id) WHERE {window}""").fetchall()}
if derived != embedded:
    raise AssertionError(
        f"attribution population mismatch -- derived late orders {sorted(derived)} "
        f"but PHASE_VARIANCE_CSV has {sorted(embedded)}"
    )

```

```

# =====
# 10. THE DEMONSTRATION
# =====

```

```

def _rule(title: str) -> str:
    return f"\n{' ' * 78}\n{title}\n{' ' * 78}"

```

```

def main() -> int:
    model = load_model()

```

```

con = build_warehouse()
failures = 0

# ---- 1. The governed answer -----
question = "What was OTD for India warehouses last month?"
print(_rule(f"1. THE GOVERNED ANSWER\n {question}"))
qi = resolve_offline(question, model)
print(f"\nThe resolver emitted this, and nothing else:\n{qi.to_json()}")
print(f"\nNote 'grain' came from the metadata, not from the question.\n")
rc, out = answer(con, qi, model)
print(out)
failures += rc
governed = execute(con, qi, model)          # kept, so the gap below is measured

```

```

# ---- 2. What the same data yields without the grain declaration -----
print(_rule("2. THE SAME DATA, COUNTED AT DELIVERY GRAIN"))
naive = con.execute(f"""
    SELECT sum(CASE WHEN l.delivery_date <= o.promised_delivery_date
                      THEN 1 ELSE 0 END) AS on_time, count(*) AS legs
    FROM fact_order_delivery o
    JOIN raw_delivery_legs l USING (order_id)
    WHERE o.is_eligible
           AND o.promised_delivery_date BETWEEN DATE '2026-07-01' AND DATE '2026-07-31'
""").fetchone()
naive_pct = 100 * naive[0] / naive[1]
split = con.execute(f"""
    SELECT count(*) FROM fact_order_delivery
    WHERE is_eligible AND delivery_count > 1
           AND promised_delivery_date BETWEEN DATE '2026-07-01' AND DATE '2026-07-31'
""").fetchone()[0]
print(f"""

```

This is the query a text-to-SQL system writes. It is syntactically valid, semantically reasonable, and joins orders to legs without collapsing to one row per order:

```

{naive[0]} / {naive[1]} = {naive_pct:.4f}%    <-- reported as {naive_pct:.1f}%

```

{split} orders in this window shipped in more than one leg, so {split} orders voted twice. The gap is {naive_pct - governed.value:.2f} points -- small enough to look like rounding, and it errs UPWARD.

A wrong number that flatters you is the one that survives to the board deck.""")

```

# ---- 3. That query, refused -----
print(_rule("3. THAT QUERY, REFUSED BY NAME -- BEFORE ANY SQL EXISTS"))
print()
rc, out = answer(con, QueryIntent(
    intent_type="descriptive", metric="on_time_delivery_pct",
    grain="delivery",          # <-- the only change
    time_window=("2026-07-01", "2026-07-31"), time_label="July 2026",
    filters=(Filter("region", "=", "IN"),),
), model)
print(out)
failures += (rc != 1)

```

```

# ---- 4. Why it dropped -----

```

```

question = "Why did on-time delivery drop for India warehouses last month?"
print(_rule(f"4. THE QUESTION BI TOOLS CANNOT TAKE\n    {question}"))
print()
rc, out = answer(con, resolve_offline(question, model), model)
print(out)
failures += rc

# ---- 5. Three more gates -----
print(_rule("5. FOUR MORE CLASSES OF WRONG QUESTION, EACH REFUSED"))
base = dict(intent_type="descriptive", metric="on_time_delivery_pct", grain="order",
            time_window=("2026-07-01", "2026-07-31"))
for why, qi in [
    ("a filter value outside the declared allowed set",
     QueryIntent(**base, filters=(Filter("region", "=", "MARS"),)),
    ("a metric that exists but was never approved",
     QueryIntent(**base, "metric": "production_cycle_time_days")),
    ("an inverted time window",
     QueryIntent(**base, "time_window": ("2026-07-31", "2026-07-01"))),
    ("a dimension reachable only through a one-to-many join",
     QueryIntent(**base, dimensions=("delivery_id",))),
]:
    rc, out = answer(con, qi, model)
    gates = ", ".join(g for g, _ in validate(qi, model).failures)
    print(f"\n-- {why}\n    REFUSED by {gates}")
    failures += (rc != 1)

print(_rule("SUMMARY"))
print(f"""
The model chose one thing: which governed metric the question meant. Everything that
had to be exactly right -- grain, exclusions, the join, the arithmetic, the window --
was compiled from metadata by code, the same way every time.

Two users asking the same question in different words get the same number, because the
number was never in the model's hands.

{'OK -- every demonstration behaved as documented.' if not failures
 else f'FAILED -- {failures} demonstration(s) did not behave as documented.'})""")
    return 1 if failures else 0

if __name__ == "__main__":
    sys.exit(main())

```

The one-line version

Natural-language querying does not fail because models can't write SQL. It fails because the metadata that makes a question answerable *correctly* — grain, cardinality, exclusions, approved definitions, process baselines — was never written down where anything could read it.

Build that, and the model's job shrinks to something it is genuinely good at: choosing.

Where the rest of it lives

This is the third of three pieces on the same build:

- **The short version** — a LinkedIn post on the two numbers, and why the flattering one is the one that survives review.
- **The implementation guide** — the long-form LinkedIn article: eight build phases, nine metadata artifacts, and the specific natural-language failure each one closes.
- **This post** — the whole architecture as one runnable file.

The full project adds the other two approved metrics, the nine artifacts as separate governed files with their own owners, data-quality rules that downgrade a trust badge, a twelve-phase process model, and the test suite that holds every figure above in place. The single file here is the part you can read in one sitting and run in one command.

So: what is your team's answer to "which of these two numbers is right"? If it is "ask Priya," you have a semantic layer — it just isn't written down.

Leave the answer in a response. The interesting replies are the ones where the honest answer turns out to be a person.

Part 6 — Appendix: The Nine Metadata Artifacts

Reproduced in full, in phase order. Each is the file the pipeline actually reads — not an excerpt rewritten for the page. The SHA-256 prefix beside each one is of the committed bytes, so a reader can confirm the copy they cloned is the copy quoted here.

6.1 00_kpi_contract.md

178 lines · sha256 9bd9dce0cdfc

Phase 0: KPI Contract — Agreed Metric Definitions

Purpose: This contract defines the metrics this semantic layer must answer, the business decisions they support, who owns each metric, and the acceptance criteria for "done". Without an agreed definition, two people asking the same question get two different numbers, and neither is wrong.

Owner: Chief Supply Chain Officer
Steward: Supply Chain Analytics Council
Last Reviewed: 2026-08-01
Status: Approved

The Two Demonstrator Questions

These are the verbatim natural-language questions the MVP must answer, drawn from real executive reporting requirements. Both answers are computed from committed sample data (Section 5 of design.md) and asserted in the acceptance criteria, so every published figure is reproducible rather than illustrative.

Question 1 (Descriptive): What was on-time delivery performance for India warehouses last month?

Resolves to: on_time_delivery_pct metric, filtered to region = 'IN' , time window July 2026.

Expected Answer:

87.3% (48 of 55 eligible orders delivered on time).

Business Context: Monthly executive scorecard. This number gates the quarterly bonus for the India supply chain team. If OTD falls below 85%, a corrective action plan is triggered.

Question 2 (Diagnostic): Why did on-time delivery drop for India warehouses last month?

Resolves to: Same metric and filters as Q1, broken out by delay_attribution_phase dimension, with a comparison against the prior month (June 2026).

Expected Answer:

87.3%, down 8.9pp from 96.2% in June (25 of 26 eligible). 7 orders were late. Attribution: quality inspection 4, transportation 2, production execution 1. On those 4 orders, QM usage decision ran an average 4.75 days over its 1.0-day standard, against 0 days of available slack — the promise left no room to absorb it. Plant 1010.

Business Context: Root-cause analysis for performance drops. This attribution drives targeted process improvements: if QM is the bottleneck, invest in faster inspection equipment or additional QM staff; if transportation dominates, renegotiate carrier SLAs or add backup carriers.

The Four Governed Metrics

Metric	Definition	Grain	Owner	Steward	Business Decision Supported
On-Time Delivery %	Share of eligible customer orders delivered on or before the promised delivery date	<code>order</code>	VP Supply Chain	Supply Chain Analytics Lead	Executive scorecard; supplier performance reviews; quarterly bonus gates
Manufacturing Schedule Adherence %	Share of production orders confirmed on or before the scheduled finish date	<code>production_order</code>	VP Manufacturing	Production Planning Lead	Manufacturing capacity planning; shift scheduling; bottleneck identification
First Pass Yield %	Share of production orders accepted at the first quality inspection decision	<code>production_order</code>	VP Quality	Quality Analytics Lead	Quality investment prioritization; defect root-cause analysis; supplier quality management
Production Cycle Time (days) <i>(deferred, not in MVP)</i>	Average days from production order release to goods receipt	<code>production_order</code>	VP Manufacturing	Industrial Engineering	Lean initiatives; process improvement projects; lead time quoting

The first three are governed and queryable. **Production Cycle Time is contracted but not implemented** (design §10.2): it is a plain average over columns the other three already materialize, so it teaches nothing new for the cost of building it. It is listed here because the contract is the place to record an agreed definition *before* implementation — but asking for it will be refused by the validator rather than answered, which is the correct behaviour for a metric that has no approved implementation.

What "last month" resolves to

Both questions above say "last month", and that phrase alone is not answerable. The governed reading is `promised_delivery_date` **within the month** — the promise is what the customer was told and what the metric is judged against. This matters concretely:

Reading of "last month"	IN, July 2026	
<code>promised_delivery_date</code> in July (governed)	48/55 = 87.27%	the answer
<code>order_date</code> in July	33/40 = 82.50%	a different question: orders <i>placed</i> in July, some still open
<code>final_delivery_date</code> in July	48/55 = 87.27%	coincides here, but drifts whenever a late order lands in the next month

An ungrounded resolver picking `order_date` returns 82.5% — plausible, precise, and wrong by 4.8pp. Nothing in the number itself reveals the substitution, which is why the date semantics are contracted here rather than left to inference.

Why These Metrics Matter

The Grain Problem (Phase 2)

Without a declared grain, the split-delivery fan-out inflates OTD from 87.3% to 88.5%.

- 6 orders ship in two legs (deliveries_raw.csv has 2 rows per order_id for SO-1009, SO-1012, SO-1014, SO-1016, SO-1017, SO-1018).
- Joining at delivery grain (one row per delivery leg) double-counts split shipments in the numerator.

- The wrong number (88.5%) is both **close enough to look plausible** and **flattering** — a performance overstatement survives review because it makes the team look good.
- The semantic model declares `grain: order` and the validator rejects delivery-grain queries.

The Rework Trap (Phase 3)

Without an exclusion rule, First Pass Yield reads 94.5% instead of 94.4%.

- SO-1004 has a rework loop: inspection lot QL-4004 has two usage decisions (decision_seq 1 = REWORK, decision_seq 2 = ACCEPT).
- Counting all decisions treats the reworked lot as both a failure and a success, understating the defect rate.
- The metric definition in `05_semantic_model.yml` specifies: *"first usage decision only (decision_seq = 1); excludes rework re-inspections"*.
- The compiler applies this exclusion unconditionally, so every query gets the same answer.

The Attribution Rule (Phase 4)

Without per-phase baselines and a variance rule, "why did OTD drop" is unanswerable.

- Each phase has a standard duration (e.g. quality inspection = 1 day, production execution = 3 days).
- For each order, `variance_days = actual_duration - standard_duration` per phase.
- `delay_attribution_phase = argmax(variance_days)` over the **9 variance spans** (phase_seq 2–10), evaluated only for late delivered eligible orders. Phase 1 is the chain's start point rather than a span, phase 11 is a 0-day event folded into `var_shipment`, and phase 12 follows delivery — so those three phases are not emittable attribution values.
- This rule is defined once in `04_process_model.yml` and implemented in `sql/02_staging_model.sql` as a derived column. The LLM never computes it.

Exclusions (What Counts as Eligible)

Metric	Exclusions	Rationale
On-Time Delivery %	Cancelled orders (<code>order_status = 'CANC'</code>)	A cancelled order was never due; including it inflates the denominator
	Not-yet-due orders (<code>promised_delivery_date > AS_OF_DATE</code>)	An order promised after the as-of date is not late, it is pending
Manufacturing Schedule Adherence %	Cancelled production orders	Same rationale as OTD
First Pass Yield %	Rework re-inspections (<code>decision_seq > 1</code>)	Prevents double-counting; measures right-first-time
Production Cycle Time	Cancelled production orders	Incomplete orders distort the average

Critical: These exclusions are not suggestions. They are implemented as predicates in `05_semantic_model.yml` and applied unconditionally by the compiler. A validator gate (`exclusions_applied`) ensures they are present before SQL is generated.

Test Oracle and Acceptance Criteria

Committed data: `data/seed_core.csv` (20 hand-authored orders) + `data/generate.py` (112 generated) → 132 total orders.

Canonical numbers (from `design.md` §5.1, verified by `tests/test_compiler.py`):

Quantity	Value	Test
OTD, IN July 2026	48/55 = 87.2727%	test_q1_otd_india_july_is_87_27_percent
OTD, IN June 2026	25/26 = 96.1538%	test_otd_india_june_is_96_15_percent
Drop	8.88pp	test_the_drop_is_8_88_pp
Late attribution, IN July	quality_inspection 4, transportation 2, production_execution 1	test_q2_attribution_breakdown
Naive delivery-grain OTD (wrong)	54/61 = 88.5246%	test_naive_delivery_grain_overstates_to_88_52
First-pass yield, IN July	51/54 = 94.4444%	test_fpy_correct_and_naive
Naive FPY counting rework (wrong)	52/55 = 94.5455%	same test
Schedule adherence, IN July	50/54 = 92.5926%	test_schedule_adherence

Definition of done:

- 1. python src/ask.py --offline Q1 returns 87.3% with lineage, exclusions applied, and trust badge = TRUSTED.
- 2. python src/ask.py --offline Q2 returns the attribution breakdown with quality_inspection = 4.
- 3. python -m pytest → 34+ tests pass, including all canonical number assertions.
- 4. No ANTHROPIC_API_KEY required for offline operation (golden intents bypass the resolver).
- 5. The demo's AS_OF_DATE appears as a literal in exactly one place, src/semantic/constants.py ; no metadata artifact restates it. (Grepping for the date here would put the literal in this file and break the rule it describes.)

Related Artifacts

- Phase 1 (Technical Metadata): 01_technical_metadata.json — SAP DDIC field mappings, shows where the data lives
- Phase 2 (Standardized Metadata): 02_standardized_metadata.json — grain declarations, prevents the 88.5% error
- Phase 3 (Glossary + Bindings): 03_glossary.yml , 03_column_bindings.yml — maps business terms to columns
- Phase 4 (Process Model): 04_process_model.yml — the 12 phases and variance rule, makes "why" computable
- Phase 5 (Semantic Model): 05_semantic_model.yml — executable metric definitions with exclusions
- Phase 6 (Data Quality): 06_dq_rules.yml — 6 dimension rules with trust badge, gates low-quality results
- Phase 7 (Catalog): 07_catalog_asset.json — governance metadata for compliance and discovery

Sign-Off

By approving this contract, the executive team agrees that:

- 1. These four metrics, as defined here, are the official versions for reporting and decision-making.
- 2. Any other calculation — even if numerically close — is unofficial and must not be used for performance evaluation or bonus determination.
- 3. Changes to metric definitions require approval from the Supply Chain Analytics Council and must be versioned in this document.
- 4. The semantic layer is the single source of truth for these metrics. Dashboard queries, ad-hoc analyses, and LLM-generated answers must all resolve through the governed semantic model.

Approved by:

- Chief Supply Chain Officer: __ Date: __

- VP Supply Chain: __ **Date:** ____
- VP Manufacturing: __ **Date:** ____
- VP Quality: __ **Date:** ____
- Chief Data Officer: __ **Date:** ____

Next Review: 2026-11-01 (quarterly)

6.2 01_technical_metadata.json

312 lines · sha256 12bde68ab0eb

```

{
  "description": "Phase 1 artifact: Technical metadata extracted from SAP DDIC (Data Dictionary). Maps raw SAP field names to their technical properties. Without this layer, an LLM cannot resolve 'when did it ship' to LIKP-WADAT-IST.",
  "source_system": "SAP ERP 6.0 EHP8",
  "extraction_date": "2026-08-01",
  "extraction_method": "SE11 / SE16 metadata query",
  "owner": "SAP Basis Team",
  "steward": "Data Integration Lead",
  "fields": [
    {
      "table": "VBAK",
      "field": "VBELN",
      "data_element": "VBELN-VA",
      "domain": "VBELN",
      "data_type": "CHAR",
      "length": 10,
      "is_key": true,
      "is_nullable": false,
      "source_system": "SAP SD",
      "description": "Sales Document Number (Sales Order ID)",
      "business_name": "Sales Order ID",
      "target_column": "order_id"
    },
    {
      "table": "VBAK",
      "field": "KUNNR",
      "data_element": "KUNNR",
      "domain": "KUNNR",
      "data_type": "CHAR",
      "length": 10,
      "is_key": false,
      "is_nullable": false,
      "source_system": "SAP SD",
      "description": "Customer Number (Sold-to Party)",
      "business_name": "Customer ID",
      "target_column": "customer_id"
    },
    {
      "table": "VBAK",
      "field": "ERDAT",
      "data_element": "ERDAT",
      "domain": "ERDAT",
      "data_type": "DATS",
      "length": 8,
      "is_key": false,
      "is_nullable": false,
      "source_system": "SAP SD",
      "description": "Date on which the record was created (Order Entry Date)",
      "business_name": "Order Date",
      "target_column": "order_date"
    }
  ]
}

```

```

    "table": "VBEP",
    "field": "EDATU",
    "data_element": "EDATU",
    "domain": "DATUM",
    "data_type": "DATS",
    "length": 8,
    "is_key": false,
    "is_nullable": true,
    "source_system": "SAP SD",
    "description": "Requested Delivery Date / Promised Delivery Date",
    "business_name": "Promised Delivery Date",
    "target_column": "promised_delivery_date"
  },
  {
    "table": "LIKP",
    "field": "WADAT_IST",
    "data_element": "WADAT_IST",
    "domain": "DATUM",
    "data_type": "DATS",
    "length": 8,
    "is_key": false,
    "is_nullable": true,
    "source_system": "SAP LE-SHP",
    "description": "Actual Goods Movement Date (Goods Issue Date). THIS IS THE UNGUESSABLE FIELD NAME
an LLM cannot resolve without technical metadata.",
    "business_name": "Goods Issue Date",
    "target_column": "goods_issue_date"
  },
  {
    "table": "LIKP",
    "field": "LFART",
    "data_element": "LFART",
    "domain": "LFART",
    "data_type": "CHAR",
    "length": 4,
    "is_key": false,
    "is_nullable": false,
    "source_system": "SAP LE-SHP",
    "description": "Delivery Type",
    "business_name": "Delivery Type",
    "target_column": "delivery_type"
  },
  {
    "table": "AFKO",
    "field": "AUFNR",
    "data_element": "AUFNR",
    "domain": "AUFNR",
    "data_type": "CHAR",
    "length": 12,
    "is_key": true,
    "is_nullable": false,
    "source_system": "SAP PP",
    "description": "Production Order Number",
    "business_name": "Production Order ID",

```

```
    "target_column": "production_order_id"
  },
  {
    "table": "AFKO",
    "field": "GSTRP",
    "data_element": "GSTRP",
    "domain": "DATUM",
    "data_type": "DATS",
    "length": 8,
    "is_key": false,
    "is_nullable": true,
    "source_system": "SAP PP",
    "description": "Scheduled Start Date (Basic)",
    "business_name": "Scheduled Start Date",
    "target_column": "scheduled_start_date"
  },
  {
    "table": "AFKO",
    "field": "GLTRP",
    "data_element": "GLTRP",
    "domain": "DATUM",
    "data_type": "DATS",
    "length": 8,
    "is_key": false,
    "is_nullable": true,
    "source_system": "SAP PP",
    "description": "Scheduled Finish Date (Basic)",
    "business_name": "Scheduled Finish Date",
    "target_column": "scheduled_finish_date"
  },
  {
    "table": "AFRU",
    "field": "BUDAT",
    "data_element": "BUDAT",
    "domain": "DATUM",
    "data_type": "DATS",
    "length": 8,
    "is_key": false,
    "is_nullable": false,
    "source_system": "SAP PP",
    "description": "Posting Date (Operation Confirmation Date)",
    "business_name": "Production Confirm Date",
    "target_column": "prod_confirm_date"
  },
  {
    "table": "QALS",
    "field": "VDATUM",
    "data_element": "VDATUM",
    "domain": "DATUM",
    "data_type": "DATS",
    "length": 8,
    "is_key": false,
    "is_nullable": true,
    "source_system": "SAP QM",
```

"description": "Usage Decision Date (QM Inspection Decision Date)",
"business_name": "QM Decision Date",
"target_column": "qm_first_decision_date, qm_final_decision_date",
"mapping_note": "One source field, two target columns. A rework loop posts a second usage decision against the same inspection lot, so the standardized model splits VDATUM by decision_seq: qm_first_decision_date takes decision_seq = 1 and qm_final_decision_date takes the maximum. first_pass_yield_pct must read the FIRST, or a reworked order that eventually passes counts as a first-pass success (lot QL-4004 is the seeded case). There is no single qm_decision_date column in the warehouse."

```
    },  
    {  
      "table": "QALS",  
      "field": "VBEWK",  
      "data_element": "QVBEWK",  
      "domain": "QVBEWK",  
      "data_type": "CHAR",  
      "length": 1,  
      "is_key": false,  
      "is_nullable": true,  
      "source_system": "SAP QM",  
      "description": "Usage Decision Code (A=Accept, R=Reject, W=Rework)",  
      "business_name": "Usage Decision",  
      "target_column": "usage_decision"  
    },  
    {  
      "table": "MKPF",  
      "field": "BUDAT",  
      "data_element": "BUDAT",  
      "domain": "DATUM",  
      "data_type": "DATS",  
      "length": 8,  
      "is_key": false,  
      "is_nullable": false,  
      "source_system": "SAP MM",  
      "description": "Posting Date in the Document (Goods Receipt Date)",  
      "business_name": "Goods Receipt Date",  
      "target_column": "gr_date"  
    },  
    {  
      "table": "MSEG",  
      "field": "BUDAT",  
      "data_element": "BUDAT",  
      "domain": "DATUM",  
      "data_type": "DATS",  
      "length": 8,  
      "is_key": false,  
      "is_nullable": false,  
      "source_system": "SAP MM",  
      "description": "Posting Date in the Document (Material Movement Date)",  
      "business_name": "Material Issue Date",  
      "target_column": "material_issue_date"  
    },  
    {  
      "table": "MSEG",
```



```

    "field": "BWART",
    "data_element": "BWART",
    "domain": "BWART",
    "data_type": "CHAR",
    "length": 3,
    "is_key": false,
    "is_nullable": false,
    "source_system": "SAP MM",
    "description": "Movement Type (e.g. 261=Goods Issue, 101=Goods Receipt)",
    "business_name": "Movement Type",
    "target_column": "movement_type"
  },
  {
    "table": "VTTK",
    "field": "DPTBG",
    "data_element": "DPTBG",
    "domain": "DATUM",
    "data_type": "DATS",
    "length": 8,
    "is_key": false,
    "is_nullable": true,
    "source_system": "SAP TM",
    "description": "Shipment Dispatch Date (Start of Transportation)",
    "business_name": "Shipment Date",
    "target_column": "shipment_date"
  },
  {
    "table": "VBRK",
    "field": "FKDAT",
    "data_element": "FKDAT",
    "domain": "DATUM",
    "data_type": "DATS",
    "length": 8,
    "is_key": false,
    "is_nullable": false,
    "source_system": "SAP SD",
    "description": "Billing Date / Invoice Date",
    "business_name": "Invoice Date",
    "target_column": "invoice_date"
  },
  {
    "table": "VBAK",
    "field": "AUGRU",
    "data_element": "AUGRU",
    "domain": "AUGRU",
    "data_type": "CHAR",
    "length": 3,
    "is_key": false,
    "is_nullable": true,
    "source_system": "SAP SD",
    "description": "Order Reason (e.g. promotion, replacement, expedite)",
    "business_name": "Order Reason",
    "target_column": "order_reason"
  },

```

```

{
  "table": "VBAK",
  "field": "VKORG",
  "data_element": "VKORG",
  "domain": "VKORG",
  "data_type": "CHAR",
  "length": 4,
  "is_key": false,
  "is_nullable": false,
  "source_system": "SAP SD",
  "description": "Sales Organization",
  "business_name": "Sales Organization",
  "target_column": "sales_org"
},
{
  "table": "LIKP",
  "field": "VSTEL",
  "data_element": "VSTEL",
  "domain": "VSTEL",
  "data_type": "CHAR",
  "length": 4,
  "is_key": false,
  "is_nullable": false,
  "source_system": "SAP LE-SHP",
  "description": "Shipping Point / Dispatch Point",
  "business_name": "Shipping Point",
  "target_column": "shipping_point"
}
],
"notes": [
  "WADAT_IST is the canonical example of why technical metadata is necessary: 'when did it ship' cannot be resolved to LIKP-WADAT_IST without this layer.",
  "Many SAP fields have generic names (BUDAT appears in AFRU, MKPF, MSEG with different business meanings). Context from the table name is essential.",
  "DATS fields are stored as YYYYMMDD character strings in SAP; the staging model casts them to proper DATE types.",
  "This extract is illustrative; a production semantic layer would include all fields from the relevant tables, versioned and refreshed quarterly as the data model evolves."
],
"lineage": {
  "upstream": [
    "SAP ECC Production System",
    "SAP BW Extractors (2LIS_*)",
    "Data Lake Bronze Layer"
  ],
  "downstream": [
    "DuckDB Warehouse (sql/02_staging_model.sql)",
    "Semantic Layer (metadata/05_semantic_model.yml)",
    "BI Dashboards",
    "Natural Language Query Interface"
  ]
},
"approval_state": "approved",

```

```
"version": "1.0"  
}
```

6.3 02_standardized_metadata.json

279 lines · sha256 [22ca4c0aeec2](#)

```

{
  "description": "Phase 2 artifact: Standardized metadata with declared grain per table. This layer maps raw SAP field names to canonical business names and explicitly declares the grain of each fact table. The grain declaration is what prevents the split-delivery fan-out that inflates OTD from 87.3% to 88.5%.",
  "owner": "Data Engineering Team",
  "steward": "Data Architecture Lead",
  "last_reviewed": "2026-08-01",
  "approval_state": "approved",
  "version": "1.0",

  "grain_declarations": [
    {
      "table": "fact_order_delivery",
      "grain": "order",
      "primary_key": "order_id",
      "description": "One row per customer sales order. All delivery legs collapsed to the order grain via max(delivery_date). This is the grain for on-time delivery measurement.",
      "enforcement": "Test test_fact_is_one_row_per_order asserts count(*) = count(DISTINCT order_id). Any fan-out from split deliveries or joins would violate this and fail the test.",
      "why_this_grain_matters": "6 orders ship in two legs (deliveries_raw has 2 rows for SO-1009, SO-1012, SO-1014, SO-1016, SO-1017, SO-1018). Measuring OTD at delivery-leg grain double-counts split shipments and produces 88.52% instead of 87.27%. The semantic model declares grain: order and the validator rejects delivery-grain queries.",
      "row_count": 132,
      "unique_keys": 132
    },
    {
      "table": "fact_production_order",
      "grain": "production_order",
      "primary_key": "production_order_id",
      "description": "One row per production order. Used for manufacturing schedule adherence and first-pass yield metrics.",
      "enforcement": "Test test_production_order_grain_claim_is_enforced asserts count(*) = count(DISTINCT production_order_id) = 131.",
      "why_this_grain_matters": "Some orders have no production order (e.g. SO-1015, make-to-stock). Measuring order-grain metrics by joining through production orders would silently drop those orders and inflate OTD. The semantic model uses LEFT JOIN and marks the join cardinality as many_to_one.",
      "row_count": 131
    },
    {
      "table": "dim_process_phase",
      "grain": "phase",
      "primary_key": "phase_seq",
      "description": "Literal VALUES table with 12 process phases and their standard durations. This is the source of truth for all phase duration constants.",
      "row_count": 12
    }
  ],

  "field_mappings": [
    {
      "source_table": "VBAK",
      "source_field": "VBELN",

```

```

    "canonical_name": "order_id",
    "target_table": "fact_order_delivery",
    "target_column": "order_id",
    "data_type": "VARCHAR",
    "is_dimension": true,
    "is_measure": false,
    "transformation": "Direct mapping, no transformation"
},
{
    "source_table": "VBAK",
    "source_field": "KUNNR",
    "canonical_name": "customer_id",
    "target_table": "fact_order_delivery",
    "target_column": "customer_id",
    "data_type": "VARCHAR",
    "is_dimension": true,
    "is_measure": false,
    "transformation": "Direct mapping"
},
{
    "source_table": "VBAK",
    "source_field": "ERDAT",
    "canonical_name": "order_date",
    "target_table": "fact_order_delivery",
    "target_column": "order_date",
    "data_type": "DATE",
    "is_dimension": false,
    "is_measure": false,
    "transformation": "CAST(VBAK-ERDAT AS DATE)"
},
{
    "source_table": "VBEP",
    "source_field": "EDATU",
    "canonical_name": "promised_delivery_date",
    "target_table": "fact_order_delivery",
    "target_column": "promised_delivery_date",
    "data_type": "DATE",
    "is_dimension": false,
    "is_measure": false,
    "transformation": "CAST(VBEP-EDATU AS DATE). If an order is re-promised, the CURRENT promise is
used, not the original."
},
{
    "source_table": "LIKP",
    "source_field": "WADAT_IST",
    "canonical_name": "goods_issue_date",
    "target_table": "fact_order_delivery",
    "target_column": "goods_issue_date",
    "data_type": "DATE",
    "is_dimension": false,
    "is_measure": false,
    "transformation": "CAST(LIKP-WADAT_IST AS DATE). For split deliveries, min(goods_issue_date) across
legs."
},

```

```

{
  "source_table": "deliveries_raw",
  "source_field": "delivery_date",
  "canonical_name": "final_delivery_date",
  "target_table": "fact_order_delivery",
  "target_column": "final_delivery_date",
  "data_type": "DATE",
  "is_dimension": false,
  "is_measure": false,
  "transformation": "max(delivery_date) across all delivery legs for the order. An order is delivered
when its LAST leg arrives. This is the split-delivery collapse that enforces order grain."
},
{
  "source_table": "AFKO",
  "source_field": "AUFNR",
  "canonical_name": "production_order_id",
  "target_table": "fact_order_delivery",
  "target_column": "production_order_id",
  "data_type": "VARCHAR",
  "is_dimension": true,
  "is_measure": false,
  "transformation": "LEFT JOIN to stg_production_orders. NULL for make-to-stock orders (e.g. SO-
1015)."
},
{
  "source_table": "AFRU",
  "source_field": "BUDAT",
  "canonical_name": "prod_confirm_date",
  "target_table": "fact_order_delivery",
  "target_column": "prod_confirm_date",
  "data_type": "DATE",
  "is_dimension": false,
  "is_measure": false,
  "transformation": "CAST(AFRU-BUDAT AS DATE). Phase 5 timestamp."
},
{
  "source_table": "QALS",
  "source_field": "VDATUM",
  "canonical_name": "qm_first_decision_date",
  "target_table": "fact_order_delivery",
  "target_column": "qm_first_decision_date",
  "data_type": "DATE",
  "is_dimension": false,
  "is_measure": false,
  "transformation": "min(QALS-VDATUM WHERE decision_seq = '1'). For first-pass yield calculation."
},
{
  "source_table": "QALS",
  "source_field": "VDATUM",
  "canonical_name": "qm_final_decision_date",
  "target_table": "fact_order_delivery",
  "target_column": "qm_final_decision_date",
  "data_type": "DATE",
  "is_dimension": false,

```

```

    "is_measure": false,
    "transformation": "max(QALS-VDATUM). For reworked lots (e.g. QL-4004), this is the FINAL decision
date, not the first. Phase 6 timestamp for variance calculation uses the final decision."
  },
  {
    "source_table": "MKPF",
    "source_field": "BUDAT",
    "canonical_name": "gr_date",
    "target_table": "fact_order_delivery",
    "target_column": "gr_date",
    "data_type": "DATE",
    "is_dimension": false,
    "is_measure": false,
    "transformation": "CAST(MKPF-BUDAT AS DATE) for goods receipt movement. Phase 7 timestamp."
  },
  {
    "source_table": "VTTK",
    "source_field": "DPTBG",
    "canonical_name": "shipment_date",
    "target_table": "fact_order_delivery",
    "target_column": "shipment_date",
    "data_type": "DATE",
    "is_dimension": false,
    "is_measure": false,
    "transformation": "CAST(VTTK-DPTBG AS DATE). Phase 10 timestamp."
  },
  {
    "source_table": "VBRK",
    "source_field": "FKDAT",
    "canonical_name": "invoice_date",
    "target_table": "fact_order_delivery",
    "target_column": "invoice_date",
    "data_type": "DATE",
    "is_dimension": false,
    "is_measure": false,
    "transformation": "MODELED as final_delivery_date + billing_standard_duration (1 day). Not
extracted from raw billing documents."
  }
],

"conformed_dimensions": [
  {
    "dimension": "region",
    "description": "Geographic region (country or country group)",
    "allowed_values": ["IN", "US", "DE", "CN", "BR"],
    "source": "Master data / organizational hierarchy",
    "used_in_tables": ["fact_order_delivery", "fact_production_order"],
    "is_slowly_changing": false
  },
  {
    "dimension": "plant",
    "description": "Manufacturing plant or fulfillment center",
    "allowed_values": ["1010", "1710", "2020", "3030"],
    "source": "SAP organizational structure",

```

```

    "used_in_tables": ["fact_order_delivery", "fact_production_order"],
    "is_slowly_changing": false
  },
  {
    "dimension": "warehouse",
    "description": "Finished goods warehouse",
    "source": "SAP EWM / LE configuration",
    "used_in_tables": ["fact_order_delivery"],
    "is_slowly_changing": false
  },
  {
    "dimension": "carrier",
    "description": "Transportation carrier for final delivery",
    "allowed_values": ["BLUEDART", "FEDEX", "DHL", "UPS", "MAERSK"],
    "source": "Carrier master data",
    "used_in_tables": ["fact_order_delivery"],
    "is_slowly_changing": false
  },
  {
    "dimension": "order_status",
    "description": "Current status of the sales order",
    "allowed_values": ["DLVD", "IN_TRANSIT", "CANC", "OPEN"],
    "source": "SAP SD order header",
    "used_in_tables": ["fact_order_delivery"],
    "is_slowly_changing": false
  },
  {
    "dimension": "delay_attribution_phase",
    "description": "Manufacturing phase that caused a late order (derived column)",
    "allowed_values": [
      "mrp_planning", "production_release", "material_staging",
      "production_execution", "quality_inspection", "goods_receipt", "picking_packing",
      "goods_issue", "transportation"
    ],
    "source": "Computed from per-phase variances in fact_order_delivery",
    "used_in_tables": ["fact_order_delivery"],
    "is_slowly_changing": false,
    "computation_rule": "argmax(variance_days) over the 9 variance spans (phase_seq 2-10), evaluated
only for late delivered eligible orders. Phase 1 is the chain start rather than a span, phase 11 is a 0-
day event absorbed into var_shipment, and phase 12 follows delivery -- so those three phase keys are not
emittable values of this dimension."
  }
],

"join_paths": [
  {
    "from_table": "fact_order_delivery",
    "to_table": "fact_production_order",
    "join_type": "LEFT",
    "cardinality": "many_to_one",
    "join_key": "production_order_id",
    "rationale": "Some orders have no production order (make-to-stock). An INNER JOIN would silently
drop those orders and inflate OTD. The semantic model declares this join with cardinality: many_to_one so
the validator can detect fan-out."
  }
]

```



```

    },
    {
      "from_table": "fact_order_delivery",
      "to_table": "deliveries_raw",
      "join_type": "ONE_TO_MANY",
      "cardinality": "one_to_many",
      "join_key": "order_id",
      "rationale": "FAN-OUT HAZARD. 6 orders ship in two legs. Joining here without re-aggregating to
order grain yields 88.52% instead of 87.27%. The semantic model declares this join path as one_to_many
and marks the delivery_id dimension as queryable_at_grain: delivery, so the no_fanout gate rejects it for
order-grain metrics."
    }
  ],

  "data_quality_notes": [
    "fact_order_delivery enforces order grain via max(delivery_date) aggregation before the final JOIN.
Test test_fact_is_one_row_per_order validates this.",
    "Cancelled orders (order_status = 'CANC') have NULL final_delivery_date and are excluded from OTD
denominator.",
    "Not-yet-due orders (promised_delivery_date > AS_OF_DATE) are excluded via is_eligible flag.",
    "S0-1013 is in-transit with final_delivery_date = NULL; it must not count as late because it is not
yet due.",
    "S0-1015 is make-to-stock with production_order_id = NULL; it must survive the LEFT JOIN and count as
eligible.",
    "S0-1009 has split_delivery = 'mixed': leg A delivered on time (2026-07-16), leg B late (2026-07-20).
Order is late using max(delivery_date) = 2026-07-20."
  ],

  "lineage": {
    "upstream": [
      "01_technical_metadata.json (SAP DDIC field definitions)",
      "data/*_raw.csv (committed sample data)",
      "sql/02_staging_model.sql (grain-enforcing SQL)"
    ],
    "downstream": [
      "metadata/05_semantic_model.yml (entity and metric definitions)",
      "src/semantic/validator.py (grain_matches gate)",
      "tests/test_warehouse.py (grain enforcement tests)"
    ]
  }
}

```

6.4 03_glossary.yml

332 lines · sha256 `d40b21b263d9`

```
# Phase 3 artifact: Business glossary with synonyms for natural language resolution
#
# This glossary maps business terminology to technical metadata. The synonyms
# list is load-bearing: it is what enables NL queries like "OTD for India last
# month" to resolve to the correct metric and filters. Without comprehensive
# synonyms, the resolver cannot map user vocabulary to governed definitions.
```

```
terms:
```

```
- term: On-Time Delivery %
```

```
definition: >
```

```
Share of eligible customer orders delivered on or before the promised
delivery date. Measured at order grain (one row per order_id). An order
counts as delivered only when its final delivery leg arrives. Excludes
cancelled orders and orders not yet due.
```

```
synonyms:
```

- OTD
- on time delivery
- on-time delivery
- delivery performance
- delivery reliability
- promise adherence
- OTIF
- on time in full

```
owner: VP Supply Chain
```

```
steward: Supply Chain Analytics Lead
```

```
data_type: percentage
```

```
calculation: (delivered_on_time / eligible_orders) * 100
```

```
- term: Manufacturing Schedule Adherence %
```

```
definition: >
```

```
Share of production orders confirmed on or before the scheduled finish
date. Measures manufacturing execution discipline against the planned
schedule.
```

```
synonyms:
```

- schedule adherence
- schedule compliance
- schedule performance
- production adherence
- manufacturing adherence
- on-schedule production

```
owner: VP Manufacturing
```

```
steward: Production Planning Lead
```

```
data_type: percentage
```

```
calculation: (confirmed_on_schedule / released_orders) * 100
```

```
- term: First Pass Yield %
```

```
definition: >
```

```
Share of production orders accepted at the first quality inspection
decision. Measures quality right-first-time. Excludes rework re-inspections
(decision_seq > 1) to prevent double-counting.
```

```
synonyms:
```

- FPY
- first pass yield

- first time right
- FTR
- quality yield
- right first time
- initial quality

owner: VP Quality

steward: Quality Analytics Lead

data_type: percentage

calculation: (first_decision_accepted / total_first_inspections) * 100

- term: Sales Order

definition: >

Customer order entered in SAP SD. Uniquely identified by order_id.

The primary business object for order-to-cash processes.

synonyms:

- order
- customer order
- SO
- sales order
- sales doc

owner: VP Sales

steward: Sales Operations Lead

primary_key: order_id

source_system: SAP SD

sap_table: VBAK

- term: Production Order

definition: >

Manufacturing order released in SAP PP. Links a sales order to production execution. Not all sales orders have production orders (e.g. make-to-stock items like SO-1015).

synonyms:

- prod order
- manufacturing order
- work order
- production work order

owner: VP Manufacturing

steward: Production Planning Lead

primary_key: production_order_id

source_system: SAP PP

sap_table: AFKO

- term: Inspection Lot

definition: >

Quality inspection batch in SAP QM. Records usage decisions (accept, reject, rework). Reworked lots have multiple decisions; first-pass yield measures decision_seq = 1 only.

synonyms:

- inspection lot
- QM lot
- quality lot
- inspection batch

owner: VP Quality

steward: Quality Analytics Lead

primary_key: inspection_lot_id

source_system: SAP QM

sap_table: QALS

- term: Region

definition: >

Geographic region where the order is fulfilled. Maps to country or country group for reporting purposes.

synonyms:

- country
- geography
- region
- market

owner: VP Global Operations

steward: Master Data Team

data_type: categorical

allowed_values:

- IN
- US
- DE
- CN
- BR

value_labels:

IN: India
US: United States
DE: Germany
CN: China
BR: Brazil

- term: Plant

definition: >

Manufacturing plant or distribution center. SAP organizational unit where production or fulfillment occurs.

synonyms:

- plant
- facility
- manufacturing plant
- production facility
- site

owner: VP Global Operations

steward: Master Data Team

data_type: categorical

allowed_values:

- "1010"
- "1710"
- "2020"
- "3030"

value_labels:

Quoted: the warehouse column is VARCHAR and 05_semantic_model.yml declares
allowed_values as strings. Unquoted, YAML makes these int keys, so a lookup
of value_labels['1010'] misses and the label silently comes back empty.
"1010": India Plant
"1710": US Plant
"2020": Germany Plant

"3030": China Plant

- term: Warehouse

definition: >

Finished goods warehouse or distribution center. May be co-located with a plant or operate independently.

synonyms:

- warehouse
- WH
- distribution center
- DC
- fulfillment center
- depot

owner: VP Logistics

steward: Warehouse Operations Lead

data_type: categorical

primary_key: warehouse

- term: Carrier

definition: >

Transportation carrier responsible for final-mile delivery to customer.

synonyms:

- carrier
- shipper
- logistics provider
- freight carrier
- courier
- transporter

owner: VP Logistics

steward: Transportation Planning Lead

data_type: categorical

allowed_values:

- BLUEDART
- FEDEX
- DHL
- UPS
- MAERSK

- term: Promised Delivery Date

definition: >

Date the customer was promised delivery. Used as the baseline for on-time delivery measurement. If an order is re-promised, the current (latest) promise is used, not the original.

synonyms:

- promise date
- committed delivery date
- delivery promise
- due date
- requested delivery date
- RDD

owner: VP Sales

steward: Order Management Lead

data_type: date

sap_field: VBEP-EDATU

- term: Delivery Date

definition: >

Actual date the order was delivered to the customer. For split deliveries, this is the date of the LAST leg (max across all delivery legs). NULL if the order is still in transit.

synonyms:

- delivery date
- actual delivery date
- delivered date
- completion date
- arrival date
- received date

owner: VP Logistics

steward: Delivery Operations Lead

data_type: date

No SAP field. LIKP-WADAT is the PLANNED goods-issue date and WADAT_IST the actual one; neither is the date the customer received the order. Binding this term to either would ground "when was it delivered" on goods issue -- earlier than delivery, and wrong in the flattering direction for OTD. The value is sourced from deliveries_raw.delivery_date (proof of delivery), which has no DDIC entry.

sap_field: null

source_column: fact_order_delivery.final_delivery_date

- term: Quality Inspection

definition: >

SAP QM process where finished goods are inspected and a usage decision is recorded. Phase 6 in the 12-phase process model. Standard duration 1 day.

synonyms:

- QM
- quality check
- inspection
- quality control
- QC
- quality assurance
- QA
- quality management

owner: VP Quality

steward: Quality Analytics Lead

phase_seq: 6

standard_duration_days: 1

- term: Delay Attribution Phase

definition: >

The manufacturing phase that caused a late order. Computed as the phase with the largest positive variance (actual_duration - standard_duration).

Evaluated only for orders that are eligible, delivered, and late.

synonyms:

- delay cause
- late reason
- delay reason
- attribution
- delay driver

- bottleneck phase

- constraint phase

owner: VP Supply Chain

steward: Process Excellence Lead

data_type: categorical

allowed_values:

- # The 9 emittable values: one per variance span, phase_seq 2-10. demand_capture

- # (phase 1) is the chain's start point rather than a span and delivery_confirmation

- # (phase 11) is absorbed into var_shipment, so neither can ever be produced --

- # declaring them would let a filter gate admit a query that returns zero rows.

- mrp_planning

- production_release

- material_staging

- production_execution

- quality_inspection

- goods_receipt

- picking_packing

- goods_issue

- transportation

- term: Order Status

definition: >

Current status of the sales order. CANC (cancelled) orders are excluded

from on-time delivery denominator because they were never due.

synonyms:

- status

- order status

- delivery status

- fulfillment status

owner: VP Sales

steward: Order Management Lead

data_type: categorical

allowed_values:

- DLVD

- IN_TRANSIT

- CANC

- OPEN

value_labels:

DLVD: Delivered

IN_TRANSIT: In Transit

CANC: Cancelled

OPEN: Open / Not Started

- term: Eligible Orders

definition: >

Orders that should be counted in the on-time delivery denominator.

An order is eligible if it is not cancelled and its promised delivery

date is on or before the as-of date (not yet due orders are excluded).

synonyms:

- eligible

- in scope

- denominator orders

- countable orders

- due orders

```
owner: VP Supply Chain
steward: Supply Chain Analytics Lead
calculation: order_status != 'CANC' AND promised_delivery_date <= AS_OF_DATE
```

```
owner: Chief Data Officer
steward: Data Governance Council
last_reviewed: 2026-08-01
approval_state: approved
version: 1.0
```

6.5 03_column_bindings.yml

269 lines · sha256 3782d9d97514


```

# Phase 3 artifact: Column bindings mapping glossary terms to physical columns
#
# This file binds business terms from 03_glossary.yml to actual columns in
# fact_order_delivery and fact_production_order. These bindings enable the
# resolver to translate natural language filters and dimensions into SQL.
#
# CRITICAL: All columns referenced here must exist in the warehouse.
# A test validates every binding against PRAGMA table_info().
#
# scope: business -- the term is defined in 03_glossary.yml, so a user can name it
#                  in natural language and the resolver can bind it.
# scope: internal -- a physical column the compiler reads (variances, flags, raw phase
#                  timestamps). Deliberately NOT in the glossary: exposing 27 more
#                  terms to the resolver widens the guess space without answering a
#                  question anyone asks. A test enforces that every business-scoped
#                  binding has a glossary entry, so this stays a decision rather
#                  than drift.

bindings:
  # Metrics (computed in semantic model, not direct column bindings)
  # On-Time Delivery %, Manufacturing Schedule Adherence %, First Pass Yield %
  # are expressed as expressions in 05_semantic_model.yml

  # Order dimensions and attributes
  - term: Sales Order
    table: fact_order_delivery
    column: order_id
    scope: business

  - term: Region
    table: fact_order_delivery
    column: region
    scope: business

  - term: Plant
    table: fact_order_delivery
    column: plant
    scope: business

  - term: Warehouse
    table: fact_order_delivery
    column: warehouse
    scope: business

  - term: Carrier
    table: fact_order_delivery
    column: carrier
    scope: business

  - term: Order Status
    table: fact_order_delivery
    column: order_status
    scope: business

```

- term: Promised Delivery Date
table: fact_order_delivery
column: promised_delivery_date
scope: business
- term: Delivery Date
table: fact_order_delivery
column: final_delivery_date
scope: business
- term: Delay Attribution Phase
table: fact_order_delivery
column: delay_attribution_phase
scope: business
- # Production order linkage
- term: Production Order
table: fact_order_delivery
column: production_order_id
scope: business
- # Production order attributes
- term: Production Order
table: fact_production_order
column: production_order_id
scope: business
- term: Sales Order
table: fact_production_order
column: order_id
scope: business
- # Phase timestamps (order grain)
- term: MRP Date
table: fact_order_delivery
column: mrp_date
scope: internal
- term: Production Release Date
table: fact_order_delivery
column: release_date
scope: internal
- term: Material Issue Date
table: fact_order_delivery
column: material_issue_date
scope: internal
- term: Production Confirm Date
table: fact_order_delivery
column: prod_confirm_date
scope: internal

- term: QM First Decision Date
table: fact_order_delivery
column: qm_first_decision_date
scope: internal

- term: QM Final Decision Date
table: fact_order_delivery
column: qm_final_decision_date
scope: internal

- term: Goods Receipt Date
table: fact_order_delivery
column: gr_date
scope: internal

- term: Picking Date
table: fact_order_delivery
column: picking_date
scope: internal

- term: Goods Issue Date
table: fact_order_delivery
column: goods_issue_date
scope: internal

- term: Shipment Date
table: fact_order_delivery
column: shipment_date
scope: internal

- term: Invoice Date
table: fact_order_delivery
column: invoice_date
scope: internal

Variance columns (actual - standard duration per phase)

- term: MRP Variance
table: fact_order_delivery
column: var_mrp
scope: internal

- term: Release Variance
table: fact_order_delivery
column: var_release
scope: internal

- term: Material Issue Variance
table: fact_order_delivery
column: var_issue
scope: internal

- term: Production Confirm Variance
table: fact_order_delivery
column: var_confirm

```
scope: internal

- term: QM Variance
  table: fact_order_delivery
  column: var_qm
  scope: internal

- term: Goods Receipt Variance
  table: fact_order_delivery
  column: var_gr
  scope: internal

- term: Picking Variance
  table: fact_order_delivery
  column: var_picking
  scope: internal

- term: Goods Issue Variance
  table: fact_order_delivery
  column: var_goods_issue
  scope: internal

- term: Shipment Variance
  table: fact_order_delivery
  column: var_shipment
  scope: internal

# Computed flags and metrics
- term: Is Delivered
  table: fact_order_delivery
  column: is_delivered
  scope: internal

- term: Is Eligible
  table: fact_order_delivery
  column: is_eligible
  scope: internal

- term: Is On Time
  table: fact_order_delivery
  column: is_on_time
  scope: internal

- term: Delay Days
  table: fact_order_delivery
  column: delay_days
  scope: internal

- term: Available Slack Days
  table: fact_order_delivery
  column: available_slack_days
  scope: internal

# Production order phase timestamps
```

- term: MRP Date
table: fact_production_order
column: mrp_date
scope: internal
- term: Production Release Date
table: fact_production_order
column: release_date
scope: internal
- term: Scheduled Finish Date
table: fact_production_order
column: scheduled_finish_date
scope: internal
- term: Material Issue Date
table: fact_production_order
column: material_issue_date
scope: internal
- term: Production Confirm Date
table: fact_production_order
column: confirm_date
scope: internal
- term: Goods Receipt Date
table: fact_production_order
column: gr_date
scope: internal
- term: QM First Decision Date
table: fact_production_order
column: qm_first_decision_date
scope: internal
- term: QM Final Decision Date
table: fact_production_order
column: qm_final_decision_date
scope: internal
- term: Is On Schedule
table: fact_production_order
column: is_on_schedule
scope: internal

owner: Data Engineering Team

steward: Data Governance Council

last_reviewed: 2026-08-01

approval_state: approved

version: 1.0

notes: >

Bindings are validated at build time against the actual warehouse schema.

Any binding referencing a non-existent column will cause `test_bindings_reference_real_fact_columns` to fail, preventing deployment of broken metadata.

6.6 `04_process_model.yml`

237 lines · sha256 `69031407f397`

```
# Phase 4 artifact: Manufacturing process model with standard durations
#
# This file defines the 12-phase Plan-to-Deliver process, baseline durations,
# and the variance attribution rule that makes "why did OTD drop" computable.
#
# CRITICAL: standard_duration_days values must match dim_process_phase in
# sql/02_staging_model.sql EXACTLY. A test compares them row by row.
#
# Phases 1-11 sum to 12 days (the delivery promise horizon).
# All 12 phases sum to 13 days (billing follows delivery, cannot consume slack).
```

phases:

- phase_seq: 1
phase_key: demand_capture
phase_name: Demand & order capture
sap_module: SAP SD
business_event: Sales order created
timestamp_field: order_date
standard_duration_days: 0
description: >
Customer order is received and entered into SAP SD. Zero-day event:
no production work occurs during this phase.
- phase_seq: 2
phase_key: mrp_planning
phase_name: Requirements planning
sap_module: SAP PP (MRP)
business_event: Planned order created
timestamp_field: mrp_date
standard_duration_days: 1
description: >
MRP run evaluates demand and creates planned orders to fulfill it.
Standard: 1 day from order entry to MRP execution.
- phase_seq: 3
phase_key: production_release
phase_name: Production order release
sap_module: SAP PP
business_event: Production order released
timestamp_field: release_date
standard_duration_days: 1
description: >
Planned order is converted to a production order and released for execution.
Standard: 1 day from MRP to release.
- phase_seq: 4
phase_key: material_staging
phase_name: Material staging & issue
sap_module: SAP MM
business_event: Goods issue to production order
timestamp_field: material_issue_date
standard_duration_days: 1
description: >

Raw materials and components are staged and issued to the production order.

Standard: 1 day from release to material availability.

- phase_seq: 5

phase_key: production_execution

phase_name: Production execution

sap_module: SAP PP

business_event: Operation confirmed

timestamp_field: prod_confirm_date

standard_duration_days: 3

description: >

Manufacturing operations are executed and confirmed. Standard: 3 days from material issue to production completion. Longest phase in the chain.

- phase_seq: 6

phase_key: quality_inspection

phase_name: Quality inspection

sap_module: SAP QM

business_event: Usage decision

timestamp_field: qm_final_decision_date

standard_duration_days: 1

description: >

Finished goods are inspected and a usage decision is recorded (accept, reject, or rework). Standard: 1 day from production confirmation to QM decision. For reworked lots, this is the FINAL decision date.

- phase_seq: 7

phase_key: goods_receipt

phase_name: Goods receipt to stock

sap_module: SAP MM / EWM

business_event: GR posted to unrestricted stock

timestamp_field: gr_date

standard_duration_days: 1

description: >

Accepted finished goods are posted to unrestricted-use inventory. Standard: 1 day from QM decision to GR.

- phase_seq: 8

phase_key: picking_packing

phase_name: Picking & packing

sap_module: SAP EWM

business_event: Picking completed

timestamp_field: picking_date

standard_duration_days: 1

description: >

Order is picked from warehouse stock and packed for shipment. Standard: 1 day from GR to picking complete.

- phase_seq: 9

phase_key: goods_issue

phase_name: Goods issue / dispatch

sap_module: SAP LE-SHP

business_event: Actual goods issue

timestamp_field: goods_issue_date


```
standard_duration_days: 0
description: >
    Packed shipment is handed over to carrier and goods issue is posted.
    Zero-day event: goods issue and picking occur in rapid succession.
```

- phase_seq: 10

```
phase_key: transportation
phase_name: Transportation
sap_module: SAP TM
business_event: Shipment dispatched
timestamp_field: shipment_date
standard_duration_days: 3
description: >
    Carrier transports goods from warehouse to customer site. Standard: 3 days
    from goods issue to delivery confirmation. This variance absorbs both
    transportation and delivery_confirmation (phase 11, a zero-day POD event).
```

- phase_seq: 11

```
phase_key: delivery_confirmation
phase_name: Delivery confirmation
sap_module: SAP TM / LE
business_event: POD / delivery confirmed
timestamp_field: final_delivery_date
standard_duration_days: 0
description: >
    Proof of delivery is received and the delivery is confirmed complete.
    Zero-day event: the moment the customer accepts the shipment.
```

- phase_seq: 12

```
phase_key: billing
phase_name: Billing
sap_module: SAP SD
business_event: Invoice created
timestamp_field: invoice_date
standard_duration_days: 1
description: >
    Invoice is created and sent to the customer. Standard: 1 day after
    delivery confirmation. Billing follows delivery and cannot consume
    delivery promise slack, so it is excluded from available_slack_days
    computation (phases 1-11 only sum to 12).
```

```
# How each phase's elapsed time is measured. END-ANCHORED: a span belongs to the phase
# it ENDS at. This is the load-bearing convention -- naming a variance after the phase it
# starts from puts every standard one phase out of step, which is exactly the defect that
# shipped and had to be corrected in the fact table.
#
# The spans are contiguous: each one starts where the previous ended, so every day between
# order_date and final_delivery_date is owned by exactly one phase and
# sum(variances) = elapsed - 12 for every delivered order. An unowned span would let a real
# delay produce no variance anywhere and return "no cause found" for a late order. The
# warehouse asserts this reconciliation directly (test_variance_chain_is_contiguous).
```

```
variances:
- column: var_mrp
  phase_key: mrp_planning
```

```

span: order_date -> mrp_date
standard_duration_days: 1
- column: var_release
  phase_key: production_release
  span: mrp_date -> release_date
  standard_duration_days: 1
- column: var_issue
  phase_key: material_staging
  span: release_date -> material_issue_date
  standard_duration_days: 1
- column: var_confirm
  phase_key: production_execution
  span: material_issue_date -> prod_confirm_date
  standard_duration_days: 3
- column: var_qm
  phase_key: quality_inspection
  span: prod_confirm_date -> qm_final_decision_date
  standard_duration_days: 1
note: >
  Uses the FINAL decision, because attribution asks how long inspection actually
  took including any rework re-inspection. first_pass_yield_pct is the opposite
  case and must use the FIRST decision.
- column: var_gr
  phase_key: goods_receipt
  span: qm_final_decision_date -> gr_date
  standard_duration_days: 1
- column: var_picking
  phase_key: picking_packing
  span: gr_date -> picking_date
  standard_duration_days: 1
- column: var_goods_issue
  phase_key: goods_issue
  span: picking_date -> goods_issue_date
  standard_duration_days: 0
- column: var_shipment
  phase_key: transportation
  span: goods_issue_date -> final_delivery_date
  standard_duration_days: 3
note: >
  Starts at goods issue, not at dispatch, and its standard absorbs
  delivery_confirmation's 0 days. Measuring from the dispatch timestamp left the
  goods-issue-to-dispatch span owned by nobody, so a dispatch hold produced no
  variance in any column. The nine standards here sum to 12, matching the 12 that
  available_slack_days subtracts.

```

attribution:

```
rule: variance_argmax
```

```
description: >
```

Delay attribution identifies which phase caused an order to be late.

For each phase, $\text{variance} = (\text{actual_duration} - \text{standard_duration})$.

The phase with the largest positive variance is the attributed cause.

```
formula: delay_attribution_phase = argmax(variance_days) over the 9 variances above
```

```
tie_breaking: earliest phase wins (lowest phase_seq)
```

```
candidate_phases: >
```

The nine spans in the variances block, covering phase_seq 2-10. Phase 1 (demand_capture) is the chain's start point rather than a span, phase 11 (delivery_confirmation) is a 0-day event absorbed into var_shipment, and phase 12 (billing) follows delivery. Only a phase with variance > 0 can be attributed.

scope: evaluated only for orders that are eligible, delivered, and late

exclusions:

- On-time orders return NULL (no delay to attribute)
- Not-yet-due orders return NULL (not late, pending)
- Cancelled orders return NULL (never delivered)
- Undelivered in-transit orders return NULL (not delivered)

notes:

- Phase 12 (billing) is excluded from attribution because it occurs after the promised delivery date and cannot explain a delivery delay.
- A phase with no variance (actual = standard) cannot be the attributed cause, even if all other phases are faster than standard.
- A phase that did not run (e.g. production for make-to-stock orders) has variance = NULL and cannot be attributed.

owner: VP Supply Chain Operations

steward: Manufacturing Excellence Team

last_reviewed: 2026-08-01

approval_state: approved

6.7 05_semantic_model.yml

234 lines · sha256 0e7c22eeb54d

```
# Phase 5 artifact: the governed semantic model.
# Tool-neutral. Portable to dbt MetricFlow, Cube, Unity Catalog, SAP Datasphere.
# This file is the single definition of every metric. BI and NL querying both read it,
# so the number cannot drift between a dashboard and a chat answer.
```

```
entities:
```

- name: fact_order_delivery
label: Order Delivery Fact
grain: order
primary_key: order_id
table: fact_order_delivery
materialized: true
description: One row per customer sales order, all delivery legs collapsed.
joins:
 - to: fact_production_order
cardinality: many_to_one
join_key: production_order_id
note: >
Orders may have no production order (make-to-stock, e.g. S0-1015).
LEFT JOIN required; an INNER JOIN silently drops eligible orders
and inflates OTD.
 - to: raw_deliveries
cardinality: one_to_many
join_key: order_id
note: >
FAN-OUT HAZARD. 6 orders ship in two legs. Joining here without
re-aggregating to order grain yields 88.52% instead of 87.27%.
Never join this path for an order-grain metric.
- name: fact_production_order
label: Production Order Fact
grain: production_order
primary_key: production_order_id
table: fact_production_order
materialized: true
description: One row per production order. Make-to-stock orders have none.
joins:
 - to: fact_order_delivery
cardinality: many_to_one
join_key: order_id
note: >
REQUIRED, not optional. fact_production_order carries NO region and NO
promised_delivery_date column, so both production metrics are unfilterable
and unwindowable without this join. Verified non-fan-out against the built
warehouse: 131 production orders over 131 distinct order_ids, zero orders
with more than one production order. many_to_one, so the no_fanout gate
must PERMIT it -- it is the counter-example to raw_deliveries.
- name: raw_deliveries
label: Raw Delivery Legs
grain: delivery
primary_key: delivery_id
table: null

materialized: false

description: >

One row per delivery leg. Split shipments create multiple rows per order.

NOT MATERIALIZED: there is no `raw\_deliveries` table or view in the warehouse (the nearest object is the `stg\_deliveries` view over deliveries_raw.csv). This entity exists so the fan-out hazard is declarable and so the no_fanout gate has a one_to_many edge to reject. `materialized: false` is what stops the compiler from emitting `FROM raw\_deliveries` for a query the validator would have refused anyway -- an unresolvable table name is a catalog error, and it should surface as a named CompilerError rather than as a DuckDB "table does not exist".

dimensions:

- name: region
label: Region
column: region
entity: fact_order_delivery
data_type: string
allowed_values: ["IN", "US", "EU", "APAC"]
description: Geographic region for sales and fulfillment.
- name: plant
label: Plant
column: plant
entity: fact_order_delivery
data_type: string
allowed_values: ["1010", "1710"]
description: Manufacturing plant code.
- name: warehouse
label: Warehouse
column: warehouse
entity: fact_order_delivery
data_type: string
description: Distribution warehouse code.
- name: carrier
label: Carrier
column: carrier
entity: fact_order_delivery
data_type: string
allowed_values: ["BLUEDART", "FEDEX", "DHL"]
description: Logistics carrier.
- name: delay_attribution_phase
label: Delay Attribution Phase
column: delay_attribution_phase
entity: fact_order_delivery
data_type: string
allowed_values:
 - quality_inspection
 - transportation
 - production_execution
 - material_staging
 - picking_packing

- mrp_planning
- production_release
- goods_receipt
- goods_issue

description: >

Phase with the largest positive variance for late orders. Exactly the 9 variance spans (phase_seq 2-10): demand_capture is the chain's start point rather than a span and delivery_confirmation is a 0-day event folded into var_shipment, so neither is emittable -- declaring either would let the filters_bound gate admit a query that returns zero rows and present it as an answer.

- name: order_status
label: Order Status
column: order_status
entity: fact_order_delivery
data_type: string
allowed_values: ["DLVD", "CANC", "IN_TRANSIT"]
description: Current status of the sales order.
- name: promised_delivery_date
label: Promised Delivery Date
column: promised_delivery_date
entity: fact_order_delivery
data_type: date
description: Date promised to customer for order delivery. Time dimension for windowing.
- name: delivery_id
label: Delivery ID
column: delivery_id
entity: raw_deliveries
reachable_via: raw_deliveries
queryable_at_grain: delivery
data_type: string
description: Delivery leg identifier. Exists to test no_fanout gate rejection of one_to_many joins.

metrics:

- name: on_time_delivery_pct
label: On-Time Delivery %
description: >
Share of eligible customer orders delivered on or before the promised delivery date. An order counts as delivered only when its final leg arrives.
entity: fact_order_delivery
grain: order
time_dimension: promised_delivery_date
expression:
type: ratio_percent
numerator: "sum(CASE WHEN is_on_time THEN 1 ELSE 0 END)"
denominator: "sum(CASE WHEN is_eligible THEN 1 ELSE 0 END)"
exclusions:
- key: cancelled_orders
predicate: "order_status <> 'CANC'"
rationale: A cancelled order was never due; including it inflates the denominator.
- key: not_yet_due

predicate: "promised_delivery_date <= DATE '__AS_OF_DATE__'"

rationale: >

An order promised after the as-of date is not late, it is pending.

Counting it as a miss understates performance.

business_rules:

- Measured against the CURRENT promised date, not the original. Re-promised orders (SO-1011) are judged on the commitment in force at delivery.
- Order-grain only. Delivery-grain aggregation double-counts split shipments.

owner: VP Supply Chain

steward: Supply Chain Data Steward

approval_state: approved

lineage: ["SAP SD", "SAP EWM", "SAP TM", "fact_order_delivery"]

- name: first_pass_yield_pct

label: First Pass Yield %

description: >

Share of production orders where the first quality inspection resulted in acceptance, without rework. Excludes rework re-inspections to prevent double-counting.

entity: fact_production_order

grain: production_order

time_dimension: promised_delivery_date

expression:

type: ratio_percent

numerator: "sum(CASE WHEN first_usage_decision = 'ACCEPT' THEN 1 ELSE 0 END)"

denominator: "count(*)"

exclusions:

- key: cancelled_production_orders

predicate: "status <> 'CANC'"

rationale: Cancelled production orders were never completed; including them inflates the denominator.

- key: rework_reinspections

predicate: "decision_count >= 1"

rationale: >

The first_usage_decision column already collapses multiple inspection decisions to the FIRST one only (decision_seq = 1). Rework lots (e.g. QL-4004) have decision_count = 2 but first_usage_decision = 'REWORK'. This exclusion documents that we read the first decision, not the final one (qm_final_decision_date), which prevents the 94.55% error where eventually-accepted rework is counted as first-pass success. The predicate decision_count >= 1 ensures the row is a production order with at least one inspection, giving the exclusions_applied gate something to record.

business_rules:

- Measures first-pass acceptance rate. Rework re-inspections are excluded.
- Windows on promised_delivery_date via join to fact_order_delivery, not scheduled_finish_date.

owner: VP Manufacturing

steward: Manufacturing Data Steward

approval_state: approved

lineage: ["SAP PP", "SAP QM", "fact_production_order", "fact_order_delivery"]

- name: mfg_schedule_adherence_pct

label: Manufacturing Schedule Adherence %

description: >

Share of production orders that were confirmed on or before the scheduled

```

    finish date. Measures manufacturing execution discipline.
entity: fact_production_order
grain: production_order
time_dimension: promised_delivery_date
expression:
  type: ratio_percent
  numerator: "sum(CASE WHEN is_on_schedule THEN 1 ELSE 0 END)"
  denominator: "count(*)"
exclusions:
  - key: cancelled_production_orders
    predicate: "status <> 'CANC'"
    rationale: Cancelled production orders were never completed; including them inflates the
denominator.
business_rules:
  - Measures adherence to scheduled_finish_date for production confirmation.
  - Windows on promised_delivery_date via join to fact_order_delivery, not scheduled_finish_date.
  - Using scheduled_finish_date for windowing yields 42/45, which is the wrong cohort.
owner: VP Manufacturing
steward: Manufacturing Data Steward
approval_state: approved
lineage: ["SAP PP", "fact_production_order", "fact_order_delivery"]

```

6.8 06_dq_rules.yml

135 lines · sha256 009cfc81003a

Phase 6 artifact: Data quality rules bound to metric inputs.

Each rule measures one DQ dimension and degrades/blocks specific answers when failing.

rules:

- rule_id: dq_completeness_delivery_date

dimension: completeness

description: >

Final delivery date completeness among eligible, past-due orders.

Scoped to exclude SO-1013 (in transit, promised after AS_OF_DATE).

sql: >

SELECT

100.0 * COUNT(CASE WHEN final_delivery_date IS NULL THEN 1 END)

/ NULLIF(COUNT(*), 0) AS observed

FROM fact_order_delivery

WHERE is_eligible = TRUE

AND promised_delivery_date <= DATE '__AS_OF_DATE__'

threshold: 2.0

comparison: lte

severity: degrading

bound_to:

- on_time_delivery_pct

defect_demonstrated: in_transit_null

- rule_id: dq_validity_enum_values

dimension: validity

description: >

Order status and plant values within declared allowed sets.

Validates filter binding and dimension value constraints.

sql: >

SELECT

100.0 * COUNT(CASE

WHEN order_status NOT IN ('DLVD', 'IN_TRANSIT', 'CANC') THEN 1

WHEN plant NOT IN ('1010', '1710') THEN 1

END) / NULLIF(COUNT(*), 0) AS observed

FROM fact_order_delivery

threshold: 0.0

comparison: lte

severity: blocking

bound_to:

- on_time_delivery_pct

- mfg_schedule_adherence_pct

- first_pass_yield_pct

defect_demonstrated: cancelled_order

- rule_id: dq_uniqueness_order_grain

dimension: uniqueness

description: >

Exactly one fact row per order_id. Defect 4: split deliveries must be

collapsed to order grain. A violation here causes the 88.52% wrong answer.

sql: >

SELECT

COUNT(*) AS observed

FROM (

```

        SELECT order_id, COUNT(*) AS row_count
        FROM fact_order_delivery
        GROUP BY order_id
        HAVING COUNT(*) > 1
    )
threshold: 0.0
comparison: lte
severity: blocking
bound_to:
    - on_time_delivery_pct
defect_demonstrated: split_delivery

- rule_id: dq_timeliness_warehouse_freshness
dimension: timeliness
description: >
    Hours between the newest invoice_date and AS_OF_DATE. A live mart would set this
    to the 26-hour load SLA from design 6.1. This demo runs on a frozen snapshot whose
    newest invoice is 72 hours before AS_OF_DATE, which no live SLA can express, so the
    threshold is 96 -- loose enough to pass on clean data, tight enough that a stale
    warehouse still trips it. The date itself is never written here: the SQL carries the
    __AS_OF_DATE__ token and constants.py is the single source of the value.
sql: >
    SELECT
        EXTRACT(EPOCH FROM (CAST(DATE '__AS_OF_DATE__' AS TIMESTAMP) - CAST(MAX(invoice_date) AS
TIMESTAMP)))) / 3600.0 AS observed
    FROM fact_order_delivery
    WHERE invoice_date IS NOT NULL
threshold: 96.0
comparison: lte
severity: degrading
bound_to:
    - on_time_delivery_pct
    - mfg_schedule_adherence_pct
    - first_pass_yield_pct
defect_demonstrated: make_to_stock_no_prod_order

- rule_id: dq_consistency_phase_ordering
dimension: consistency
description: >
    Phase timestamp ordering: release <= gr <= goods_issue <= final_delivery.
    Defects 7 and 8: QM-dominated and non-QM delays. Validates attribution logic.
sql: >
    SELECT
        COUNT(*) AS observed
    FROM fact_order_delivery
    WHERE release_date IS NOT NULL
        AND gr_date IS NOT NULL
        AND goods_issue_date IS NOT NULL
        AND final_delivery_date IS NOT NULL
        AND NOT (
            release_date <= gr_date
            AND gr_date <= goods_issue_date
            AND goods_issue_date <= final_delivery_date
        )

```

```

threshold: 0.0
comparison: lte
severity: blocking
bound_to:
  - on_time_delivery_pct
  - mfg_schedule_adherence_pct
defect_demonstrated: qm_dominated_delay non_qm_delay

- rule_id: dq_accuracy_otd_certification
dimension: accuracy
description: >
  Recomputed IN/July OTD vs certified 87.2727%. Defect 5: repromised orders
  measured against current promise. Absolute percentage point difference.
sql: >
  SELECT
    ABS(
      100.0 * SUM(CASE WHEN is_on_time THEN 1 ELSE 0 END)
      / NULLIF(SUM(CASE WHEN is_eligible THEN 1 ELSE 0 END), 0)
      - 87.2727
    ) AS observed
  FROM fact_order_delivery
  WHERE region = 'IN'
    AND promised_delivery_date BETWEEN DATE '2026-07-01' AND DATE '2026-07-31'
threshold: 0.1
comparison: lte
severity: blocking
bound_to:
  - on_time_delivery_pct
defect_demonstrated: repromised_date rework_loop

```

6.9 07_catalog_asset.json

92 lines · sha256 66692f171672

```

{
  "asset_id": "semantic.metric.on_time_delivery_pct",
  "asset_type": "certified_metric",
  "name": "On-Time Delivery %",
  "technical_name": "on_time_delivery_pct",
  "description": "Share of eligible customer orders delivered on or before the promised delivery date. An order counts as delivered only when its final leg arrives.",
  "definition_sql": "100.0 * sum(CASE WHEN is_on_time THEN 1 ELSE 0 END) / sum(CASE WHEN is_eligible THEN 1 ELSE 0 END)",
  "grain": "order",
  "time_dimension": "promised_delivery_date",
  "unit": "percent",
  "primary_asset": "fact_order_delivery",

  "owner": "VP Supply Chain",
  "steward": "Supply Chain Data Steward",
  "certification_state": "certified",
  "certified_on": "__AS_OF_DATE__",
  "review_cadence": "quarterly",

  "data_subject_category": "business_customer_contact",
  "personal_data_category": "limited_personal_data",
  "personal_data_fields": [
    {
      "column": "customer_id",
      "classification": "pseudonymous_identifier",
      "note": "Party key, not a name. Resolves to a legal entity in SAP KNA1; for named-individual sole proprietors it is personal data under DPDP."
    },
    {
      "column": "ship_to_city",
      "classification": "location_coarse",
      "note": "City-level only. No street address is carried into the mart."
    }
  ],
  "processing_purpose": "Supply chain performance measurement and delivery-commitment management",
  "legal_basis": "legitimate_interest_contract_performance",
  "access_group": "supply-chain-analysts",
  "retention_period": "P7Y",
  "retention_basis": "Statutory commercial record-keeping period; delivery performance is contract-performance evidence.",
  "masking_policy": "customer_id pseudonymised at ingestion; ship_to_city truncated to city; no masking applied to aggregate metric output",
  "cross_border_transfer": "none_declared",

  "lineage": [
    "SAP SD (VBAK/VBAP) - sales order created",
    "SAP PP (PLAF/AFKO/AFRU) - planned order, production order, confirmation",
    "SAP QM (QALS) - inspection lot and usage decision",
    "SAP MM/EWM (MSEG/MKPF) - goods issue to production, goods receipt to stock, picking",
    "SAP TM/LE-SHP (LIKP/VTTK) - actual goods issue, shipment, proof of delivery",
    "data lake - raw extract landing (data/*.csv)",
    "sql/02_staging_model.sql - staging views, canonical names and types",
  ]
}

```

```

    "fact_order_delivery - order-grain mart, delivery legs collapsed",
    "metadata/05_semantic_model.yml - governed metric definition",
    "semantic layer - validated intent compiled to SQL",
    "consumer - NL answer, BI dashboard"
  ],

  "upstream_systems": ["SAP SD", "SAP PP", "SAP QM", "SAP MM", "SAP EWM", "SAP TM"],
  "related_metrics": [
    "first_pass_yield_pct",
    "mfg_schedule_adherence_pct",
    "production_cycle_time_days (contracted, deferred)"
  ],

  "related_assets": [
    "fact_order_delivery",
    "fact_production_order",
    "dim_process_phase"
  ],

  "quality_contract": {
    "rules_artifact": "metadata/06_dq_rules.yml",
    "blocking_rules": [
      "dq_validity_enum_values",
      "dq_uniqueness_order_grain",
      "dq_consistency_phase_ordering",
      "dq_accuracy_otd_certification"
    ],
    "degrading_rules": [
      "dq_completeness_delivery_date",
      "dq_timeliness_warehouse_freshness"
    ],
    "badge_semantics": "TRUSTED = all rules pass. DEGRADED = a degrading rule fails; the number is served with the failing rule named. BLOCKED = a blocking rule fails; a refusal is returned instead of a number."
  },

  "known_limitations": [
    "Measured against the CURRENT promised date, not the original. Re-promised orders (SO-1011) are judged on the commitment in force at delivery, which flatters performance where re-promising is routine.",
    "Order grain only. Delivery-grain aggregation returns 88.52% instead of 87.27% on this dataset by double-counting split shipments.",
    "Make-to-stock orders (SO-1015) have no production order, so manufacturing phase attribution is legitimately absent for them rather than zero.",
    "A single order-grain fact table. Multi-fact conformed-dimension queries are out of scope for this reference implementation."
  ],

  "governance_note": "These fields make personal data discoverable and support India DPDP readiness work - data-subject categories, processing purpose, legal basis, retention, and declared masking. This catalog entry DOES NOT ENFORCE any of them. It is not a substitute for enforcement: access control, consent management, retention automation, and masking all live in the platform layer, and a field declared here is a statement of intent until a control implements it. Treat a populated governance block as a discovery aid and an audit trail of decisions, never as evidence that a control is operating.",

  "as_of_note": "certified_on carries the __AS_OF_DATE__ token rather than a literal date.

```

```
src/semantic/constants.py is the single home of the as-of date; substituting it at read time is what  
stops this artifact from silently expiring."  
}
```